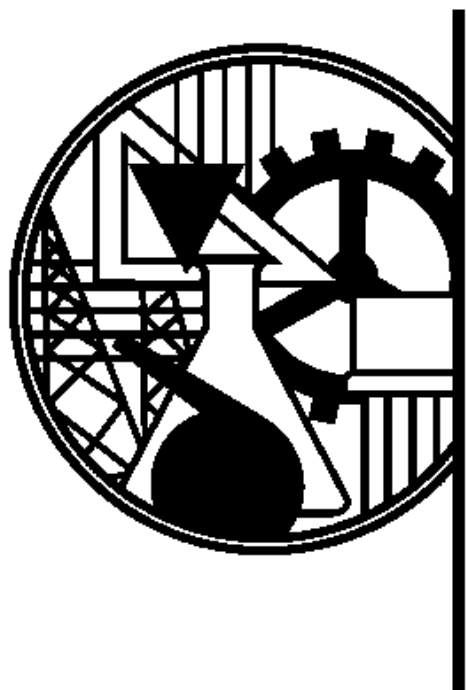




ISEL

INSTITUTO SUPERIOR DE
ENGENHARIA DE LISBOA

RELATÓRIO

PROJETO FINAL

ENTREGA 2

Licenciatura em Engenharia Informática e Multimédia
Inteligência Artificial para Sistemas Autónomos
Semestre de Verão 2023/24

ÍNDICE

1. INTRODUÇÃO	1
2. ENQUADRAMENTO TEÓRICO	2
2.1. Introdução à Inteligência Artificial.....	2
2.1.1. Principais Paradigmas de IA.....	3
2.1.2. Sistemas Autónomos.....	5
2.2. Introdução à Engenharia de Software	7
2.2.1 Problema da Complexidade	7
2.2.2 Arquitetura de Software	8
2.2.3 Modelação de um Sistema Computacional.....	10
2.3. Arquitetura de Agentes Reativos	12
2.3.1 Mecanismos de Reação	12
2.3.2. Agente com Controlo Reativo	15
2.3.3. Agentes Reativos sem Memória	16
2.3.4. Agentes Reativos com Memória	16
2.3.5. Arquitetura de Subsunção.....	17
2.4. Raciocínio Automático.....	18
2.4.1. Procura em Espaço de Estados	19
2.5. Arquitetura de Agentes Deliberativos	22
2.5.1 Planeamento Automático com PEE	23
2.5.2. Processos de Decisão Sequencial.....	24
2.5.3. Planeamento Automático com PDM.....	26
2.6. Aprendizagem por Reforço	27
3. PROJETO - PARTE 1	28
3.1. Organização e Arquitetura do Projeto	28
3.2. Implementação.....	33
4. PROJETO - PARTE 2	46
4.1. Organização e Arquitetura do Projeto	47
4.2. Implementação.....	52
5. PROJETO - PARTE 3	63
5.1. Organização e Arquitetura do Projeto	64
5.2. Implementação.....	71
6. PROJETO - PARTE 4	85
6.1. Organização e Arquitetura do Projeto	86
6.2. Implementação.....	91
7. REVISÃO DO PROJETO	103
8. CONCLUSÃO	104
9. BIBLIOGRAFIA	105

ÍNDICE DE FIGURAS

Figura 1 - Diagrama das perspetivas	2
Figura 2 - Diagrama representativo do paradigma simbólico	3
Figura 3 - Construção de significado	3
Figura 4 - Modelo geral de um agente inteligente.....	5
Figura 5 - Modelo Reativo.....	6
Figura 6 - Modelo Deliberativo	6
Figura 7 - Modelo Híbrido.....	6
Figura 8 - Diagrama demonstrativo do crescimento da complexidade	7
Figura 9 - Factorização.....	9
Figura 10 - Modelo de Dinâmica	10
Figura 11 - Modelo função de transformação	11
Figura 12 - Associação Estímulo-Resposta.....	12
Figura 13 - Módulo Comportamental	13
Figura 14 - Comportamento Composto	13
Figura 15 - Ações paralelas.....	14
Figura 16 - Prioridade de ações	14
Figura 17 - Hierarquia de ações	14
Figura 18 - Combinação de ações.....	15
Figura 19 - Módulo de controlo reativo	15
Figura 20 - Arquitetura reativa com memória	16
Figura 21 - Exemplo de grafo de espaço de estados com solução	19
Figura 22 - Exemplo de árvore de procura	20
Figura 23 - Modelo da arquitetura deliberativa.....	22
Figura 24 - Modelo do planeamento automático	23
Figura 25 - Arquitetura de um agente deliberativo	23
Figura 26 - Modelo do planeador PDM.....	26
Figura 27 - Organização das pastas e módulos do projeto	28
Figura 28 - Subsistema Ambiente e subsistema Agente.....	29
Figura 29 - Subsistema Ambiente do Jogo.....	29
Figura 30 - Subsistema Personagem.....	30
Figura 31 - Controlo da Personagem.....	30
Figura 32 - Subsistema Jogo	31
Figura 33 - Subsistema Máquina de Estados.....	31
Figura 34 - Controlo da Personagem com Máquina de Estados.....	32
Figura 35 - Interface Ambiente	33
Figura 36 - Interface Evento.....	33
Figura 37 - Interface Comando	34
Figura 38 - Classe Agente.....	34
Figura 39 - Interface Controlo	35
Figura 40 - Classe Percepcao	36
Figura 41 - Classe Accao	36
Figura 42 - Classe Ambiente Jogo	37

Figura 43 - Enumerado Evento Jogo	38
Figura 44 - Enumerado Comando Jogo	39
Figura 45 - Classe Personagem	39
Figura 46 - Classe MáquinaEstados	40
Figura 47 - Classe Estado.....	41
Figura 48 - Classe Transicao.....	42
Figura 49 - Classe ControloPersonagem (Parte 1).....	43
Figura 50 - Classe ControloPersonagem (Parte 2).....	44
Figura 51 - Modelo de interação do Jogo	44
Figura 52 -Classe Jogo.....	45
Figura 53 - Exemplo de uma jogada.....	45
Figura 54 - Hierarquia do comportamento Recolher	46
Figura 55 - Organização das pastas e módulos do projeto	47
Figura 56 - Subsistema Reação	48
Figura 57 - Subsistema Comportamento	48
Figura 58 - Controlo Reativo	49
Figura 59 - Comportamento Explorar.....	49
Figura 60 - Comportamento Aproximar Alvo	50
Figura 61 - Comportamento Evitar Obstáculos.....	50
Figura 62 - Respostas a estímulos.....	51
Figura 63 - Interface Comportamento	52
Figura 64 - Classe Reacao.....	52
Figura 65 - Interface Estimulo.....	53
Figura 66 - Classe Resposta.....	53
Figura 67 - Classe ComportComp.....	54
Figura 68 - Classe Hierarquia	55
Figura 69 - Classe Prioridade	55
Figura 70 - Classe ControloReact	56
Figura 71 - Classe Explorar	56
Figura 72 - Classe AproximarAlvo	57
Figura 73 - Classe AproximarDir.....	57
Figura 74 - Classe EstimuloAlvo.....	58
Figura 75 - Classe EvitarObst	58
Figura 76 - Classe EvitarDir.....	59
Figura 77 - Classe EstimuloObst.....	59
Figura 78 - Classe RespostaMover	60
Figura 79 - Classe RespostaEvitar	60
Figura 80 - Classe Recolher	61
Figura 81 - Classe AgenteReactivo.....	61
Figura 82 - Aplicação Agente Reativo.....	62
Figura 83 - Organização das pastas e módulos do projeto	64
Figura 84 - Modelo de problema	65
Figura 85 - Sistema mecanismo de procura	65
Figura 86 - Classe No do sistema mecanismo de procura	66
Figura 87 - Fronteira de procura não informada.....	66
Figura 88 - Mecanismos de procura não informada.....	67
Figura 89 - Procuras em Profundidade.....	67

Figura 90 - Fronteira de procura com prioridade	68
Figura 91 - Procura Melhor-Primeiro	68
Figura 92 - Procura de Custo Uniforme	69
Figura 93 - Procuras Informadas	69
Figura 94 - Avaliadores de Nós	70
Figura 95 - Classe Problema	71
Figura 96 - Interface Operador	71
Figura 97 - Classe Estado	72
Figura 98 - Classe MecanismoProcura	73
Figura 99 - Classe No	74
Figura 100 - Classe Solucao	75
Figura 101 - Classe PassoSolucao	76
Figura 102 - Classe Fronteira	76
Figura 103 - Classe FronteiraLIFO	77
Figura 104 - Classe FronteiraFIFO	77
Figura 105 - Classe ProcuraProfundidade	77
Figura 106 - Classe ProcuraProfLim	78
Figura 107 - Classe ProcuraProflter	78
Figura 108 - Classe ProcuraGrafo	79
Figura 109 - Classe ProcuraLargura	79
Figura 110 - Classe FronteiraPrioridade	80
Figura 111 - Classe ProcuraMelhorPrim	80
Figura 112 - Classe ProcuraCustoUnif	81
Figura 113 - Interface Avaliador	81
Figura 114 - Classe AvaliadorCustoUnif	81
Figura 115 - Classe ProcuraInformada	82
Figura 116 - Classe ProcuraAA	82
Figura 117 - Classe AvaliadorAA	82
Figura 118 - Classe ProcuraSofrega	83
Figura 119 - Classe AvaliadorSof	83
Figura 120 - Interface Heuristica	83
Figura 121 - Classe AvaliadorHeur	84
Figura 122 - Organização das pastas e módulos do projeto	86
Figura 123 - Sistema Controlo Deliberativo	87
Figura 124 - Estado do Agente	87
Figura 125 - Sistema do Planeamento Automático	88
Figura 126 - Relação ModeloMundo - ModeloPlan	88
Figura 127 - Planeador com base em PEE	89
Figura 128 - Problema de Planeamento	89
Figura 129 - Mecanismo PDM	90
Figura 130 - Planeador com base em PDM	90
Figura 131 - Classe EstadoAgente	91
Figura 132 - Interface ModeloPlan	91
Figura 133 - Classe ModeloMundo	92
Figura 134 - Classe MecDelib	93
Figura 135 - Classe ControloDelib	94
Figura 136 - Classe OperadorMover	95

Figura 137 - Interface Planeador..... 96

Figura 138 - Interface Plano..... 96

Figura 139 - Classe PlaneadorPEE..... 97

Figura 140 - Classe PlanoPEE..... 97

Figura 141 - Classe HeurDist..... 98

Figura 142 - Classe ProblemaPlan..... 98

Figura 143 - Interface ModeloPDM..... 99

Figura 144 - Classe MecUtil..... 99

Figura 145 - Classe PDM..... 100

Figura 146 - Classe ModeloPDMPlan..... 101

Figura 147 - Classe PlanoPDM..... 102

Figura 148 - Classe PlaneadorPDM..... 102

1. INTRODUÇÃO

A convergência entre a **Inteligência Artificial** e a **Engenharia de Software** tem provocado o desenvolvimento de sistemas autónomos cada vez mais sofisticados e eficientes. No âmbito da unidade curricular **Inteligência Artificial para Sistemas Autónomos**, foram abordados os fundamentos teóricos e práticos que promovem essa relação, visando a criação de agentes inteligentes capazes de operar de forma autónoma em ambientes dinâmicos e desafiantes.

Neste relatório, será apresentada uma análise detalhada das metodologias, técnicas e ferramentas utilizadas durante o desenvolvimento do projeto, bem como os resultados obtidos e as lições aprendidas ao longo deste processo.

Os principais tópicos abordados e que serão explorados ao longo deste relatório são:

1. Introdução à Inteligência Artificial
2. Introdução à Engenharia de Software
3. Arquitetura de agentes reativos
4. Raciocínio Automático
5. Arquitetura de agentes deliberativos
6. Parte 1 do projeto
7. Parte 2 do projeto
8. Parte 3 do projeto
9. Parte 4 do projeto

2. ENQUADRAMENTO TEÓRICO

Neste capítulo, é feito o enquadramento teórico dos temas que foram lecionados durante as aulas da unidade curricular de IASA e que serviram de suporte para o projeto final.

2.1. Introdução à Inteligência Artificial

A **Inteligência Artificial** é um campo científico que se dedica ao desenvolvimento de sistemas computacionais capazes de exibir comportamento inteligente. Esta disciplina adota duas perspetivas principais:

1. **Analítica:** Utilizando uma abordagem empírica, baseada na prática, experiências e observações, procura desenvolver modelos e teorias para explicar os fenómenos observados. O objetivo é reproduzir esses fenómenos em sistemas artificiais, ou seja, em dispositivos criados especificamente para esse fim.

2. **Sintética:** Com base nos modelos e teorias estabelecidos, pretende-se o desenvolvimento de sistemas que sejam capazes de manifestar características e comportamentos associados ao conceito de inteligência. Esta abordagem visa a criação de sistemas que possam agir de forma autónoma e adaptativa, emulando a inteligência humana em diversas tarefas e contextos.

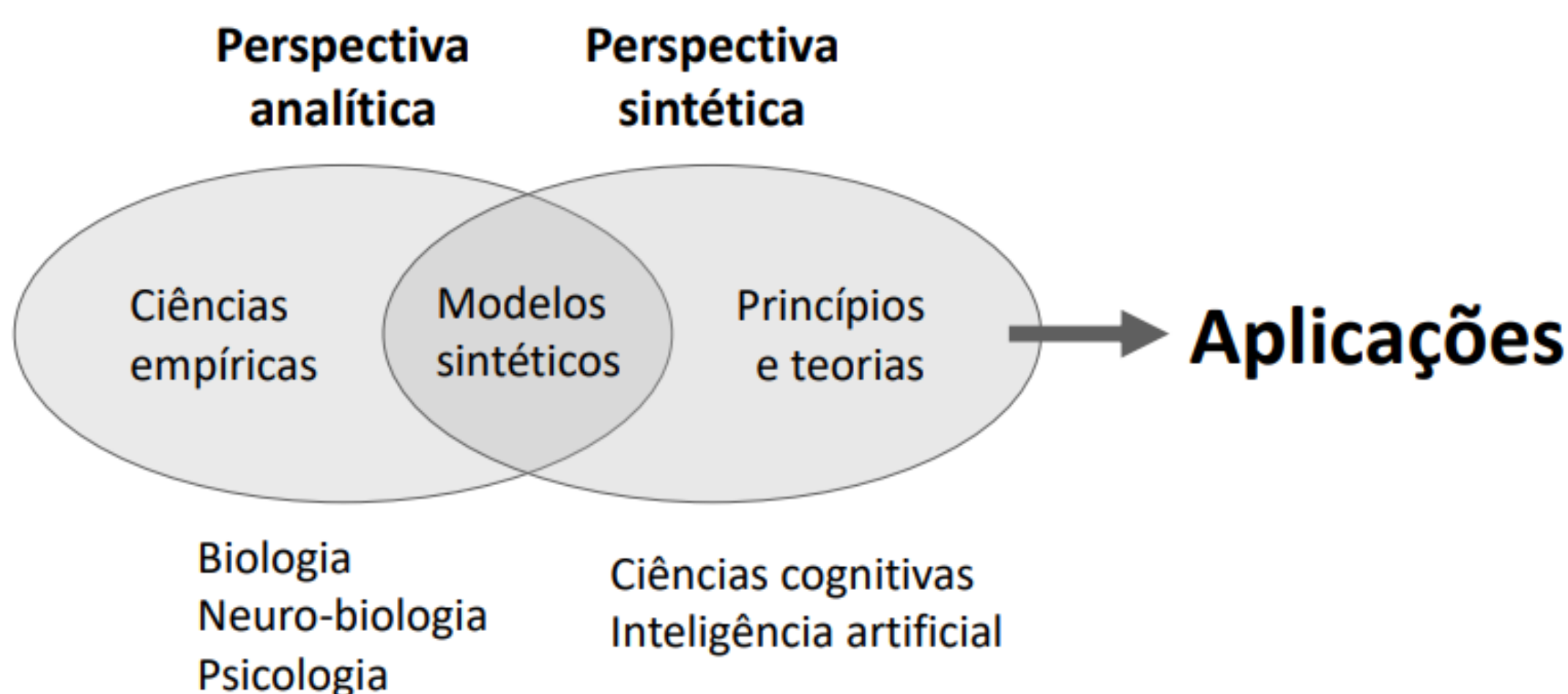


Figura 1 - Diagrama das perspetivas

2.1.1. Principais Paradigmas de IA

Um paradigma é um modelo ou padrão de pensamento amplamente aceite numa área específica, influenciando a forma como problemas são abordados e soluções são encontradas. Existem três principais paradigmas na área da IA que são o **paradigma simbólico**, **paradigma conexionista** e **paradigma comportamental**.

O **paradigma simbólico** define que a inteligência é resultante da ação de processos computacionais sobre estruturas simbólicas, em que os símbolos são centrais no processamento de informação e na representação de conceitos relacionados com o domínio de um problema. Destaca-se a **Hipótese do Sistema de Símbolos Físicos**, proposta por Alan Newell e Herbert Simon em 1976, que sugere que um sistema de símbolos físicos tem os meios necessários e suficientes para demonstrar inteligência. Assim, a computação simbólica tornou-se um dos suportes principais da IA.

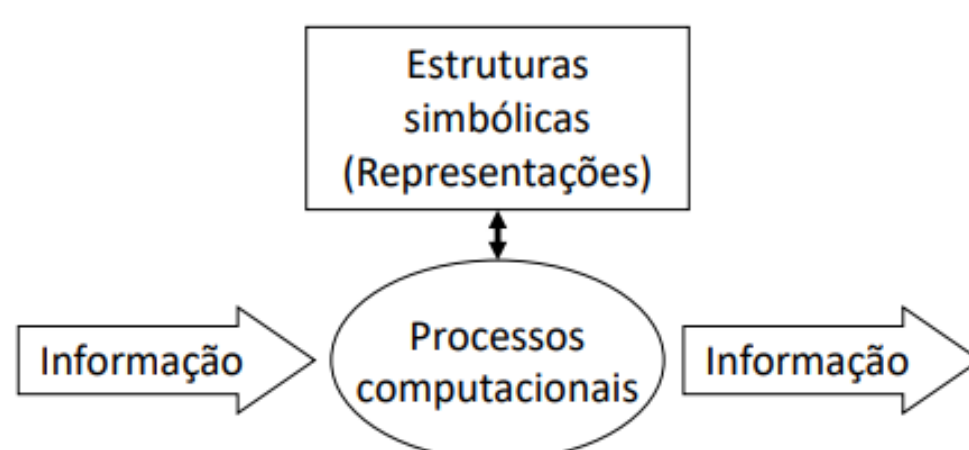


Figura 2 - Diagrama representativo do paradigma simbólico

As **representações simbólicas** não têm significado intrínseco, o significado resulta das relações entre estruturas de símbolos, bem como das relações dessas estruturas com as entradas e saídas do sistema. Esta relação é designada por **ancoragem simbólica**. Num sistema artificial, as relações que formam o significado podem ser representadas através de um modelo organizado num grafo de conceitos inter-relacionados, designado por **rede semântica**. Estas permitem criar uma representação do conhecimento referente ao um determinado domínio. Estas representações são designadas por **ontologias**. Ser capaz de representar significado e conhecimento é um dos passos importantes para artificialmente mostrar inteligência.

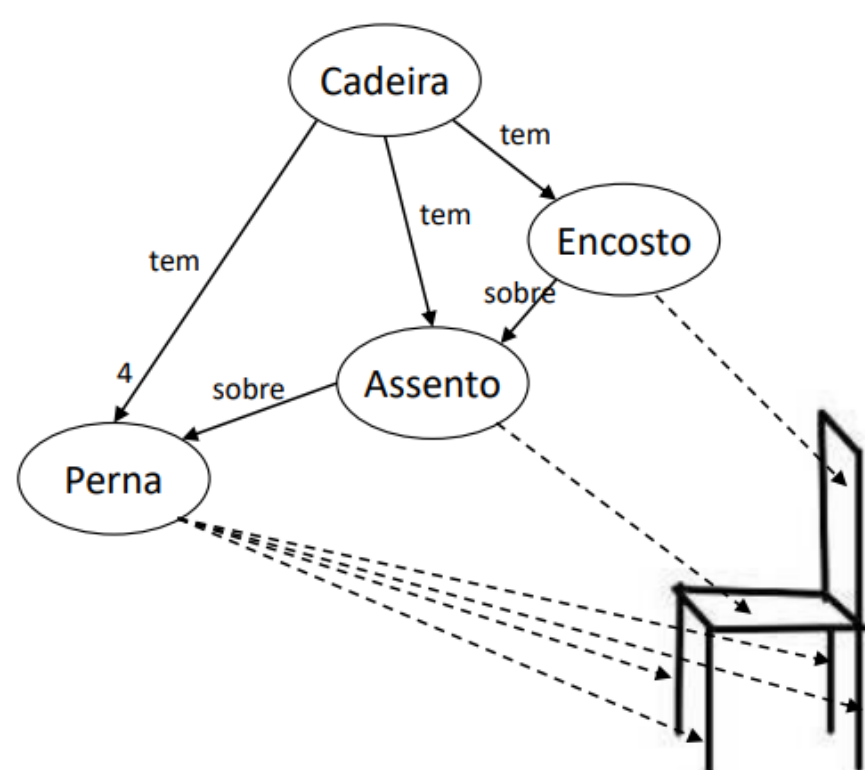


Figura 3 - Construção de significado

○ **paradigma conexionista** define que a inteligência é uma propriedade proveniente das interações de unidades elementares de processamento interligadas entre si, designadas **neurónios**, sendo a informação mantida e processada nessas redes em vez de símbolos. Este paradigma foi inspirado na biologia do sistema nervoso dos animais.

○ **paradigma comportamental** define que a inteligência resulta do comportamento individual e conjunto de múltiplos sistemas. O processamento de informação é baseado em relações entre estímulos e respostas, modeladas sob a forma de reações e comportamentos (conjunto de reações).

2.1.2. Sistemas Autónomos

A **autonomia** é a capacidade de um sistema atuar por si próprio, de modo independente de outros sistemas. Um sistema autónomo não é necessariamente inteligente, mas **para ser inteligente tem de ser autónomo**.

Um **sistema autónomo inteligente** opera num ciclo realimentado **percepção-processamento-ação**, através do qual é capaz de alcançar a sua finalidade.

2.1.2.1 Agente Inteligente

A representação computacional de um sistema autónomo inteligente designa-se **agente inteligente**. O modelo geral de um agente inteligente implementa o ciclo percepção-processamento-ação.

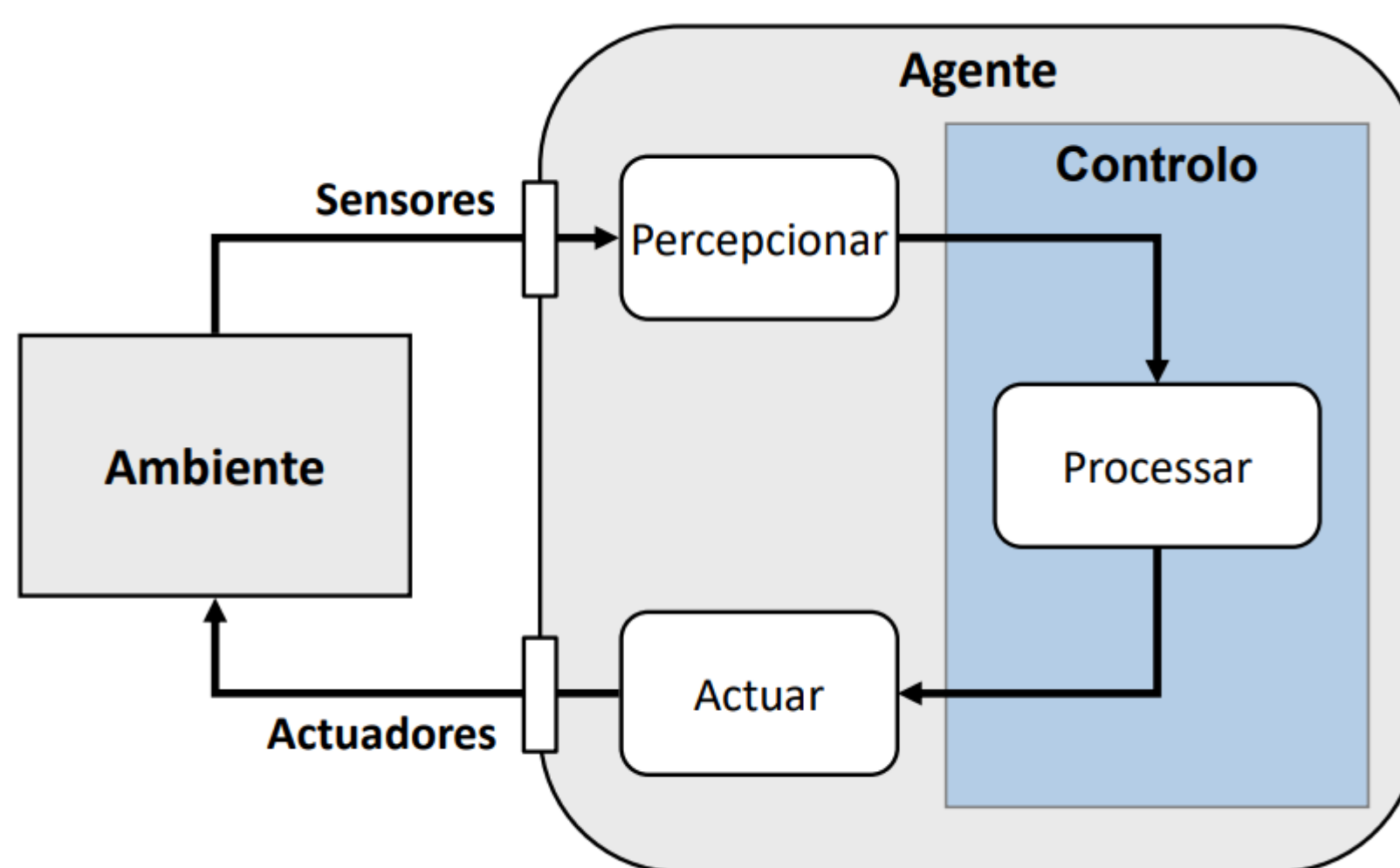


Figura 4 - Modelo geral de um agente inteligente

O ambiente onde se encontra o agente pode ser real, com sensores e atuadores físicos, ou virtual. Um ambiente pode ser: discreto ou contínuo; determinístico ou estocástico; estático ou dinâmico; totalmente ou parcialmente observável; conter um único agente ou múltiplos.

Subjacentes ao conceito de inteligência estão duas vertentes, a **cognição** e a **racionalidade**, que devem também ser duas principais características de um agente inteligente. A cognição é expressa através da capacidade do agente de realizar a ação adequada dadas as condições do ambiente. A racionalidade expressa-se na capacidade de decidir no sentido de conseguir o melhor resultado possível perante o objetivo que o agente pretende atingir.

2.1.2.2 Arquiteturas de Agente

Para realizar as várias características que um agente deve apresentar, este assume uma organização interna específica consoante o paradigma sob o qual o agente é implementado. A esta organização interna chama-se **arquitetura de agente** e existem as seguintes:

- **Modelo Reativo**

Neste modelo, que segue o paradigma comportamental, o comportamento do sistema é gerado de forma reativa, com base em associações entre estímulos (referentes às perceções) e respostas (referentes às ações).

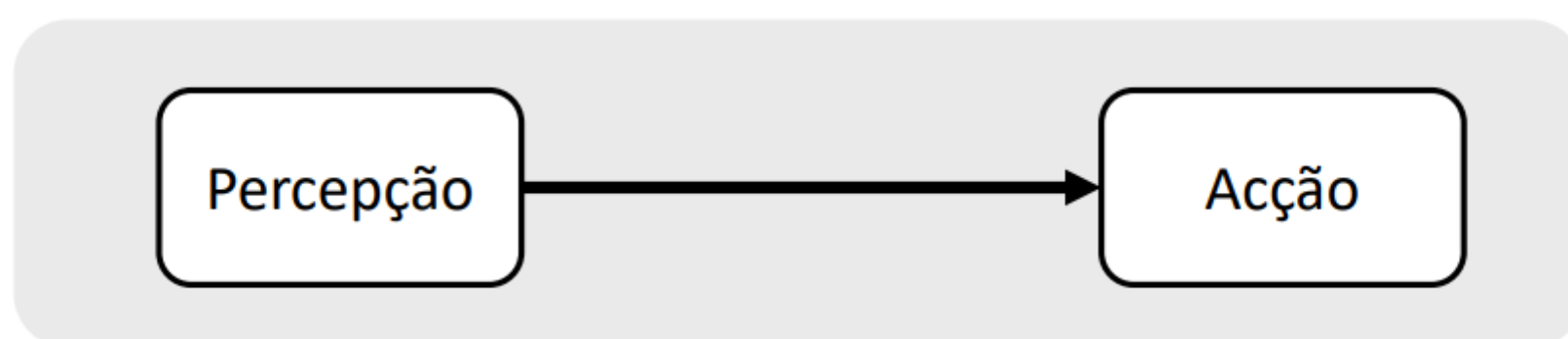


Figura 5 - Modelo Reativo

- **Modelo Deliberativo**

Neste modelo, que segue o paradigma simbólico, o comportamento do sistema é gerado com base em mecanismos de deliberação (raciocínio e tomada de decisão), utilizando representações internas que incluem a representação explícita de objetivos.

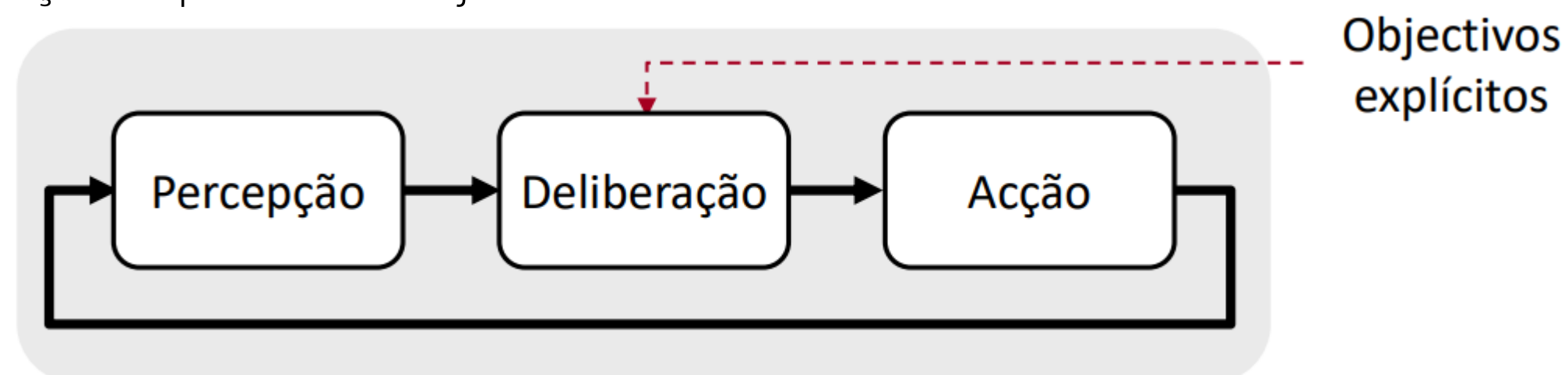


Figura 6 - Modelo Deliberativo

- **Modelo Híbrido**

Neste modelo, o comportamento do sistema é gerado com base em processamento que integra as vertentes reativa e deliberativa.

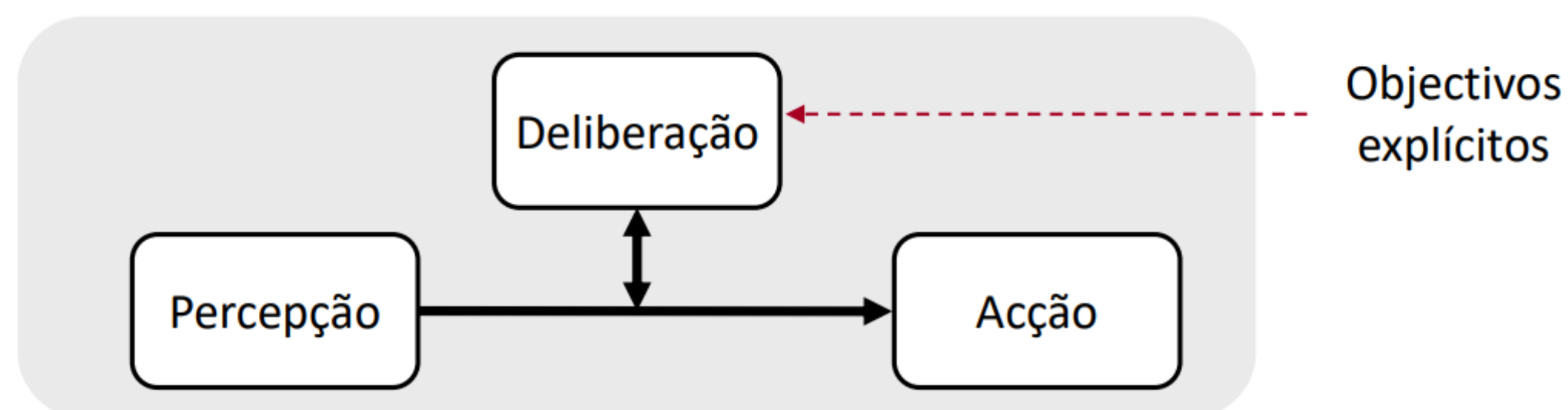


Figura 7 - Modelo Híbrido

2.2. Introdução à Engenharia de Software

A **engenharia de software** é uma área de engenharia orientada para a especificação, desenvolvimento e manutenção de software, que tem por objetivo o desenvolvimento, operação e manutenção de software de modo **sistemático**, com a capacidade de realizar o desenvolvimento de software de forma organizada e previsível, e **quantificável**, utilizando métodos e processos adequados para garantir a qualidade e o desempenho do software produzido.

O **desenvolvimento** de sistemas com base em **inteligência artificial** requer métodos adequados de engenharia de software, nomeadamente, modelação de sistemas e linguagens de modelação, como é o caso da linguagem UML. Tem associado **dois principais problemas** de relevância crescente: a **mudança**, expressa no ritmo crescente a que o software necessita de ser produzido ou modificado, e a **complexidade**, que será explorada em detalhe já de seguida.

2.2.1 Problema da Complexidade

A complexidade de um sistema expressa-se na crescente **dificuldade** de **desenvolvimento**, **operação** e **manutenção** de software, na crescente sofisticação do software produzido, bem como na quantidade crescente de recursos envolvidos, nomeadamente, em termos de capacidade de processamento e de memória utilizada. Na prática, essa complexidade traduz-se na dificuldade e **esforço crescente** para a conceção e implementação de software pois, um sistema com duas vezes mais partes é muito mais que duas vezes mais complexo, ou seja, o crescimento da complexidade é **exponencial** (explosão combinatória). Existem dois tipos de complexidade associados à organização de um sistema, que são:

- **Complexidade Organizada:** Resulta de padrões de inter-relacionamento entre partes correlacionáveis no espaço e no tempo.
- **Complexidade Desorganizada:** Resulta do número e heterogeneidade das partes de um sistema. As partes podem interagir entre si, mas a interação é irregular.

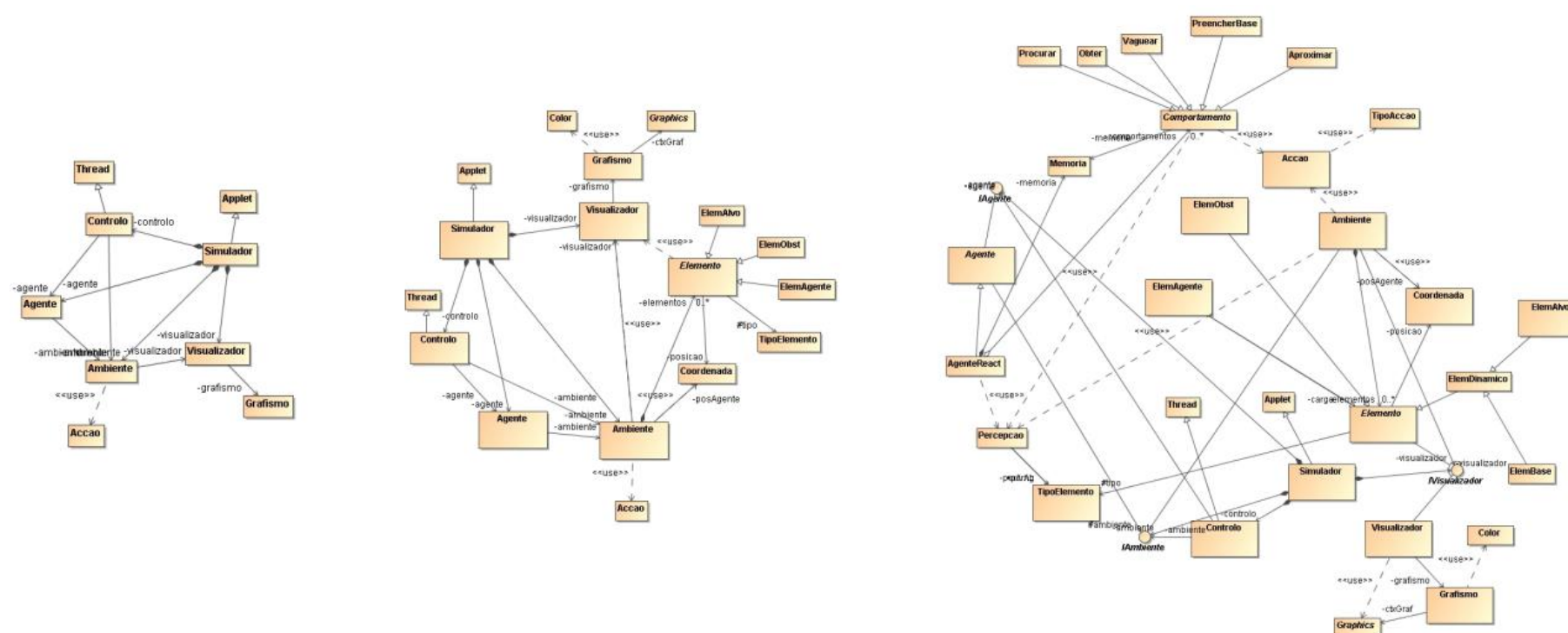


Figura 8 - Diagrama demonstrativo do crescimento da complexidade

2.2.2 Arquitetura de Software

A arquitetura de software tem por objetivo abordar o problema da complexidade com base numa organização adequada do software a produzir. Tem por base três vertentes principais: **métricas**, **princípios** e **padrões**.

2.2.2.1 Métricas de Arquitetura

As métricas de arquitetura definem medidas de quantificação da arquitetura de um software indicadores da qualidade dessa arquitetura.

- **Acoplamento:** O conceito de acoplamento refere-se ao **grau de dependência** entre diferentes partes de um sistema, sendo maior quanto maior for essa dependência. O objetivo é produzir software com o menor acoplamento possível, de modo a reduzir a complexidade e a facilitar a sua manutenção e evolução.
- **Coesão:** O conceito de coesão refere-se à forma como os elementos de um sistema estão **agrupados de forma coerente** entre si, será maior quanto mais relacionados entre si forem os elementos agrupados. O objetivo é criar software que seja claro e fácil de entender, manter e evoluir, facilitando a reutilização e aumentando assim a eficiência do desenvolvimento.
- **Simplicidade:** Nível de facilidade de compreensão/comunicação da arquitetura
- **Adaptabilidade:** Nível de facilidade de alteração da arquitetura para incorporação de novos requisitos ou de alterações nos requisitos previamente definidos.

2.2.2.2 Princípios de Arquitetura

Definem meios orientadores da conceção de arquitetura de software, no sentido de garantir a qualidade da arquitetura produzida, nomeadamente, no que se refere à minimização do acoplamento, à maximização da coesão e à gestão da complexidade.

- **Abstração:** A abstração é uma ferramenta base para lidar com a complexidade. Consiste em **identificar** as características comuns a diferentes partes, **realçar** o que é essencial, **simplificar** reduzindo a complexidade da descrição das partes, **focar** nos aspetos mais relevantes para o contexto da descrição e criar **modelos** que funcionam como descrições abstratas do sistema.
- **Modularização:** O conceito de modularização refere-se à capacidade de organização de um sistema em partes coesas, ou **módulos**, que podem ser interligados entre si para produzir a função do sistema, está relacionado com dois aspetos principais, **decomposição** e **encapsulamento**.
 - **Decomposição:** Consiste na decomposição do sistema em partes coesas de forma a sistematizar interações e lidar com a explosão combinatória. A principal forma de decomposição é a **factorização**.
 - **Factorização:** Refere-se à decomposição das partes do sistema de modo a eliminar a **redundância** (partes repetidas). Relaciona-se com o conceito matemático de decomposição de uma expressão em fatores. Em suma, as partes repetidas são eliminadas, as partes mantidas são partilhadas. Os principais mecanismos de factorização são a **herança** (acoplamento baixo) e **delegação** (acoplamento alto).

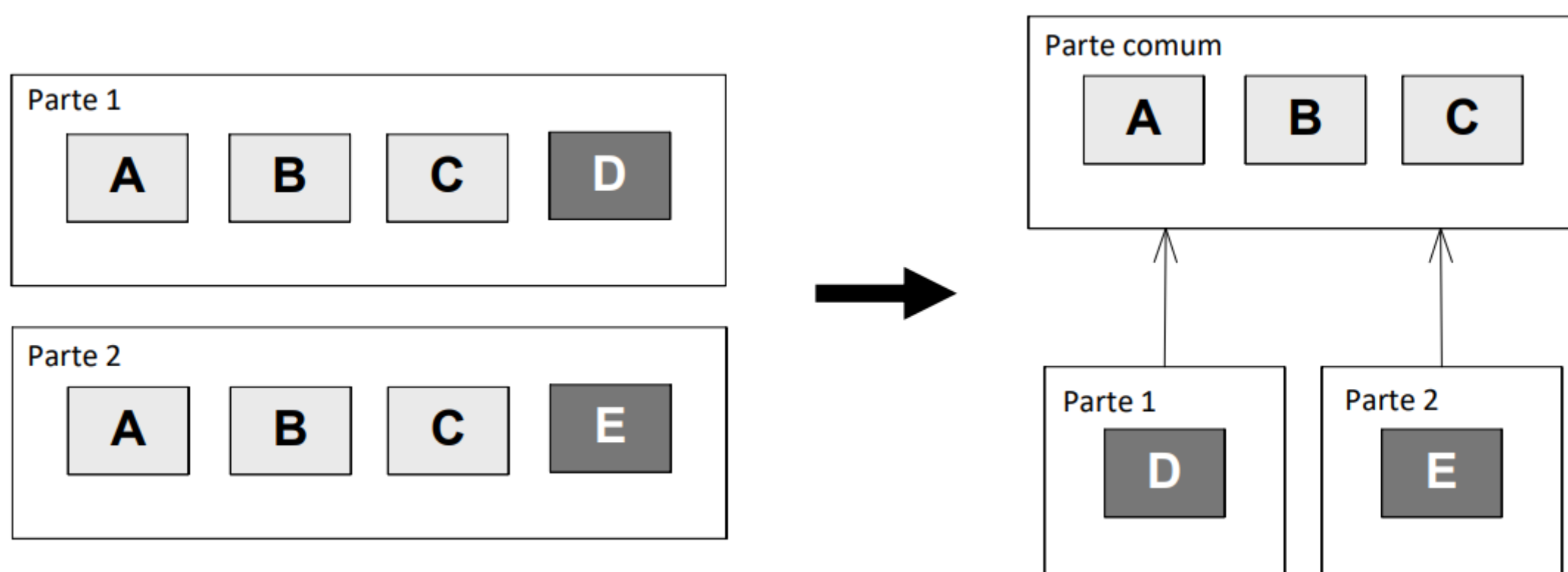


Figura 9 - Factorização

- **Encapsulamento:** Consiste no isolamento dos detalhes internos das partes de um sistema em relação ao exterior para reduzir dependências, relacionar estrutura e função no contexto de uma parte e acesso exclusivo através de **interfaces** (contratos funcionais para interação com o exterior) disponibilizadas.

2.2.3 Modelação de um Sistema Computacional

A **dinâmica** de um sistema refere-se aos diferentes estados que o sistema pode assumir e como esses estados evoluem ao longo do tempo para produzir o **comportamento** do sistema. O comportamento do sistema é a maneira como ele responde às entradas do ambiente externo, gerando saídas com base nessas entradas e no estado interno do sistema. Em termos abstratos, um sistema computacional pode ser caracterizado por meio de uma **representação do seu estado interno** em cada momento e por uma **função de transformação** que determina as **saídas** e o **próximo estado** do sistema com base nas **entradas** e no **estado atual**.

2.2.3.1 Modelo de Dinâmica de um Sistema

A dinâmica pode ser expressa como uma função de transformação que, perante o estado atual e as entradas atuais, produz o estado seguinte e as saídas seguintes. Esta caracterização de um sistema computacional é independente da forma concreta como este possa ser implementado em termos físicos.

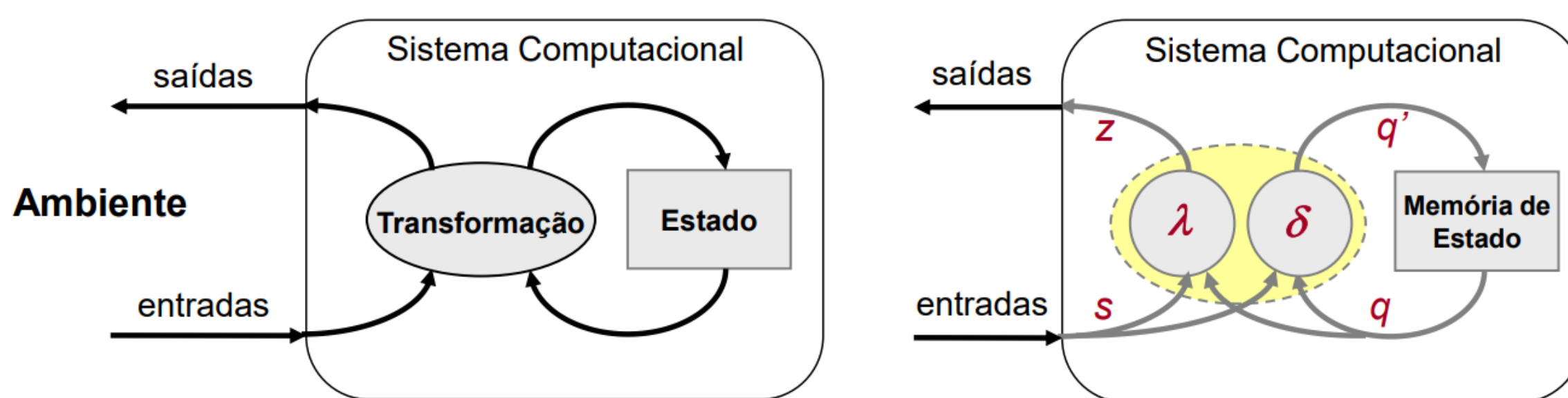


Figura 10 - Modelo de Dinâmica

2.2.3.2 Modelo Formal de Computação

Neste modelo, as entradas e saídas são abstraídas em termos dos conjuntos de símbolos que nelas podem ocorrer, esses conjuntos de símbolos são designados alfabetos. Consideremos um **alfabeto de entrada** Σ , um **alfabeto de saída** Z e o **estado interno** do sistema descrito em termos de um conjunto de estados possíveis Q . A **função de transformação** do sistema descrita com base em duas funções distintas δ e λ .

- Função de transição de estado:
 - $\delta : Q \times \Sigma \rightarrow Q$
- Função de saída:
 - $\lambda : Q \times \Sigma \rightarrow Z$

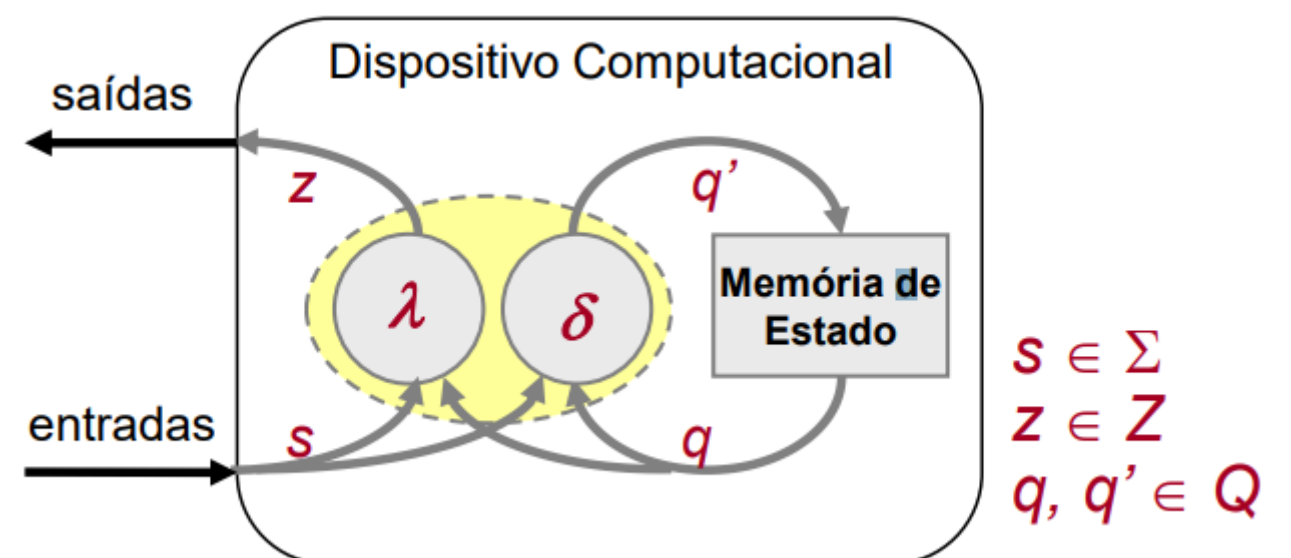


Figura 11 - Modelo função de transformação

Este tipo de modelo descreve um mecanismo designado **Máquina de Estados**. Existem duas formulações distintas da função de saída λ :

- **Máquinas de Mealy**, nas quais a função de saída depende das entradas:
 - $\lambda : Q \times \Sigma \rightarrow Z$
- **Máquinas de Moore**, onde a função de saída não depende das entradas:
 - $\lambda : Q \rightarrow Z$

2.3. Arquitetura de Agentes Reativos

Este tipo de arquitetura de agente cumpre o **Modelo Reativo** estudado anteriormente onde o comportamento do sistema é gerado de forma **reativa**, com base em associações entre **estímulos** (referentes às percepções) e **respostas** (referentes às ações). Os objetivos são implícitos, ou seja, não estão explicitamente representados, estão então implícitos nas associações estímulo-resposta que definem o comportamento do agente. Existe um **acoplamento forte com o ambiente**, em alguns casos direto, através de associações entre os estímulos derivados das percepções e a respostas que produzem ações.

2.3.1 Mecanismos de Reação

Nas **Associações Estímulo-Resposta** as ações são diretamente ativadas em função das percepções. Não são utilizadas representações internas do mundo, as respostas são rápidas quando há alterações no ambiente e as respostas são fixas e predefinidas aos estímulos do ambiente.

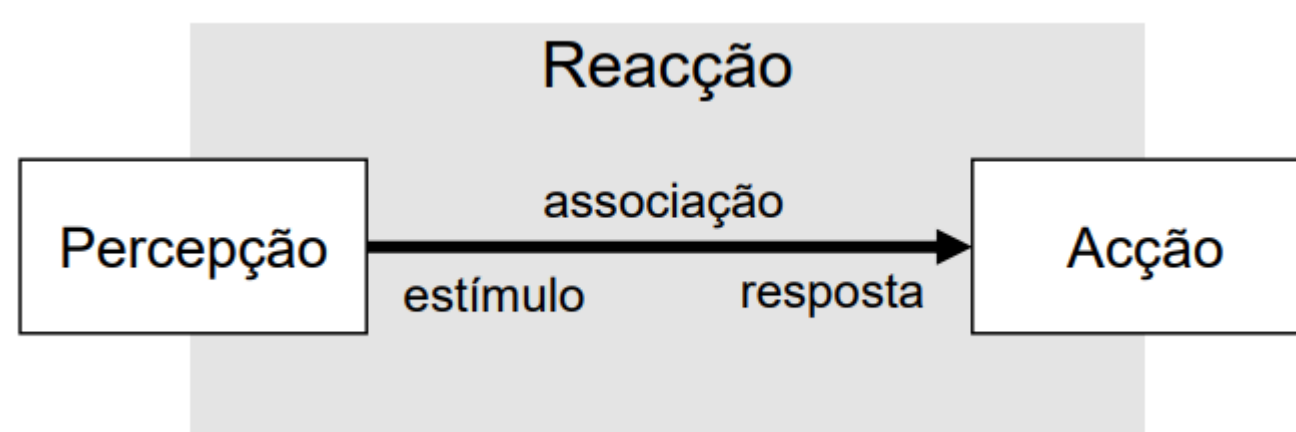


Figura 12 - Associação Estímulo-Resposta

No caso geral, as reações podem envolver **processamento mais complexo**, incluindo a **manutenção de estado interno** com base em mecanismos de memória. Nessa situação, os vários elementos envolvidos numa reação **devem ser modularizados**, de forma a poderem ser organizados de forma versátil e a encapsular a complexidade necessária à sua função. Em particular, devem ser definidos os seguintes elementos:

- **Estímulo:** Define informação ativadora de uma reação.
- **Resposta:** Define uma resposta a estímulos, em termos de ação a realizar e da respetiva prioridade.
- **Reação:** Módulo que associa estímulos a respostas.

2.3.1.1 Comportamentos

Um agente reativo é composto por **conjuntos de reações**, as quais devem ser organizadas de forma modular em módulos comportamentais designados **comportamentos**, de modo a reduzir a complexidade e a facilitar o desenvolvimento e manutenção do agente. Um comportamento é um **conjunto de reações** relacionadas entre si no sentido de produzirem um resultado específico, por exemplo, evitar um obstáculo.

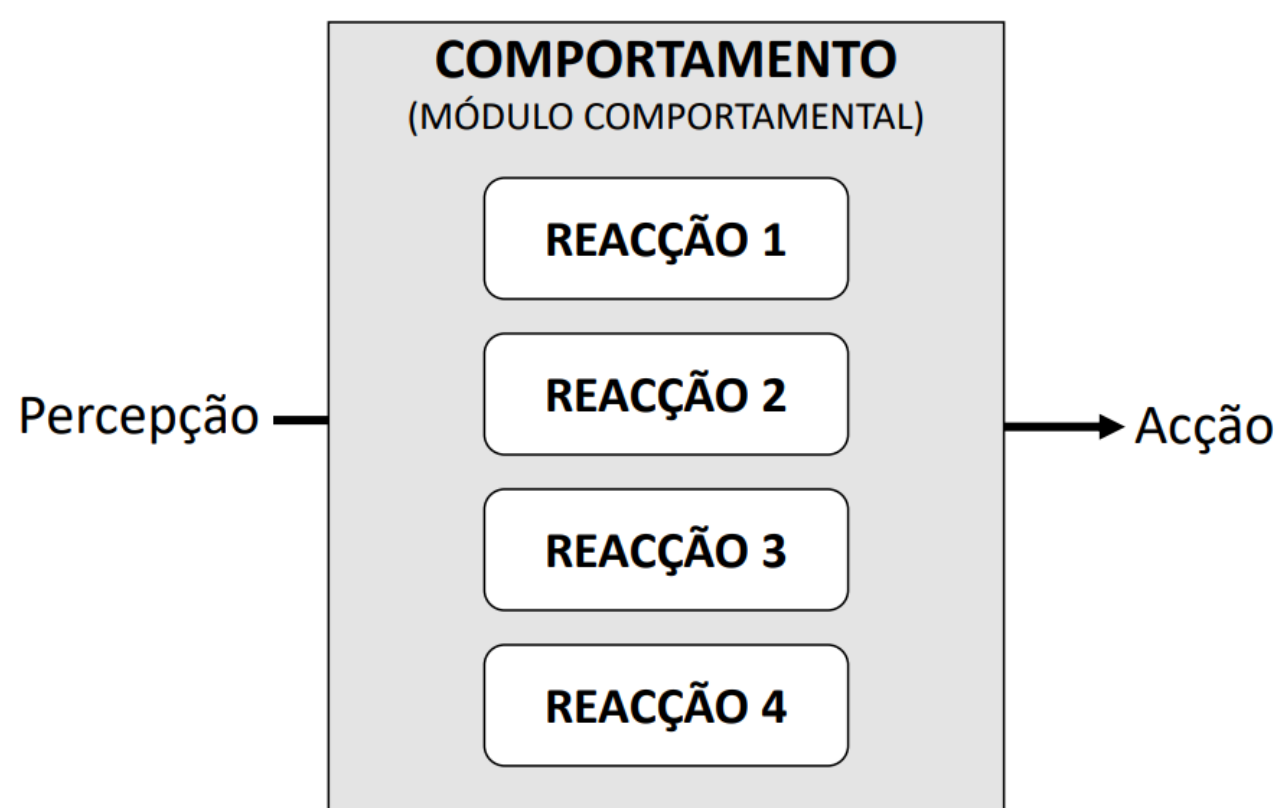


Figura 13 - Módulo Comportamental

Um **comportamento** realiza de forma modular e coesa o **encapsulamento das reações internas**, contribuindo assim para **reduzir o acoplamento e a complexidade** da arquitetura de um agente reativo.

Um **comportamento composto** agrega conjuntos de comportamentos. Requer um mecanismo de **selecção de acção** para determinar a acção a realizar em função das respostas dos vários comportamentos internos. Existem então **dois** tipos de comportamento: a **reação** (comportamento simples) e o **comportamento composto**.

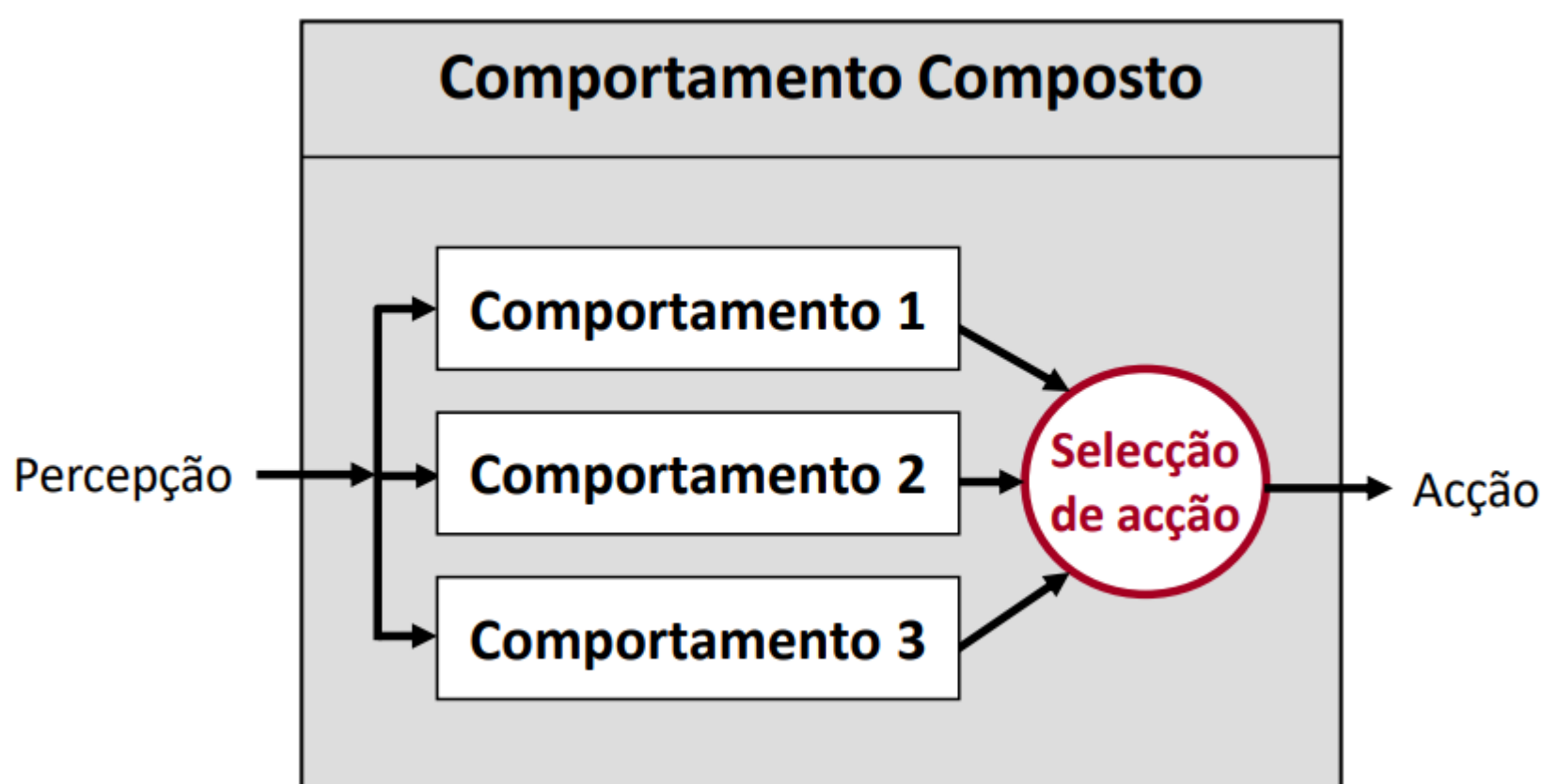


Figura 14 - Comportamento Composto

2.3.1.2 Mecanismos de Seleção de Ação

Num comportamento composto, uma perceção pode potencialmente ativar múltiplas reações, as quais geram diferentes ações.

- **Ações Paralelas:** As ações que não interferem entre si podem ser executadas em paralelo. É viável se a infraestrutura o suportar, por exemplo, se existirem múltiplos atuadores distintos.

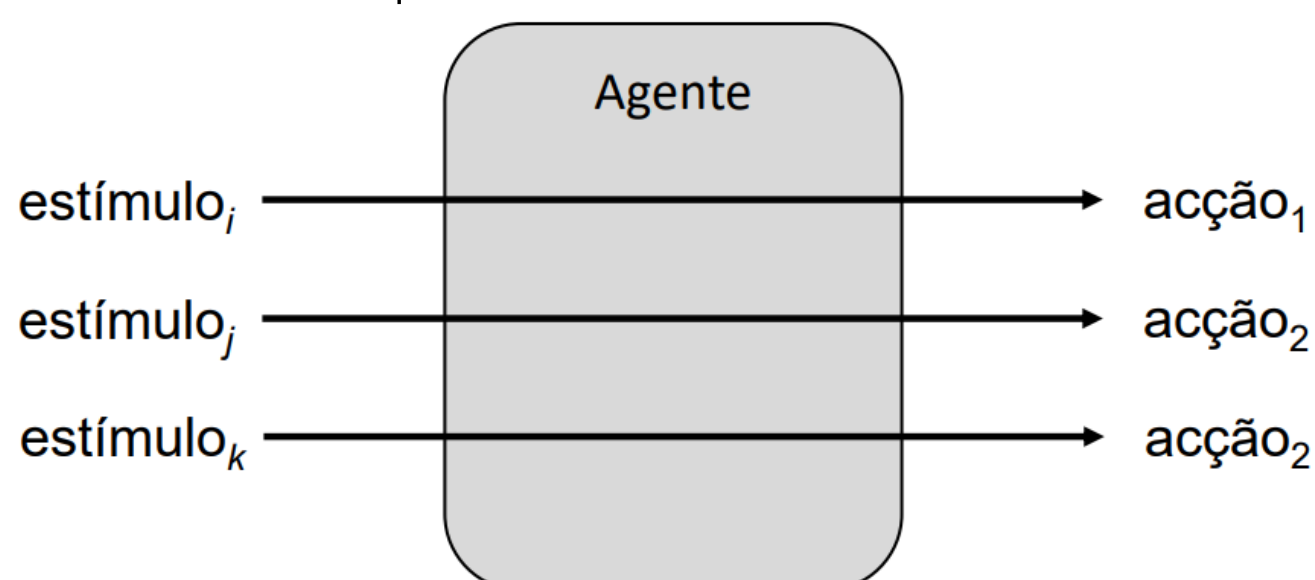


Figura 15 - Ações paralelas

- **Prioridade de Ações:** As ações são selecionadas de acordo com uma prioridade associada que varia ao longo da execução.

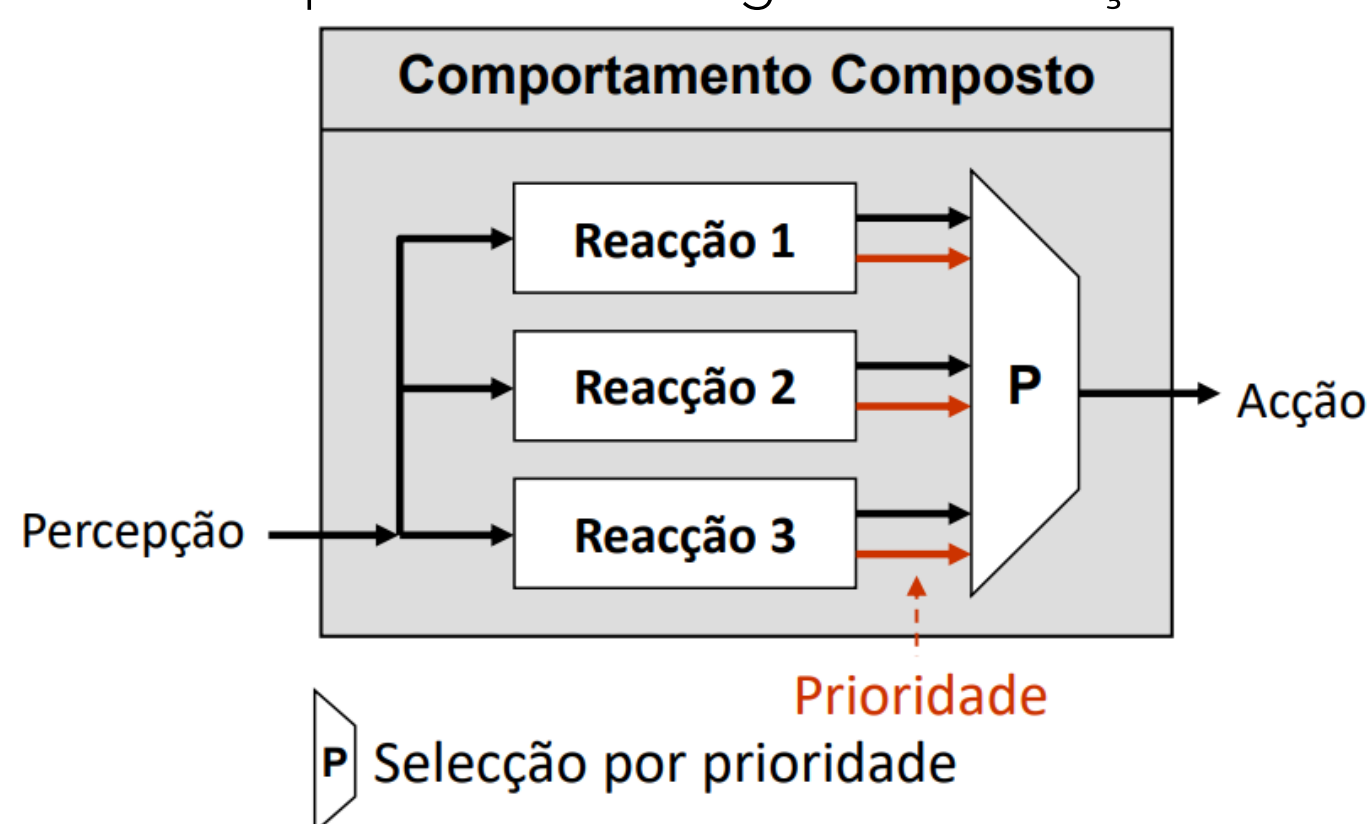


Figura 16 - Prioridade de ações

- **Hierarquia:** Os comportamentos estão organizados numa hierarquia fixa de subsunção (supressão e substituição).

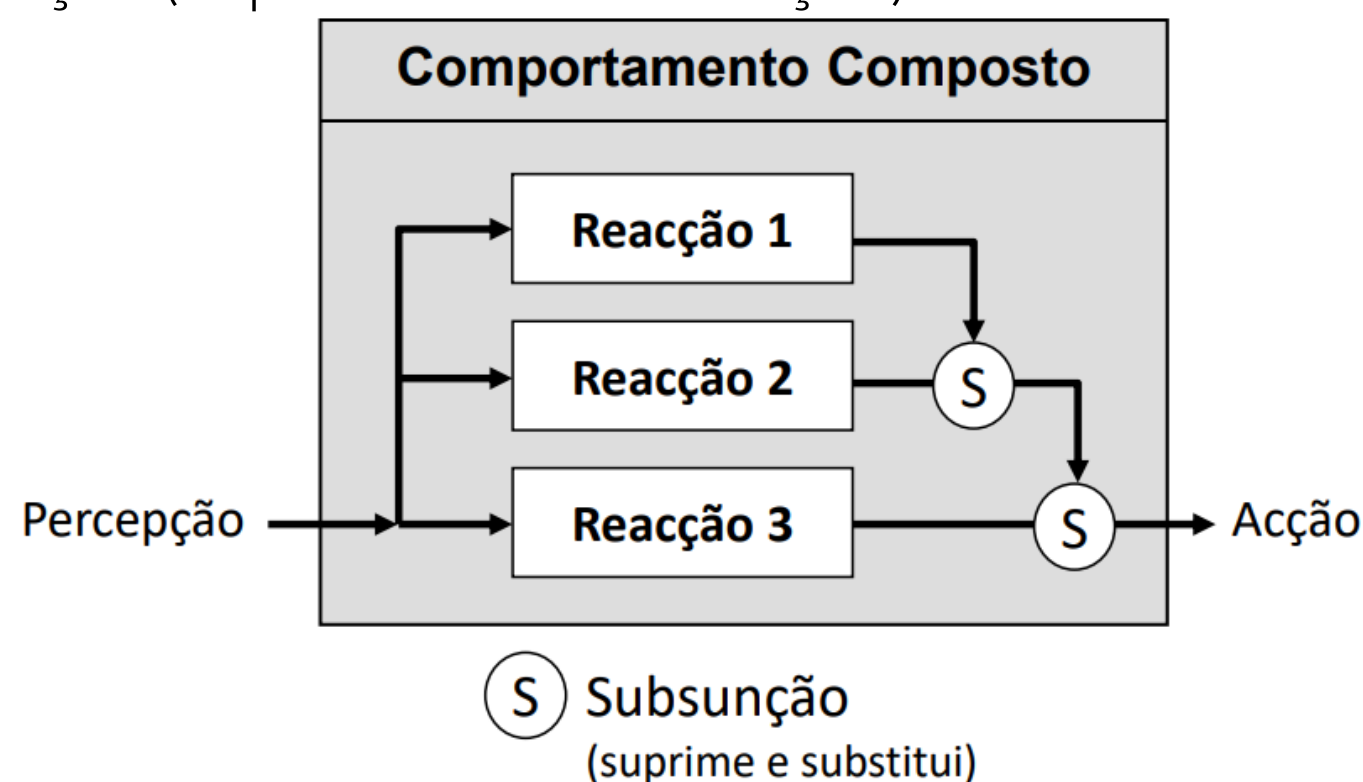


Figura 17 - Hierarquia de ações

- **Combinação:** As respostas são combinadas numa única resposta por composição (por exemplo, soma vetorial).

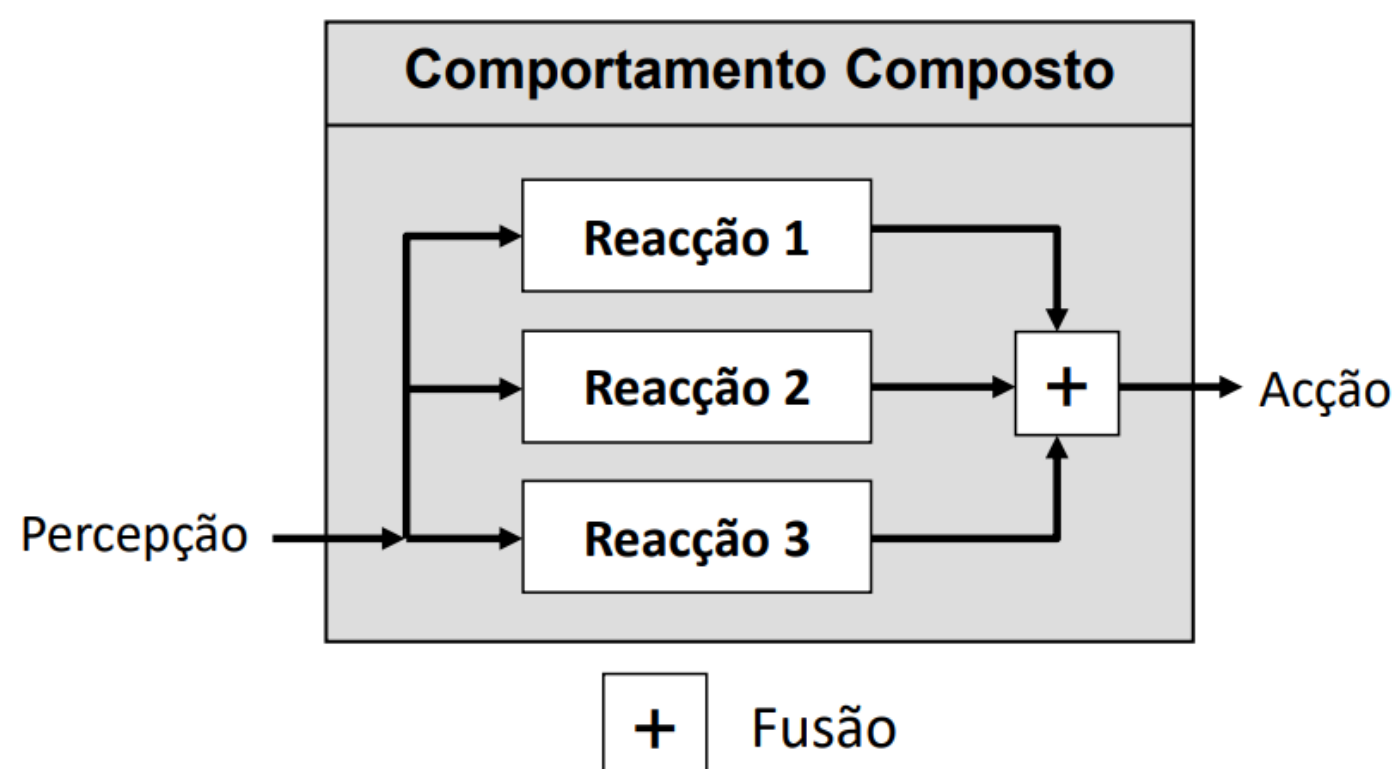


Figura 18 - Combinação de ações

2.3.2. Agente com Controlo Reativo

Na concretização de uma arquitetura de agente, o **processamento interno** que relaciona perceções com ações, pode ser modularizado com base num **módulo de controlo**. No caso da arquitetura de um agente reativo, esse controlo será um **controlo reativo**, em que o processar das perceções é realizado com base num módulo comportamental, também designado **comportamento**, o qual representa o comportamento geral do agente que pode ser constituído por diferentes sub-comportamentos.

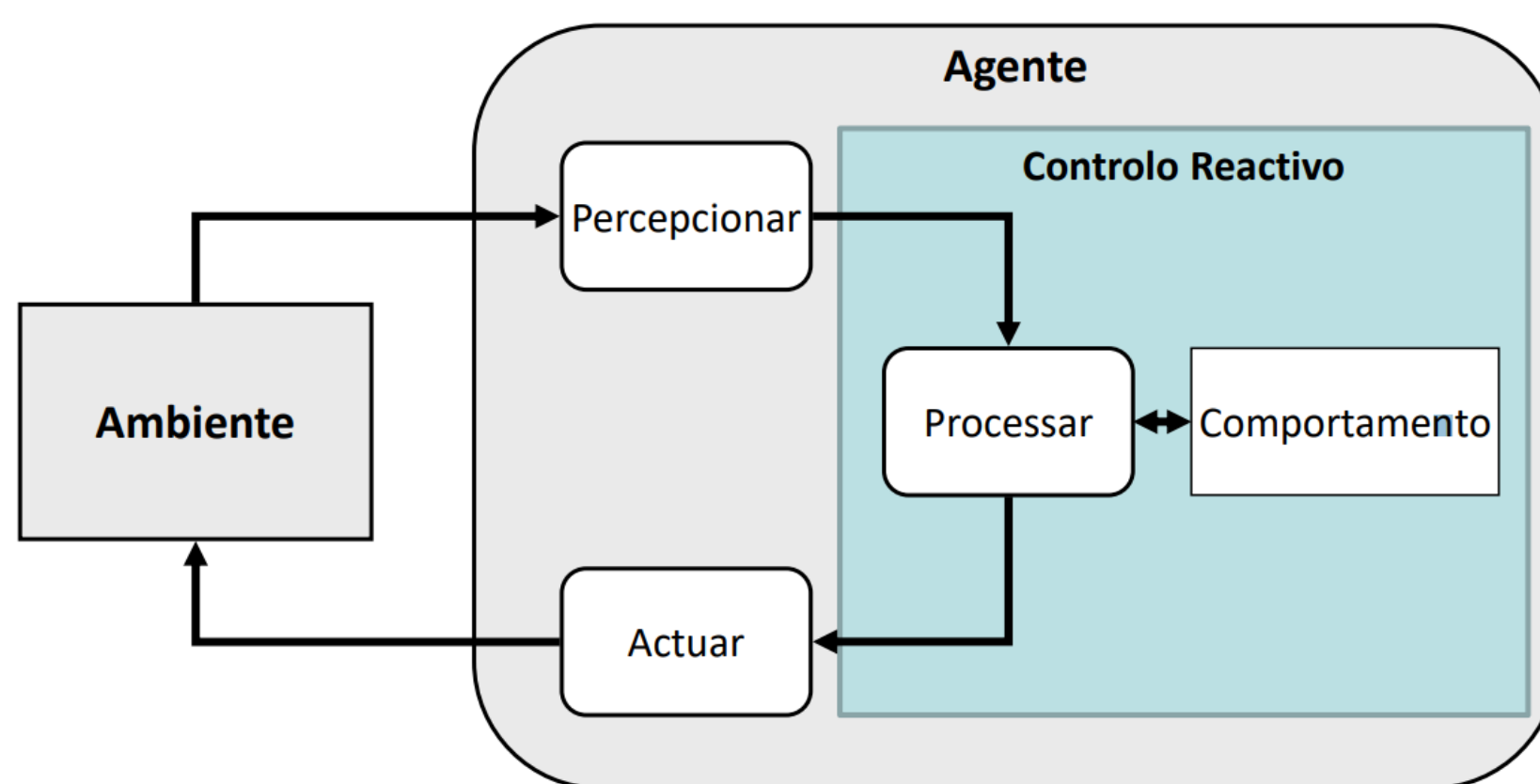


Figura 19 - Módulo de controlo reativo

2.3.3. Agentes Reativos sem Memória

Os agentes reativos sem memória têm várias desvantagens como, por exemplo, forte dependência das capacidades sensoriais e das características do ambiente, não evitam configurações já exploradas e podem ficar presos em comportamentos cíclicos. Existe, portanto, a necessidade de manutenção de estado.

2.3.4. Agentes Reativos com Memória

Numa arquitetura reativa com memória, as reações dependem não só das percepções, mas também da **memória de percepções anteriores** (ou de informação delas derivada) para gerar as ações. Para esse efeito é necessário **manter internamente memória**, a qual é atualizada a partir das percepções e das reações ativadas, influenciando essas mesmas reações, bem como as ações geradas.

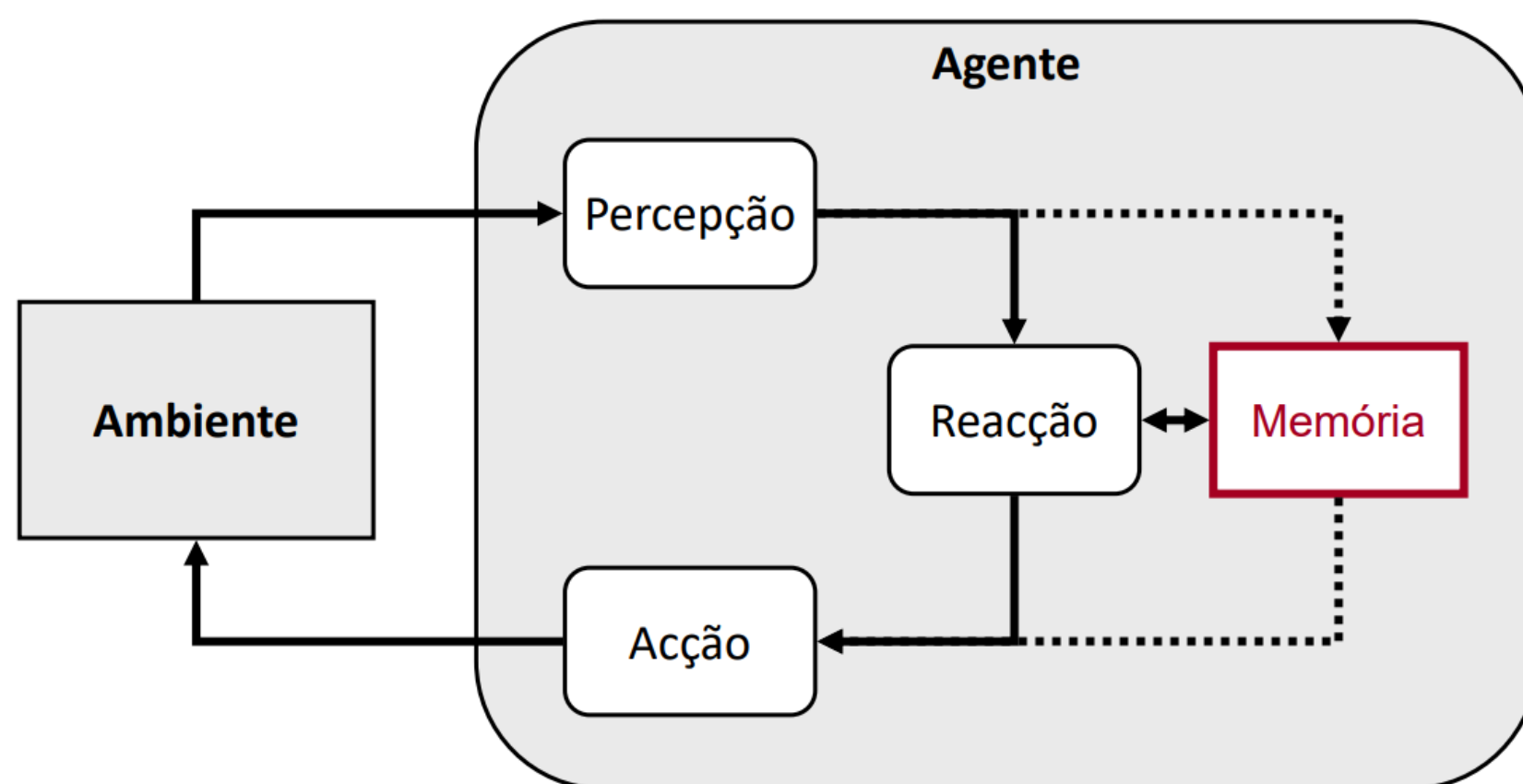


Figura 20 - Arquitetura reativa com memória

Nesta arquitetura, as reações podem envolver não apenas percepções, mas também estado interno (memória), ou seja, tem de ser possível a manipulação de estado, de forma a promover comportamentos com memória. Isto pode ser atingido com o uso de uma máquina de estados interna.

- **Vantagens da manutenção de estado:** É possível produzir todo o tipo de comportamento; possibilita a representação de dinâmicas temporais; possibilita a existência de comportamentos mais complexos baseados na evolução de estado e possui a capacidade de lidar com situações de falha por exploração de ações não realizadas anteriormente
- **Desvantagens da manutenção de estado:** É necessário memória (espaço), o que leva ao aumento da complexidade espacial; é necessário manter as representações de estado, o que leva ao aumento da complexidade computacional e, mesmo com a manutenção de estado, as arquiteturas reativas não suportam representações complexas, nem exploram planos alternativos de ação.

2.3.5. Arquitetura de Subsunção

Na arquitetura de subsunção, os comportamentos são organizados em **camadas** (níveis de competência) e são responsáveis pela concretização independente de um objetivo. O **resultado** do comportamento pode ser a **entrada** de outro comportamento. Existe a possibilidade de comportamentos das **camadas superiores assumirem o controlo sobre comportamentos das camadas inferiores**, no entanto, as camadas inferiores **não têm conhecimento** das camadas superiores (hierarquia de comportamentos). Os principais tipos de mecanismos de controlo associados são:

- **Inibição** de entradas (a informação de entrada não é considerada);
- **Supressão** das saídas (a informação de saída não é considerada);
- **Reinício** do estado interno (reset).

2.4. Raciocínio Automático

O **raciocínio automático** é uma área da inteligência artificial orientada para a resolução automática de problemas com recurso a meios algorítmicos. Este está relacionado com a capacidade de um computador de **resolver problemas** automaticamente, tendo como entrada uma **representação de conhecimento**, específica a um determinado contexto ou domínio, e como saída **conclusões baseadas nesse conhecimento**. Isto implica a **conversão de informação concreta** do domínio do problema em **estruturas simbólicas** e vice-versa, realizada por processos de codificação e decodificação de informação.

As **duas principais vertentes** do raciocínio automático são:

- **Exploração de opções:** Através de raciocínio prospetivo, que consiste na antecipação do que pode acontecer, ou através da simulação interna do domínio do problema, que implica que existam representações internas do domínio do problema.
- **Avaliação de opções:** Avalia as opções com base nos recursos gastos (custo) ou nos ganhos ou perdas que essas podem trazer (valor).

Um dos principais passos no raciocínio automático é **representar o problema** em questão. Essa representação é designada **modelo do problema** e consiste em estruturas simbólicas, mantidas na memória do sistema, essenciais à resolução do problema. O modelo do problema é o suporte do raciocínio automático e as suas estruturas simbólicas são:

- **Estado:** Representa uma configuração específica do problema num determinado momento. Um conjunto de estados e transições de estado é designado por **espaço de estados**.
- **Operador:** Representa uma ação, isto é, uma transição de um estado para outro.

O modelo do problema geralmente mantém o **estado inicial** do problema, uma **lista dos operadores** possíveis e o **objetivo**, representado por um estado que se deseja atingir.

2.4.1. Procura em Espaço de Estados

A procura em espaço de estados é um método de resolução de problemas, nomeadamente **problemas de planeamento**, que envolve raciocínio automático. Neste tipo de problemas, o objetivo é encontrar uma **solução** que consiste numa **sequência de ações** a realizar, partindo de uma situação inicial, até se atingir uma situação final. A resolução destes problemas tem por base um **espaço de estados**, em que cada configuração é representada como um **estado** e as várias ações são representadas como **operadores**. A solução corresponde a uma **sequência** de estados e operadores, que formam um **percurso** no espaço de estados, ligando o estado inicial ao estado final (objetivo).

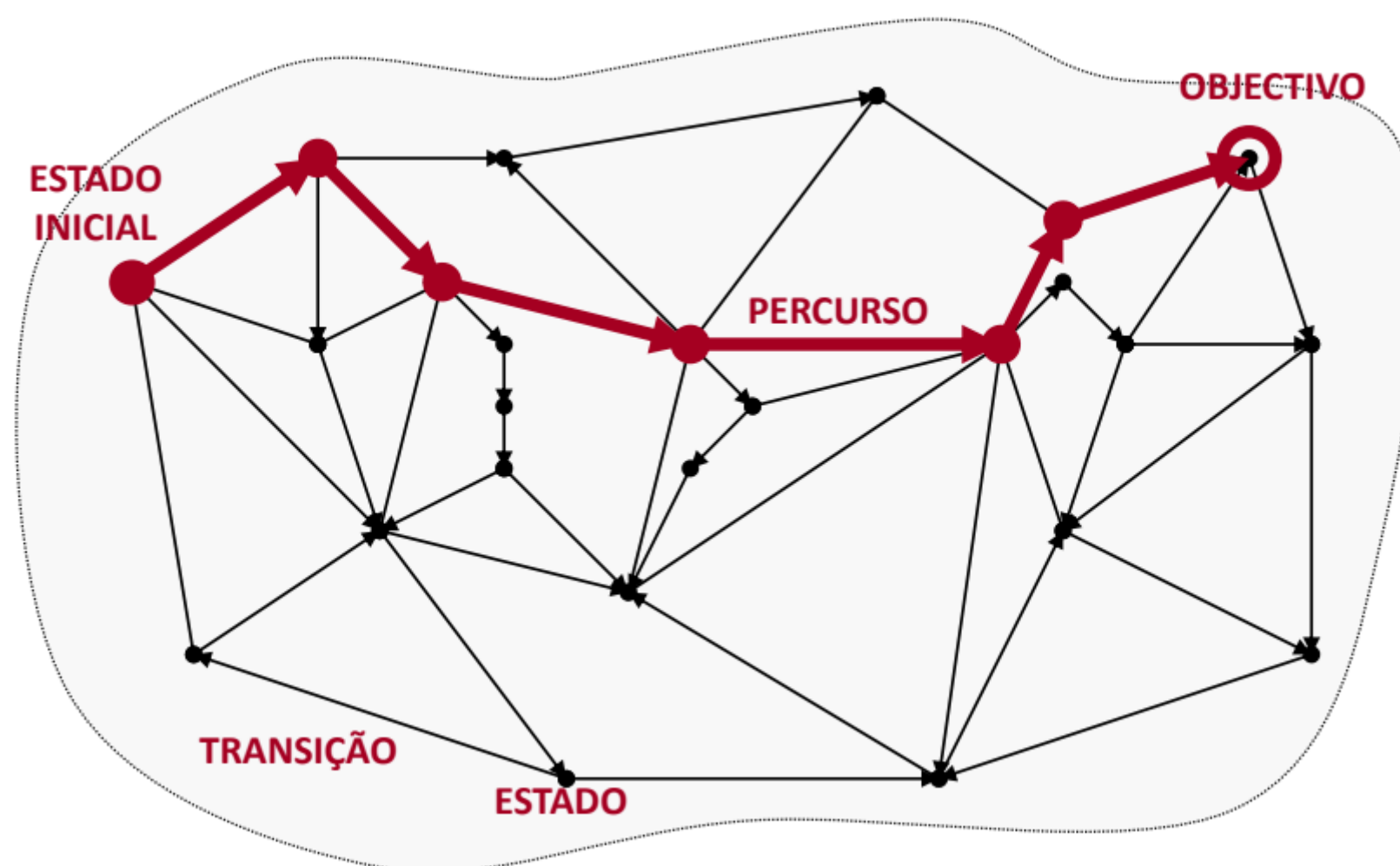


Figura 21 - Exemplo de grafo de espaço de estados com solução

O espaço de estados é representado em grafo, onde cada vértice corresponde a um estado e cada arco corresponde a uma transição entre estados. O processo de procura ocorre por **exploração do espaço de estados** a partir do estado inicial. É verificado se o **estado atual corresponde ao objetivo**. Se sim, o processamento **termina** e é retornado o **caminho do estado inicial ao estado final**. Se não, o estado atual é **expandido**, gerando **estados sucessores** por **aplicação de operadores**. Este processo é repetido para todos os estados sucessores até que se atinja o objetivo ou não haja sucessores para expandir.

De forma a armazenar toda esta informação, é mantida uma estrutura designada **árvore de procura**, organizada em **nós**, que mantém informação relativa a cada transição de estado realizada. Relaciona cada nó com o seu **antecessor** e mantém informação correspondente ao **nó** e ao **operador** que originou a **transição** de estado respetiva. Para além da árvore de procura, é mantida também uma estrutura de dados **ordenada**, designada **fronteira de exploração**, que determina a estratégia de controlo da procura e que mantém os nós que serão explorados.

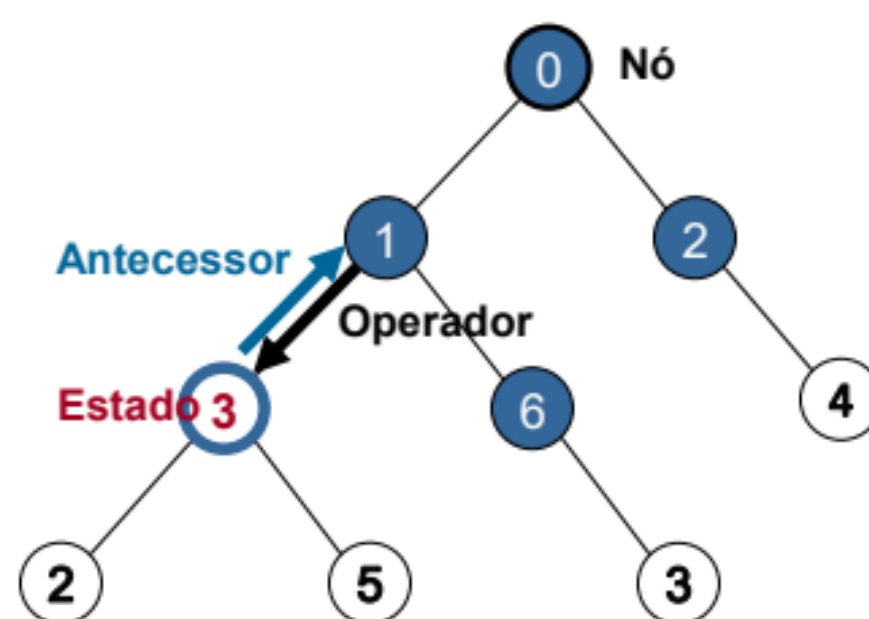


Figura 22 - Exemplo de árvore de procura

- **Procura em Profundidade:** A procura em profundidade (*Depth-First Search*) explora primeiro os nós mais **recentes**, aumentando a **profundidade** do ramo de procura corrente.
- **Procura em Largura:** A procura em largura (*Breadth-First Search*) explora primeiro os nós mais **antigos**, realizando uma **exploração exaustiva** de cada nível de procura antes de passar para os níveis de maior profundidade.

Existe um fenómeno que pode ocorrer nestes tipos de procura que contribui para o **desperdício de recursos**, quer seja de tempo de processamento ou de memória. Existe a possibilidade de haver **vários caminhos** para o mesmo estado e de a procura ficar presa em **ciclos**, revisitando o mesmo estado repetidamente. De forma a prevenir a exploração de nós correspondentes a estados repetidos, é conveniente **verificar** se o nó sucessor corresponde a um estado **já explorado**. Se isso acontecer, é **mantido** apenas o nó que corresponde ao caminho com o **menor custo**. Para facilitar esta gestão, pode ser mantida uma estrutura de dados correspondente aos nós **explorados**, constituída pelos nós já **expandidos (fechados)** e pelos nós **gerados, mas não expandidos (abertos, na fronteira de exploração)**.

- **Procura Melhor-Primeiro:** Utiliza uma função de avaliação, $f(n)$, para determinar a qualidade de cada nó gerado. O valor de $f(n)$ representa uma estimativa do custo da solução através do nó (n). Quanto menor o valor de $f(n)$, mais promissor é o nó. Neste método, a fronteira de exploração é ordenada em ordem crescente de $f(n)$.
- **Procura de Custo Uniforme:** É um caso particular da procura Melhor-Primeiro, em que $f(n)$ corresponde ao custo associado a cada nó. Este método prioriza a exploração de caminhos com menor custo. Desta forma, é garantido que a solução encontrada seja ótima, caso existam várias soluções possíveis. A estratégia de controlo é explorar primeiro os percursos com menor custo de transição.

Estes são **mecanismos de procura não informada**, ou seja, não utilizam conhecimento do domínio do problema para guiar a procura. No entanto,

existem outras características que são importantes de considerar num método de procura, que são:

- **Completo:** O método de procura garante que a solução é encontrada, caso exista.
- **Ótimo:** O método de procura garante que a solução encontrada é a melhor, caso existam várias soluções.
- **Complexidade:** Complexidade temporal, tempo necessário para encontrar uma solução e complexidade espacial, memória necessária para encontrar uma solução.

Além disso, é importante compreender a **complexidade computacional** dos métodos de procura. A complexidade computacional descreve como a **quantidade de recursos** necessários (espaço e tempo) **cresce** em função da **dimensão do problema**. Esta é expressa por meio da notação $O(g)$, que indica a ordem de crescimento em relação a um tipo de processo computacional. Os métodos de procura não informada são estratégias de exploração do espaço de estados que **não se baseiam no conhecimento** específico do problema para ordenar a fronteira de exploração. Estes envolvem a **exploração exaustiva** do espaço de estados, sem utilizar nenhuma orientação.

Existem **métodos de procura informada**, que **tiram partido do conhecimento** do domínio do problema para ordenar a fronteira de exploração. Em vez de realizarem uma exploração exaustiva, realizam uma **exploração seletiva**. Utilizam **heurísticas** como forma de avaliação de nós. Esses métodos são, mais uma vez, casos particulares da procura melhor-primeiro, mas, a sua função de avaliação é baseada em estimativas de custo (heurística).

- **Procura Sôfrega:** Neste caso, o objetivo é **minimizar a estimativa** de custo para atingir o objetivo. A procura Sôfrega não leva em consideração o custo do percurso explorado, focando-se **apenas na heurística**. No entanto, esta abordagem pode levar a **soluções sub-ótimas**, pois não considera o custo real do percurso.
- **Procura A*:** Este método pretende **minimizar o custo global**, que é a **soma do custo acumulado** até o nó (n) com a **estimativa de custo** até o objetivo. A procura A* utiliza uma **heurística admissível** (a estimativa de custo é sempre inferior ou igual ao custo efetivo mínimo) para orientar a exploração do espaço de estados, garantindo que a estimativa de custo nunca seja superestimada. Assim, é capaz de encontrar **soluções ótimas**.

2.5. Arquitetura de Agentes Deliberativos

A arquitetura de agentes deliberativos é uma abordagem de desenvolvimento de agentes inteligentes na qual o agente usa processos internos complexos para **tomar decisões e planejar ações**. Esta arquitetura é caracterizada pela sua capacidade de **antecipar** antes de agir, considerando não apenas o estado atual do ambiente, mas também memórias passadas e **simulações de estados futuros**.

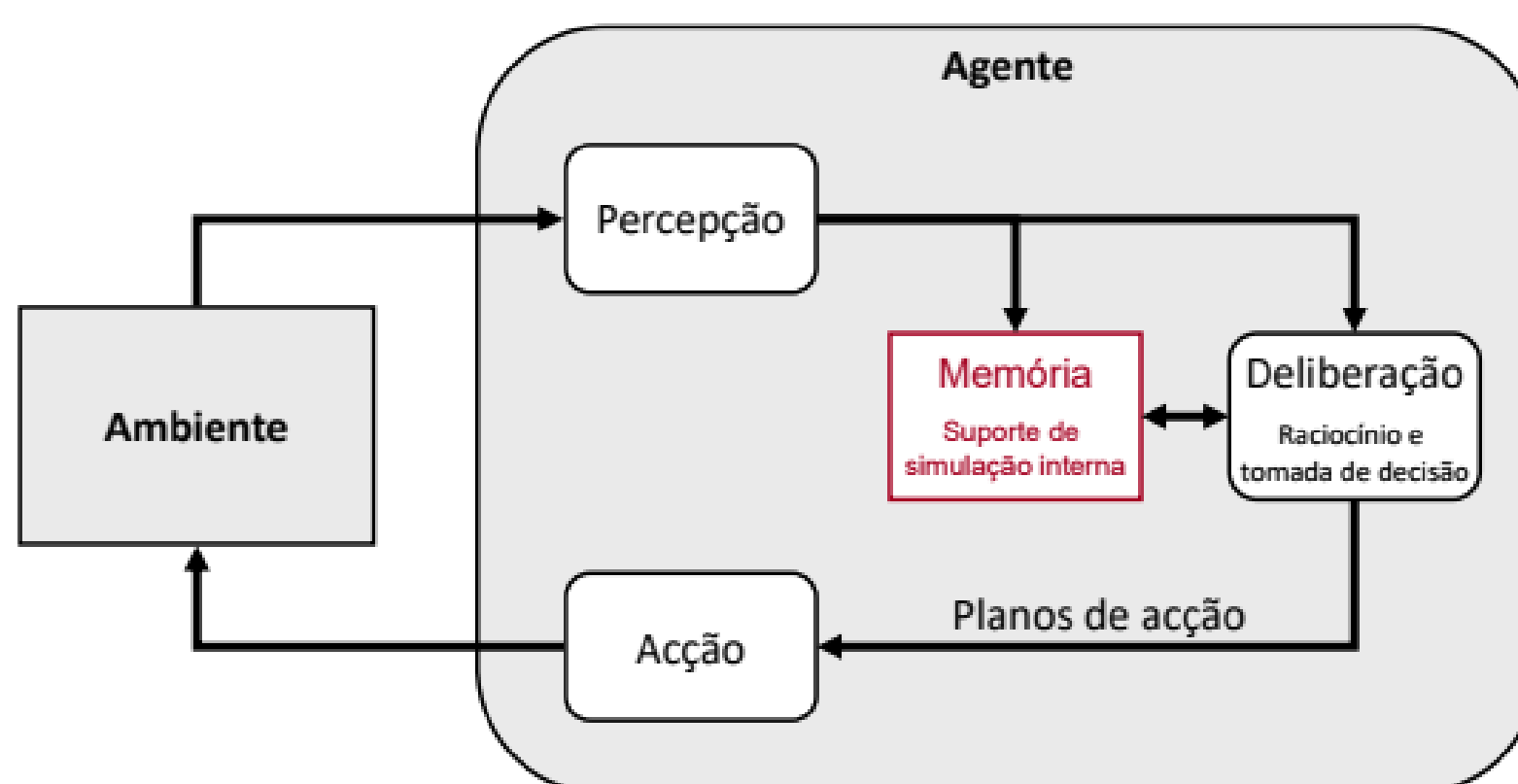


Figura 23 - Modelo da arquitetura deliberativa

No contexto de um agente autónomo que opera num determinado ambiente, é particularmente relevante o **raciocínio orientado para a ação**, designado **raciocínio prático**. Nesse tipo de raciocínio são utilizadas **representações simbólicas dos objetivos** a atingir, das **ações que o agente é capaz de realizar** e do **ambiente**, sendo gerados **planos de ação** que determinam o **comportamento do agente**.

O **raciocínio prático** possui duas partes principais: o **raciocínio sobre os fins** a serem alcançados, **deliberação**, que resulta na definição de um conjunto de objetivos a serem cumpridos, e o **raciocínio sobre os meios** (ações e recursos) necessários para alcançar esses fins, **planeamento**, que gera planos de ação para atingir os objetivos estabelecidos pela deliberação. Estas partes suportam o **processo de tomada de decisão e ação** de um agente deliberativo, que é caracterizado pelos passos seguintes:

1. **Observar** o mundo, gerando percepções.
2. **Atualizar** o modelo do mundo, com base nas percepções.
3. **Deliberar** o que fazer, gerando um conjunto de objetivos.
4. **Planear** como fazer, gerando um plano de ação.
5. **Executar** o plano de ação.

Tendo em conta que o ambiente em que o agente se encontra é dinâmico, este pode sofrer alterações durante o raciocínio do agente. Nestas situações, é oportuno realizar uma **reconsideração**, isto é, repetir os passos 3 e 4 para que o plano de ação seja consistente com a situação do ambiente.

2.5.1 Planeamento Automático com PEE

O planeamento automático é um processo deliberativo cujo objetivo é gerar sequências de ações (planos) para alcançar objetivos pré-estabelecidos. Esse planeamento é realizado com base em métodos de raciocínio automático, como a procura em espaços de estados (PEE) ou processos de decisão de Markov (PDM). Para planejar, é necessário um modelo de planeamento que se define o estado inicial do problema e um conjunto de estados válidos, além da definição dos objetivos a serem alcançados.

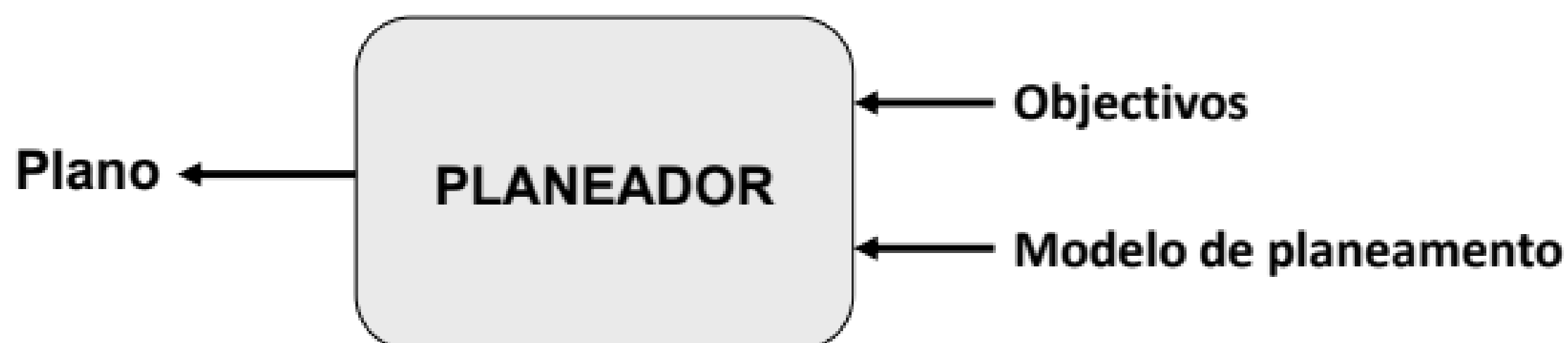


Figura 24 - Modelo do planeamento automático

Numa arquitetura de agente deliberativo, o planeador produz planos de ação com base nos objetivos gerados pelo mecanismo de deliberação. No caso de um planeador baseado em procura em espaços de estados, é necessário considerar o modelo do problema de planeamento, a heurística a utilizar (se necessário) e o mecanismo de procura.

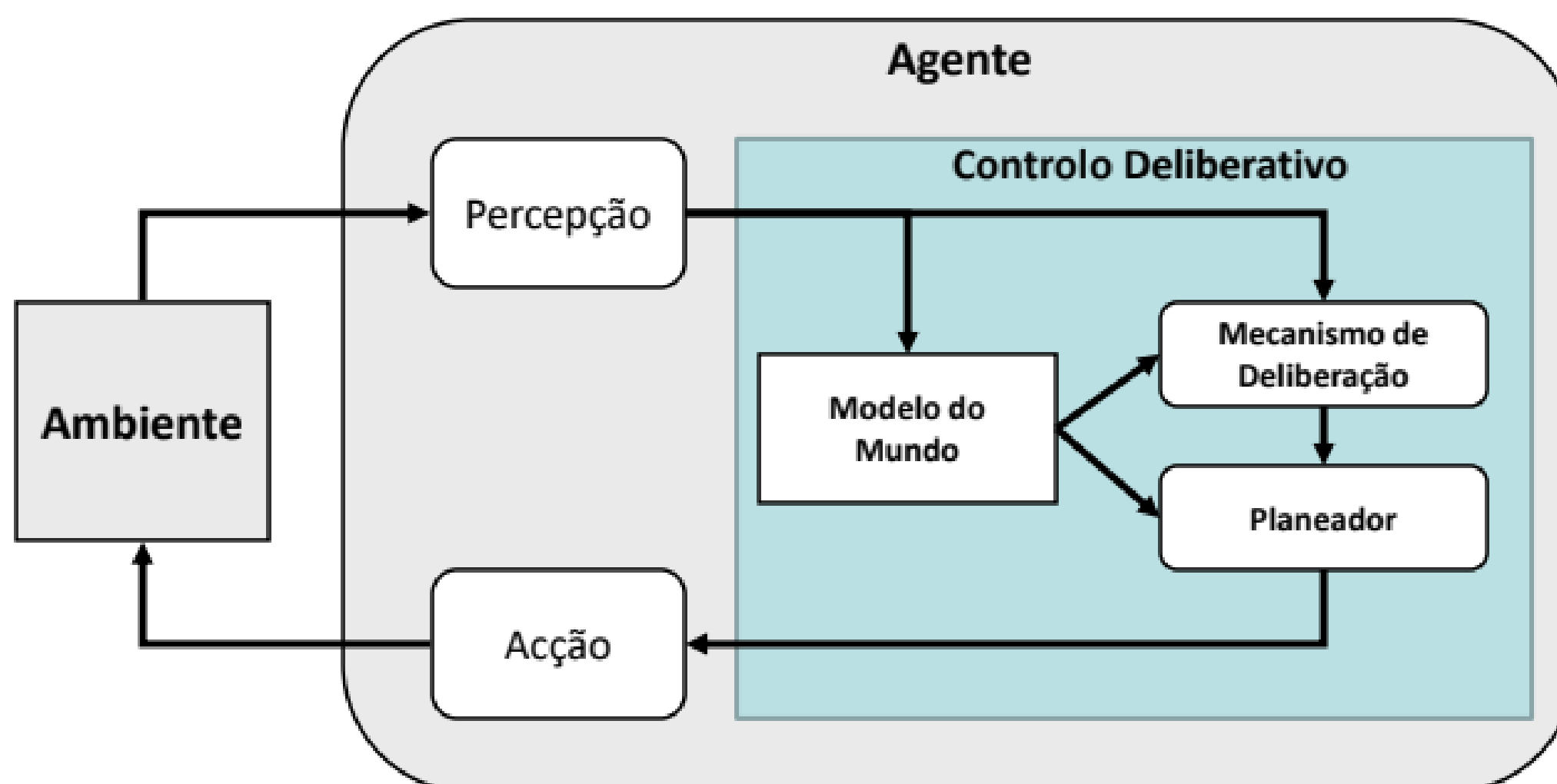


Figura 25 - Arquitetura de um agente deliberativo

2.5.2. Processos de Decisão Sequencial

No **problema de tomada de decisão** para um agente que opera em um determinado ambiente, os estados e ações do agente mudam ao longo do tempo, dependendo das **decisões** feitas, **ações** escolhidas e os **resultados** alcançados. A questão que surge é como **prever e controlar** a interação contínua entre o agente e o ambiente, visando um objetivo de longo prazo.

Num processo de decisão sequencial, cada estado pode levar a **múltiplos estados** e **decisões** futuras através de ações. O **valor** (ou utilidade) desses estados e decisões pode não ser conhecido de imediato, sendo revelado com o tempo, à medida que mais estados e decisões se desenrolam. Além disso, a evolução dos estados em função das ações realizadas pode ser **incerta** e **não determinista**.

Dentro deste contexto, **processos de decisão sequencial** surgem, onde a transição entre estados ocorre por meio de **ações** que podem ser **imprevisíveis**. Isso significa que o resultado de uma ação pode não ser completamente conhecido, introduzindo **incertezas**. Esses processos são representados por **espaços de estados não deterministas**, onde as transições entre estados acontecem com **probabilidades** específicas. Cada transição pode estar associada a uma **recompensa**, que indica o ganho ou perda resultante dessa mudança de estado.

A **propriedade de Markov** define que um **processo estocástico** possui essa propriedade se a distribuição probabilística dos estados futuros **depende unicamente do estado atual**. Isso implica que para prever os estados futuros, é suficiente conhecer **apenas o estado presente**.

No contexto dos processos de decisão sequencial, o mundo pode ser modelado como um **Processo de Decisão de Markov**. Um PDM é composto por um **conjunto de estados do mundo S** , um **conjunto de ações** possíveis em cada estado $A(s)$, a **probabilidade de transição** entre os estados dados uma ação $T(s, a, s')$ e a **recompensa** esperada para essa transição $R(s, a, s')$.

A **utilidade** em processos de decisão sequencial representa o **efeito cumulativo** da evolução da situação ao longo do tempo. É calculada com base nas recompensas, refletindo os ganhos ou perdas em estados específicos. A utilidade de uma sequência de estados $(s_0, s_1, \dots, s_n)$ é a soma das recompensas em cada estado: $R(s_0) + R(s_1) + R(s_2) + \dots$

A utilidade também pode ser calculada considerando o desconto no tempo, onde as recompensas futuras são ajustadas por um fator de desconto (γ), que varia entre 0 e 1, resultando em: $U_h([s_0, s_1, s_2, \dots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) \dots$

Neste caso, o fator de desconto reduz a importância das recompensas futuras em relação às recompensas imediatas. É importante notar que as **recompensas podem variar continuamente**, não estando limitadas a valores finitos. A **utilidade é crucial para avaliar e comparar ações** e sequências de decisões, permitindo que o agente planeie estratégias para atingir seus objetivos a longo prazo.

A **política comportamental** define como o agente deve agir em cada estado, especificando qual ação tomar, seja de forma determinista ($\pi: S \rightarrow A(s)$) ou não determinista ($\pi: S \times A(s) \rightarrow [0,1]$).

O **Princípio da Solução Ótima**, fundamentado na Programação Dinâmica, é essencial nesse contexto, pois permite **calcular as utilidades dos estados com base nas utilidades dos estados subsequentes**. Essa abordagem é baseada nas **Equações de Bellman**, que fornecem uma forma de determinar a melhor política para o agente seguir a fim de maximizar as suas recompensas a longo prazo.

2.5.3. Planeamento Automático com PDM

O planeamento automático com base em processos de decisão de Markov é baseado numa **representação do problema** sob a forma de um **processo de decisão de Markov**:

- S : conjunto de estados do mundo
- $A(s)$: conjunto de ações possíveis no estado s pertencente a S
- $T(s,a,s')$: probabilidade de transição de s para s' através de a
- $R(s,a,s')$: retorno esperado na transição de s para s' através de a
- γ : taxa de desconto para recompensas diferidas no tempo

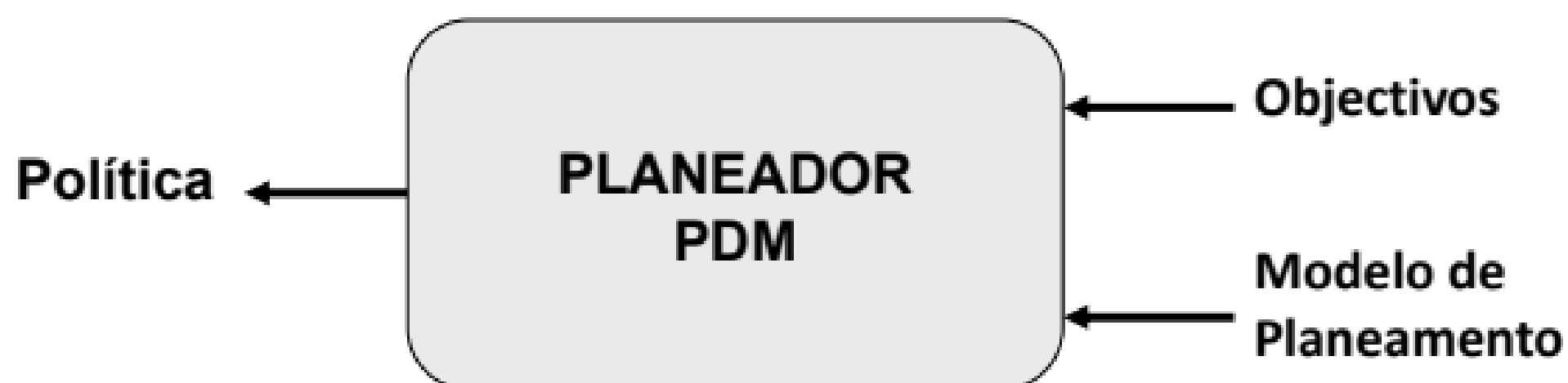


Figura 26 - Modelo do planeador PDM

Produz como resultado uma **política de ação**, que define a respetiva ação a realizar para cada estado possível. Envolve o cálculo da **função de utilidade** (valor), que define o **valor associado a cada estado** do problema, a qual serve de suporte ao cálculo da política de ação, nomeadamente, a **política ótima**, que **determina a seleção das ações** que maximizam o valor de ação em cada estado.

2.6. Aprendizagem por Reforço

A aprendizagem por reforço é um método utilizado para **melhorar o desempenho numa tarefa específica** (T) com base na experiência adquirida. Isto é feito através da **definição de uma medida de desempenho** (D) e da **utilização dessa experiência** (E) para tomar ações que maximizem essa medida.

Existem várias abordagens para a aprendizagem por reforço, cada uma focando-se em diferentes aspetos da tarefa. A **aprendizagem concetual** envolve a criação de abstrações e modelos para entender conceitos e padrões de comportamento. Em contraste, a **aprendizagem comportamental** foca-se na definição de ações que devem ser realizadas em várias situações.

No contexto da aprendizagem por reforço, **os agentes interagem** com o ambiente e **aprendem com base em três componentes** principais: **estado**, **ação** e **reforço**. O estado representa a situação atual do agente, a ação é a escolha feita pelo agente e o reforço é uma medida que indica se a ação foi eficaz ou não em relação à tarefa em questão.

Para estimar o valor de uma ação num determinado estado, são utilizados métodos como a **Aprendizagem de Valor de Ação**. Isso envolve escolher repetidamente diferentes ações e receber as recompensas correspondentes. O valor médio de uma ação é calculado incrementalmente, tendo em consideração as recompensas obtidas ao longo do tempo.

Um dos **desafios na aprendizagem por reforço** é o **equilíbrio entre exploração e aproveitamento** do conhecimento adquirido. Exploração envolve a escolha de ações que permitam ao agente obter mais informações sobre o ambiente, enquanto aproveitamento consiste em selecionar as ações que resultam nas melhores recompensas. Estratégias como o **método ϵ -greedy** ajudam a balancear exploração e aproveitamento.

Existem algoritmos específicos para a aprendizagem por reforço, como **SARSA e Q-Learning**. Esses algoritmos permitem que os agentes aprendam com a experiência, atualizando os valores Q (valor de estado-ação) com base nas recompensas recebidas e nas transições entre estados.

Finalmente, o **objetivo do processo de aprendizagem é encontrar uma política ótima**, que indica a melhor ação a ser tomada em cada estado. Essa política é derivada da função valor Q, que representa o valor estimado de realizar uma determinada ação em um estado específico.

3. PROJETO – PARTE 1

Na primeira parte do projeto, pretendeu-se e implementar um jogo com uma personagem virtual que interage com um jogador humano. O jogo consiste num ambiente onde a personagem tem por objetivo registar a presença de animais através de fotografias. Quando o jogo se inicia a personagem fica numa situação de procura de animais. Quando deteta algum ruído aproxima-se e fica em inspeção da zona, procurando a fonte do ruído. Quando volta a haver silêncio a personagem volta a uma situação de procura de animais. Quando deteta um animal a personagem aproxima-se e fica em observação. Caso o animal continue presente, a personagem observa o animal e fica preparada para o registo, se ocorrer a fuga do animal a personagem fica em inspeção da zona, à procura de uma fonte de ruído. Na situação de registo, se o animal continuar presente fotografa-o, caso ocorra a fuga do animal ou a personagem tenha conseguido uma fotografia do animal, a personagem fica novamente numa situação de procura. A interação com o jogador foi realizada em modo de texto. Esta parte do projeto foi implementada com recurso à linguagem de programação Java.

3.1. Organização e Arquitetura do Projeto



```
graph TD
    iasa_jogo --> src
    src --> agente
    src --> ambiente
    src --> jogo
    src --> maquest
    agente --> Accao.java
    agente --> Agente.java
    agente --> Controlo.java
    agente --> Percepcao.java
    ambiente --> Ambiente.java
    ambiente --> Comando.java
    ambiente --> Evento.java
    jogo --> ambiente_jogo[ambiente]
    jogo --> personagem
    jogo --> Jogo.java
    ambiente_jogo --> AmbienteJogo.java
    ambiente_jogo --> ComandoJogo.java
    ambiente_jogo --> EventoJogo.java
    personagem --> ControloPersonagem.java
    personagem --> Personagem.java
    maquest --> Estado.java
    maquest --> MaquinaEstados.java
    maquest --> Transicao.java
```

- ▽ iasa_jogo
 - ▽ src
 - ▽ agente
 - J Accao.java
 - J Agente.java
 - J Controlo.java
 - J Percepcao.java
 - ▽ ambiente
 - J Ambiente.java
 - J Comando.java
 - J Evento.java
 - ▽ jogo
 - ▽ ambiente
 - J AmbienteJogo.java
 - J ComandoJogo.java
 - J EventoJogo.java
 - ▽ personagem
 - J ControloPersonagem.java
 - J Personagem.java
 - J Jogo.java
 - ▽ maquest
 - J Estado.java
 - J MaquinaEstados.java
 - J Transicao.java

Figura 27 - Organização das pastas e módulos do projeto

- Subsistema Ambiente e subsistema Agente

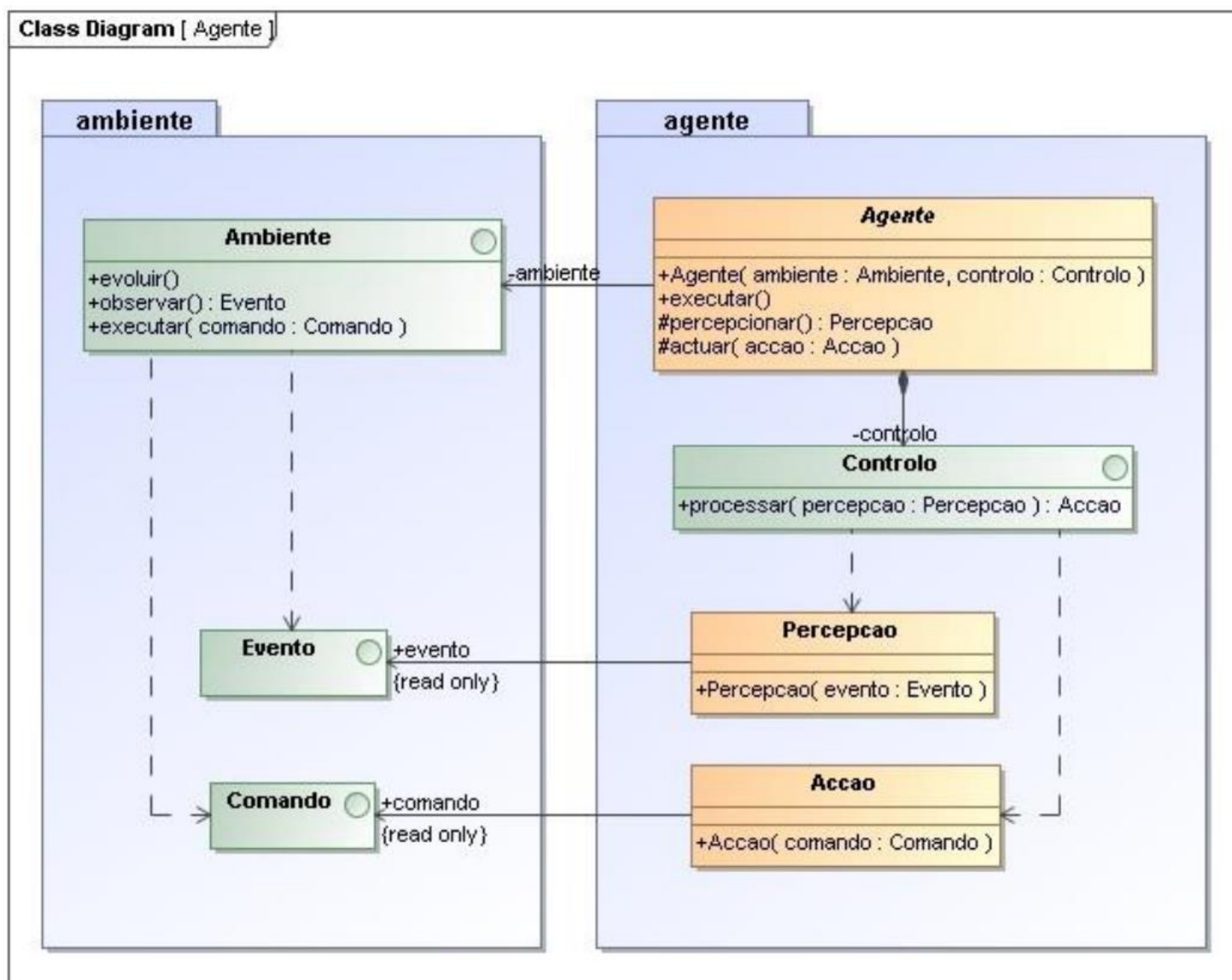


Figura 28 - Subsistema Ambiente e subsistema Agente

- Subsistema Ambiente do Jogo

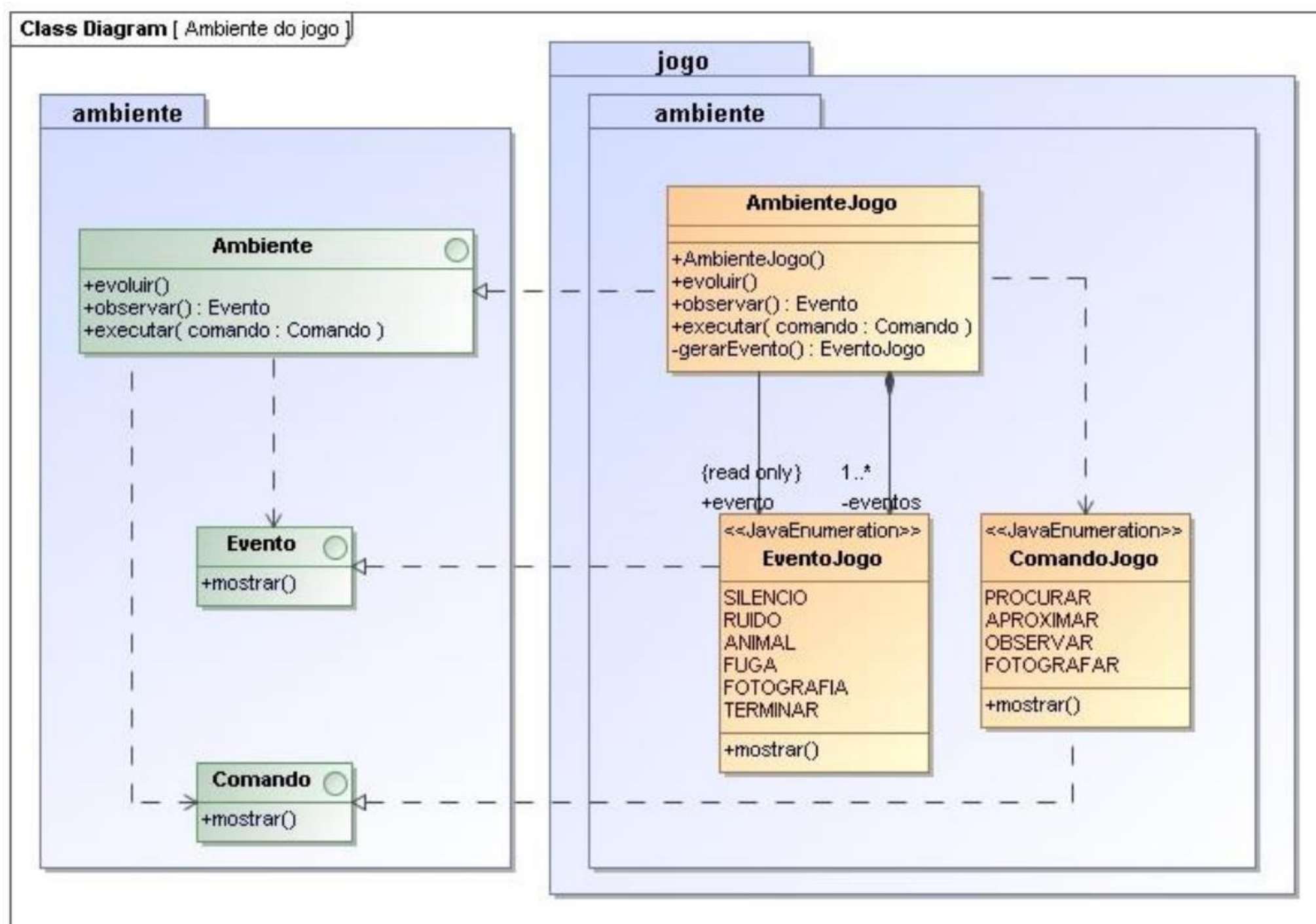


Figura 29 - Subsistema Ambiente do Jogo

- Subsistema Personagem

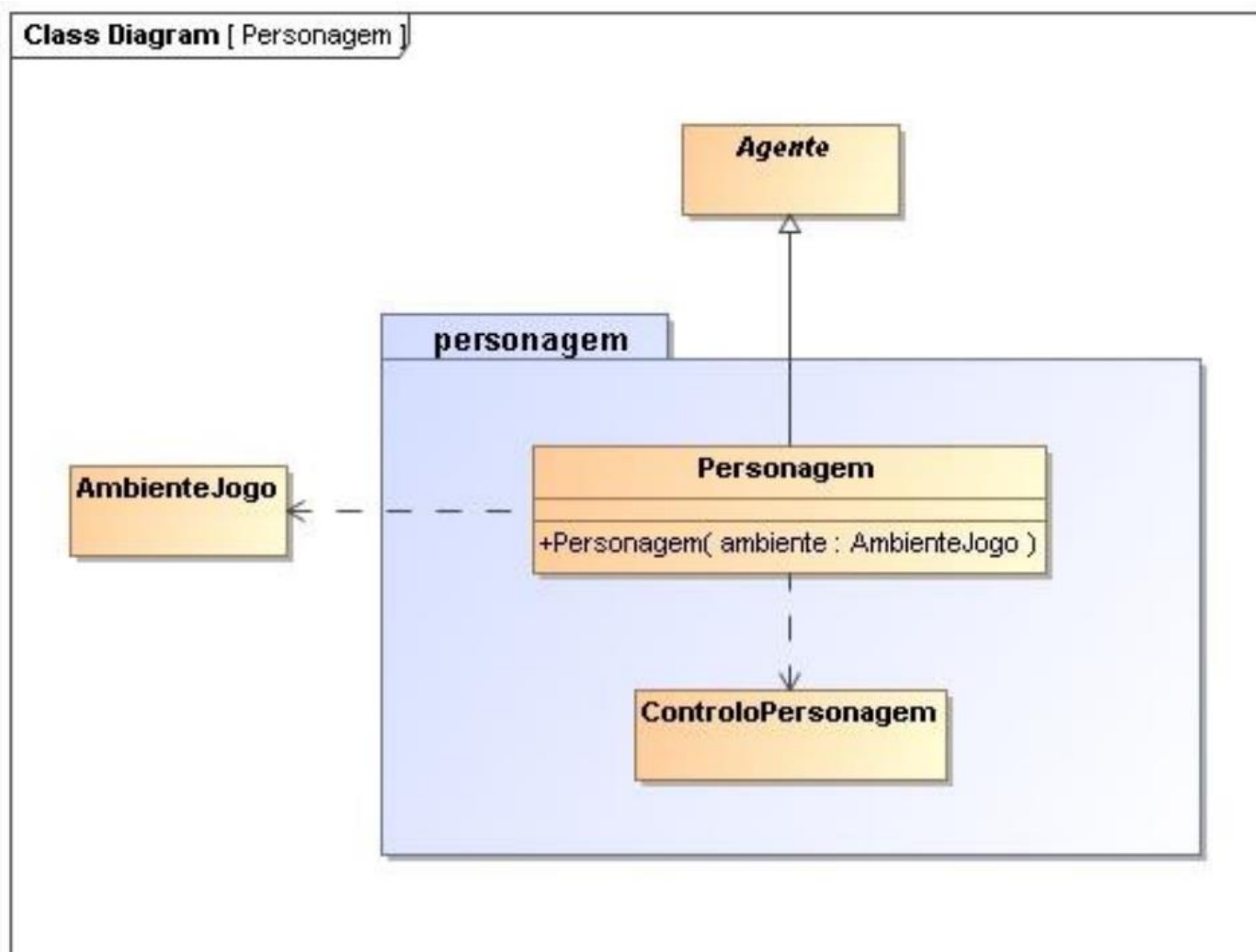


Figura 30 - Subsistema Personagem

- Controlo da Personagem

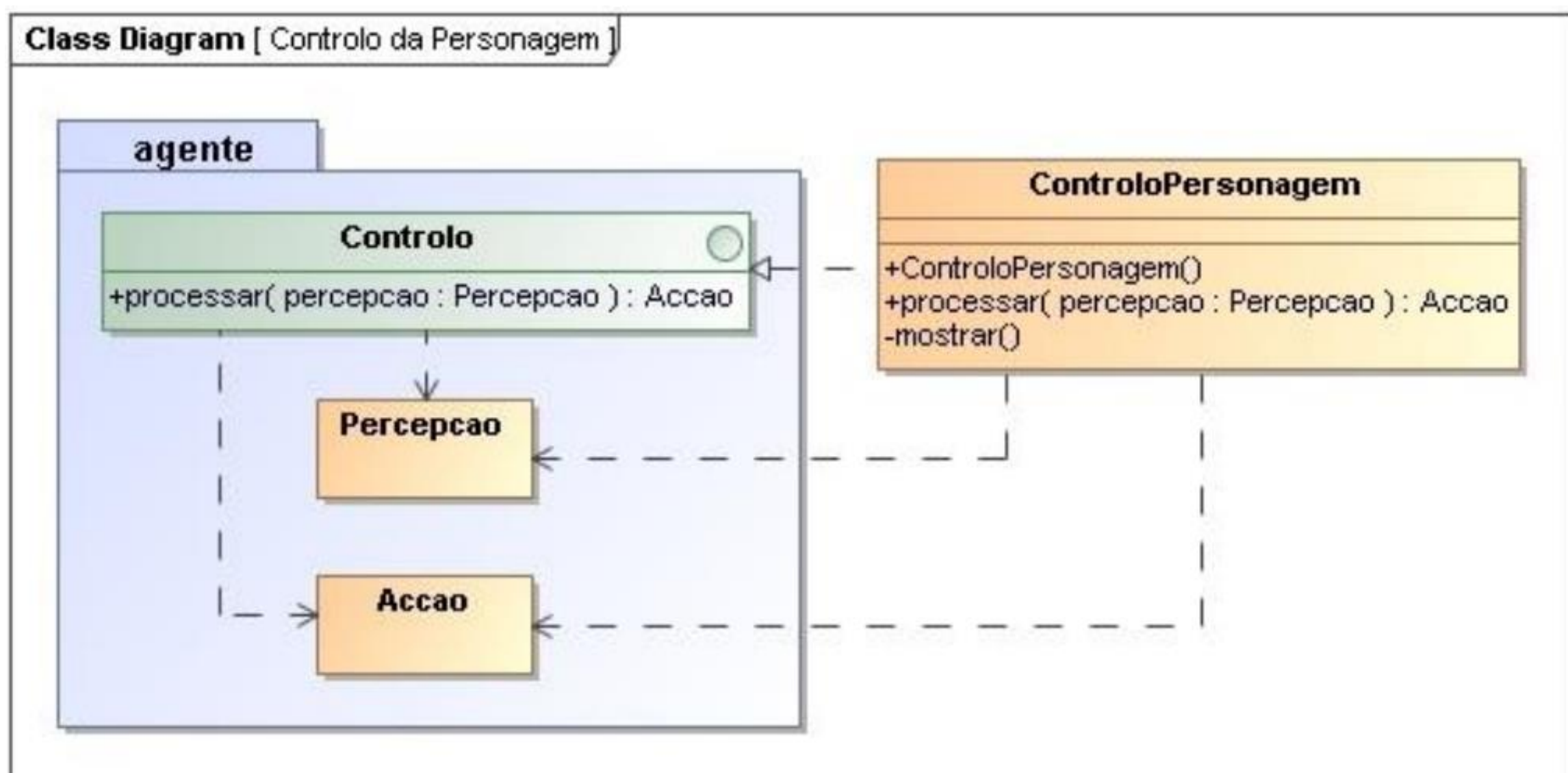


Figura 31 - Controlo da Personagem

• Subsistema Jogo

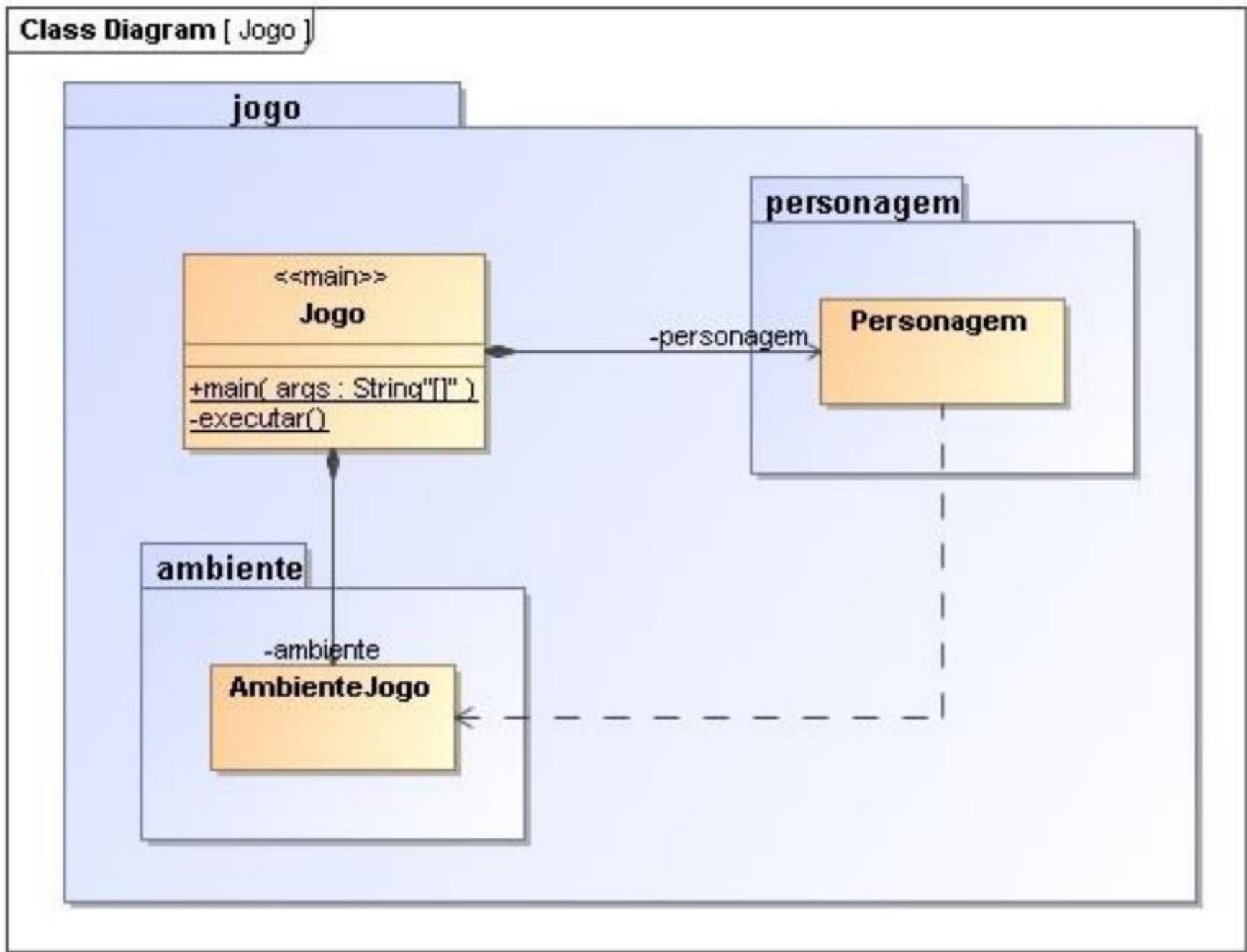


Figura 32 - Subsistema Jogo

• Subsistema Máquina de Estados

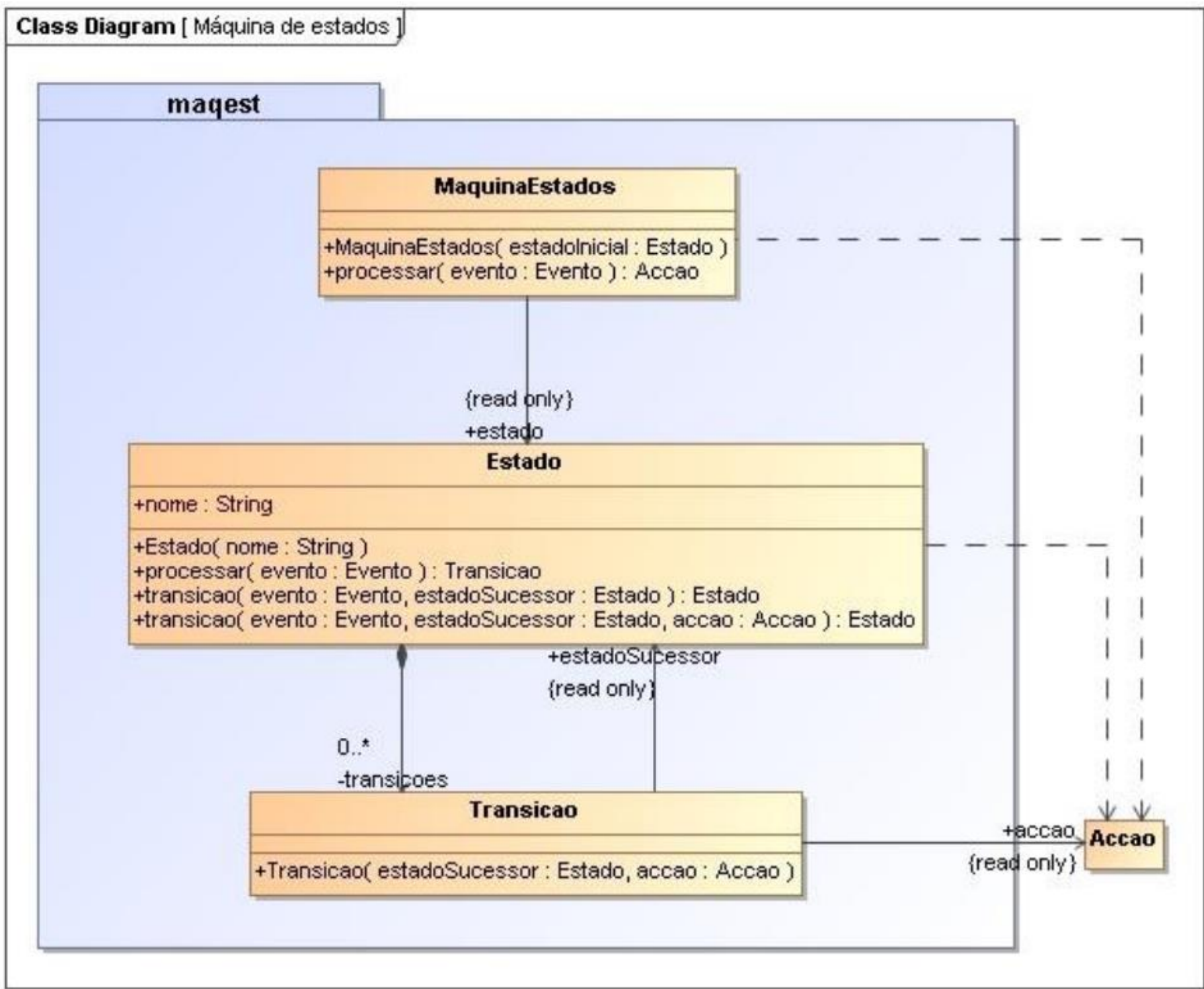


Figura 33 - Subsistema Máquina de Estados

- Controlo da Personagem com Máquina de Estados

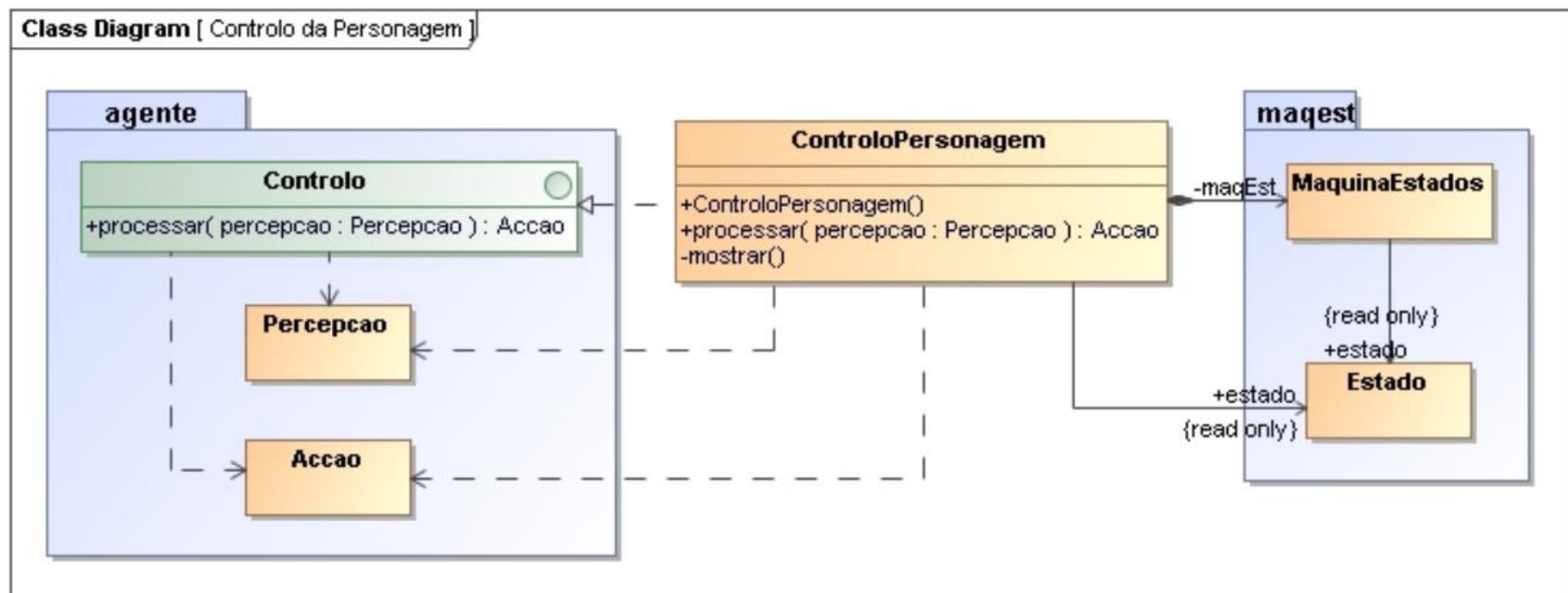


Figura 34 - Controlo da Personagem com Máquina de Estados

3.2. Implementação

- Interface Ambiente

```
1 package ambiente;  
2  
3 public interface Ambiente {  
4     public void evoluir();  
5     public Evento observar();  
6     public void executar(Comando comando);  
7 }
```

Figura 35 - Interface Ambiente

A interface **Ambiente** representa o meio onde se encontra o agente, com o qual este consegue interagir e observar. Esta classe tem uma relação de dependência com as interfaces **Evento** e **Comando** pois utiliza localmente instâncias dessas interfaces. Uma interface representa um contrato funcional independente da implementação, ou seja, os métodos definidos pela interface não são implementados nesta, só serão implementados nas classes que a realizarem, assim promovendo modularidade através do encapsulamento da implementação.

Contém os métodos **evoluir()**, **observar()** e **executar()**, que serão implementados pelas classes que realizarem esta interface.

- Interface Evento

```
1 package ambiente;  
2  
3 public interface Evento {  
4     public void mostrar();  
5 }
```

Figura 36 - Interface Evento

A interface **Evento** representa o resultado da evolução do ambiente, ou seja, após a execução de um comando no ambiente, é gerado um Evento.

Contém o método **mostrar()** que será implementado pela classe que realizar esta interface.

- Interface Comando

```
1 package ambiente;  
2  
3 public interface Comando {  
4     public void mostrar();  
5 }
```

Figura 37 - Interface Comando

A interface **Comando** representa um comando para ser executado sobre o ambiente de modo a gerar um evento.

Contém o método **mostrar()** que será implementado pela classe que realizar esta interface.

- Classe Agente

```
1 package agente;  
2  
3 import ambiente.Ambiente;  
4 import ambiente.Evento;  
5  
6 public class Agente {  
7     private Ambiente ambiente;  
8     private Controllo controllo;  
9  
10    public Agente(Ambiente ambiente, Controllo controllo) {  
11        this.ambiente = ambiente;  
12        this.controllo = controllo;  
13    }  
14  
15    public void executar(){  
16        Percepcao percepcao = percepcionar();  
17        Accao accao = controllo.processar(percepcao);  
18        actuar(accao);  
19    }  
20  
21    protected Percepcao percepcionar(){  
22        Evento evento = ambiente.observar();  
23        return new Percepcao(evento);  
24    }  
25  
26    protected void actuar(Accao accao){  
27        if (accao != null) {  
28            ambiente.executar(accao.getComando());  
29        }  
30    }  
31 }
```

Figura 38 - Classe Agente

A classe **Agente** representa o interveniente no ambiente, que é capaz de observar o ambiente, formando percepções, processar as percepções criadas gerando ações, e atuar sobre essas ações. Esta classe tem uma relação de associação com a interface **Ambiente** porque o **Agente** necessita de conhecer o ambiente para poder interagir com ele, e por isso tem um atributo do tipo **Ambiente**. Tem também uma relação de composição com a interface **Controlo**, porque o controlo é uma parte de um agente, por isso, quando é criado um agente, também é criado um controlo. Isto significa que existe um atributo do tipo **Controlo** que tem de ser inicializado no construtor. Como uma interface não pode ser instanciada, a instância é recebida como parâmetro do construtor.

Contém o método **percepcionar()** que é usado para formar percepções através da observação de eventos no ambiente; o método **executar()** que serve para processar as percepções criadas, gerando a ação adequada e atuando sobre ela usando o método **actuar()**.

- Interface Controlo

```
1  package agente;  
2  
3  public interface Controlo {  
4      public Accao processar(Percepcao percepcao);  
5  }
```

Figura 39 - Interface Controlo

A interface **Controlo** representa o "pensamento" do agente porque permite que este utilize uma percepção e forme uma ação adequada, através do método **processar()**. Esta classe tem uma relação de **dependência** com as classes **Percepcao** e **Accao** pois utiliza localmente instâncias dessas classes.

Contém o método **processar()**, que apenas será implementado por classes que realizem esta interface, que é usado no método **executar()** do Agente para processar uma percepção e gerar uma ação.

- Classe Percepcao

```
1  package agente;
2
3  import ambiente.Evento;
4
5  public class Percepcao {
6      private Evento evento;
7
8      public Percepcao(Evento evento) {
9          this.evento = evento;
10     }
11
12     public Evento getEvento() {
13         return evento;
14     }
15 }
```

Figura 40 - Classe Percepcao

A classe **Percepcao** representa a interpretação formada pelo agente após observar um evento no ambiente. Tem uma relação de **associação** com a interface **Evento** pois necessita conhecer o evento que lhe está associado. Contém um atributo do tipo Evento disponível apenas para leitura (*read only*). Esta propriedade foi implementada declarando o atributo como privado e gerando um método *getter* público, para possibilitar o acesso ao valor do atributo. Não é criado um método *setter* para que o valor não possa ser alterado.

- Classe Accao

```
1  package agente;
2
3  import ambiente.Comando;
4
5  public class Accao {
6      private Comando comando;
7
8      public Accao(Comando comando) {
9          this.comando = comando;
10     }
11
12     public Comando getComando() {
13         return comando;
14     }
15 }
```

Figura 41 - Classe Accao

A classe **Accao** representa a consequência do processamento de uma percepção por parte do **Controlo** e que dará origem a um determinado comando que será executado sobre o ambiente. Tem uma relação de **associação** com a interface **Comando** pois necessita conhecer o comando que lhe está associado. Contém um atributo do tipo **Comando** disponível apenas para leitura (*read only*). Esta propriedade foi implementada declarando o atributo como privado e gerando um método *getter* público, para possibilitar o acesso ao valor do atributo. Não é criado um método *setter* para que o valor não possa ser alterado.

- Classe Ambiente Jogo

```
1  package jogo.ambiente;
2
3  import java.util.HashMap;
4  import java.util.Map;
5  import java.util.Scanner;
6
7  import ambiente.Ambiente;
8  import ambiente.Comando;
9  import ambiente.Evento;
10
11 public class AmbienteJogo implements Ambiente{
12     private Evento evento;
13     private Map<String, EventoJogo> eventos;
14     private Scanner scanner = new Scanner(System.in);
15
16     public AmbienteJogo(){
17         eventos = new HashMap<String, EventoJogo>();
18         eventos.put(key:"s", EventoJogo.SILENCIO);
19         eventos.put(key:"r", EventoJogo.RUIDO);
20         eventos.put(key:"a", EventoJogo.ANIMAL);
21         eventos.put(key:"f", EventoJogo.FUGA);
22         eventos.put(key:"o", EventoJogo.FOTOGRAFIA);
23         eventos.put(key:"t", EventoJogo.TERMINAR);
24     }
25
26     public Evento getEvento(){
27         return evento;
28     }
29
30     public void evoluir(){
31         evento = gerarEvento();
32     }
33
34     public Evento observar(){
35         if (evento != null) {
36             evento.mostrar();
37         }
38         return evento;
39     }
40
41     public void executar(Comando comando){
42         comando.mostrar();
43     }
44
45     private EventoJogo gerarEvento() {
46         System.out.println(x:"\nEvento?");
47         String comando = scanner.next();
48         return eventos.get(comando);
49     }
50 }
```

Figura 42 - Classe Ambiente Jogo

A classe **AmbienteJogo** é uma realização da interface **Ambiente** e, por isso, especifica a implementação dos métodos **evoluir**, **observar** e **executar**. Tem uma relação de **dependência** com a classe **ComandoJogo** e uma relação de **associação** e **composição** com a classe **EventoJogo**. Como consequência da relação de composição com **EventoJogo**, é necessário criar uma estrutura de dados para conter eventos. De forma a simplificar a implementação do método **gerarEvento**, essa estrutura de dados é um dicionário, neste caso um *HashMap*, que associa as teclas que serão usadas pelo utilizador ao evento correspondente.

Contém o método **evoluir()**, que serve para afetar o atributo **evento** com um evento novo, gerado pelo método **gerarEvento()**; o método **gerarEvento()** que pergunta ao utilizador o evento para gerar, espera pelo *input* e retorna o evento correspondente ao selecionado pelo utilizador; o método **observar()** que mostra e retorna o evento atual; o método **executar()** que mostra o comando que vai ser executado no ambiente.

- Enumerado Evento Jogo

```
1  package jogo.ambiente;
2
3  import ambiente.Evento;
4
5  public enum EventoJogo implements Evento{
6      SILENCIO,
7      RUIDO,
8      ANIMAL,
9      FUGA,
10     FOTOGRAFIA,
11     TERMINAR;
12
13     @Override
14     public void mostrar(){
15         System.out.printf(format:"Evento: %s\n", this);
16     }
17 }
```

Figura 43 - Enumerado Evento Jogo

O enumerado **EventoJogo** é uma realização da interface **Evento**, que enumera todos os possíveis eventos no contexto deste jogo e que implementa o método **mostrar()** da interface.

Contém o método **mostrar()** que permite mostrar ao utilizador, através da consola, o evento atual.

- Enumerado Comando Jogo

```
1  package jogo.ambiente;
2
3  import ambiente.Comando;
4
5  public enum ComandoJogo implements Comando{
6      PROCURAR,
7      APROXIMAR,
8      OBSERVAR,
9      FOTOGRAFAR;
10
11     @Override
12     public void mostrar() {
13         System.out.printf(format:"Comando: %s\n", this);
14     }
15
16 }
```

Figura 44 - Enumerado Comando Jogo

O enumerado **ComandoJogo** é uma realização da interface **Comando**, que enumera todos os possíveis comandos que podem ser executados no contexto deste jogo e que implementa o método **mostrar()** da interface.

Contém o método **mostrar()** que permite mostrar ao utilizador, através da consola, o comando atual.

- Classe Personagem

```
1  package jogo.personagem;
2
3  import agente.Agente;
4  import jogo.ambiente.AmbienteJogo;
5
6  public class Personagem extends Agente{
7      public Personagem(AmbienteJogo ambiente) {
8          super(ambiente, new ControloPersonagem());
9      }
10 }
```

Figura 45 - Classe Personagem

A classe **Personagem** tem uma relação de **generalização** com a classe **Agente**, sendo uma **especialização** deste, fazendo uso do mecanismo de **factorização** designado como **herança**. O construtor da classe pai (**Agente**) tem de ser invocado no construtor da classe **Personagem**. É também um exemplo de **polimorfismo** porque apesar de **Personagem** ser um **Agente**, o seu construtor apresenta ligeiras diferenças.

- Classe MáquinaEstados

```
1  package maquest;
2
3  import agente.Accao;
4  import ambiente.Evento;
5
6  public class MaquinaEstados {
7      private Estado estado;
8
9
10     public MaquinaEstados(Estado estadoInicial) {
11         estado = estadoInicial;
12     }
13
14     public Estado getEstado() {
15         return estado;
16     }
17
18     public Accao processar(Evento evento){
19         Transicao transicao = estado.processar(evento);
20
21         if (transicao != null) {
22             estado = transicao.getEstadoSucessor();
23             return transicao.getAccao();
24         }
25
26         return null;
27     }
28 }
```

Figura 46 - Classe MáquinaEstados

Classe **MaquinaEstados** que representa a implementação de uma máquina de estados geral. No contexto do nosso jogo, o alfabeto de entrada é representado pelas teclas de input que o utilizador usa para indicar o evento observado e o conjunto de comandos definidos no enumerado **ComandoJogo** representa o alfabeto de saída. A função de transformação do sistema é implementada pela classe **Estado**.

Contém o método **processar()** que processa um evento a partir do estado atual e descobre a transição que lhes está associada. Partindo dessa transição, consegue descobrir o estado sucessor e a ação correspondentes.

- Classe Estado

```
1  package maquest;
2
3  import java.util.HashMap;
4  import java.util.Map;
5
6  import agente.Accao;
7  import ambiente.Evento;
8
9  public class Estado {
10     private String nome;
11     private Map<Evento, Transicao> transicoes;
12
13     public Estado(String nome) {
14         this.nome = nome;
15         transicoes = new HashMap<Evento, Transicao>();
16     }
17
18     public String getNome() {
19         return nome;
20     }
21
22     public Transicao processar(Evento evento) {
23         return transicoes.get(evento);
24     }
25
26     public Estado transicao(Evento evento, Estado estadoSucessor){
27         return transicao(evento, estadoSucessor, accao:null);
28     }
29
30     public Estado transicao(Evento evento, Estado estadoSucessor, Accao accao) {
31         Transicao transicao = new Transicao(estadoSucessor, accao);
32         transicoes.put(evento, transicao);
33         return this;
34     }
35 }
```

Figura 47 - Classe Estado

A classe **Estado** representa um estado no contexto de uma máquina de estados. O estado representa uma configuração particular do sistema num determinado momento. Por composição com a classe **Transicao**, a classe tem como atributo um dicionário que associa um evento a uma ou mais transições de estado.

Os métodos **transicao()**, são responsáveis por implementar a função de transformação do sistema. São exemplo de polimorfismo por terem o mesmo nome, mas implementações e parâmetros ligeiramente diferentes porque são usados em contextos diferentes. São exemplo de factorização porque o método mais abrangente realiza a maior parte da implementação e o método menos abrangente utiliza o outro método de modo a eliminar a redundância. O método menos abrangente implementa apenas a função **delta** em que a entrada é o evento, o estado atual é o próprio (*this*) e é definido o estado sucessor. O método mais abrangente implementa a função **lambda** em que a saída é a ação.

- Classe Transicao

```
1  package maqest;
2
3  import agente.Accao;
4
5  public class Transicao {
6      private Estado estadoSucessor;
7      private Accao accao;
8
9      public Transicao(Estado estadoSucessor, Accao accao) {
10         this.estadoSucessor = estadoSucessor;
11         this.accao = accao;
12     }
13
14     public Estado getEstadoSucessor() {
15         return estadoSucessor;
16     }
17
18     public Accao getAccao() {
19         return accao;
20     }
21 }
```

Figura 48 - Classe Transicao

No contexto de uma máquina de estados, os objetos desta classe são utilizados para definir as transições entre os estados do sistema, especificando o próximo estado a ser alcançado e a ação causada por essa transição.

- Classe ControloPersonagem

```
1  package jogo.personagem;
2
3  import agente.Accao;
4  import agente.Controlo;
5  import agente.Percepcao;
6  import ambiente.Evento;
7  import jogo.ambiente.ComandoJogo;
8  import jogo.ambiente.EventoJogo;
9  import maqest.Estado;
10 import maqest.MaquinaEstados;
11
12 public class ControloPersonagem implements Controlo{
13     private MaquinaEstados maqEst;
14
15     public ControloPersonagem(){
16         // definição do alfabeto de entrada
17         Estado procura = new Estado(nome:"Procura");
18         Estado inspeccao = new Estado(nome:"Inspeccao");
19         Estado observacao = new Estado(nome:"Observacao");
20         Estado registo = new Estado(nome:"Registo");
21
22         // definição do alfabeto de saida
23         Accao procurar = new Accao(ComandoJogo.PROCURAR);
24         Accao aproximar = new Accao(ComandoJogo.APROXIMAR);
25         Accao observar = new Accao(ComandoJogo.OBSERVAR);
26         Accao fotografar = new Accao(ComandoJogo.FOTOGRAFAR);
27
28         // definição das transições, ou seja, especificação da função de transformação
29         procura
30             .transicao(EventoJogo.RUIDO, inspeccao, aproximar)
31             .transicao(EventoJogo.SILENCIO, procura, procurar)
32             .transicao(EventoJogo.ANIMAL, observacao, aproximar);
33
34         inspeccao
35             .transicao(EventoJogo.SILENCIO, procura)
36             .transicao(EventoJogo.RUIDO, inspeccao, procurar)
37             .transicao(EventoJogo.ANIMAL, observacao, aproximar);
38
39         observacao
40             .transicao(EventoJogo.FUGA, inspeccao)
41             .transicao(EventoJogo.ANIMAL, registo, observar);
42
43         registo
44             .transicao(EventoJogo.FUGA, procura)
45             .transicao(EventoJogo.FOTOGRAFIA, procura)
46             .transicao(EventoJogo.ANIMAL, registo, fotografar);
47
48         maqEst = new MaquinaEstados(procura);
49     }
```

Figura 49 - Classe ControloPersonagem (Parte 1)


```
51     public Estado getEstado(){
52         return maqEst.getEstado();
53     }
54
55     @Override
56     public Accao processar(Percepcao percepcao) {
57         Evento evento = percepcao.getEvento();
58         Accao accao = maqEst.processar(evento);
59         mostrar();
60         return accao;
61     }
62
63     private void mostrar(){
64         System.out.printf(format:"Estado: %s\n", getEstado().getNome());
65     }
66
67 }
```

Figura 50 - Classe ControloPersonagem (Parte 2)

A classe **ControloPersonagem** é uma realização da interface e, por isso, especifica a implementação do método **processar()**. O controlo da personagem do nosso jogo é baseado numa máquina de estados, definida no construtor da classe. Começou-se por definir os alfabetos de entrada e saída e, em seguida, todas as transições associadas a cada estado.

O método **processar()** vai processar um evento obtido através de uma perceção, gerando uma ação com a ajuda da máquina de estados definida no construtor.

• Classe Jogo

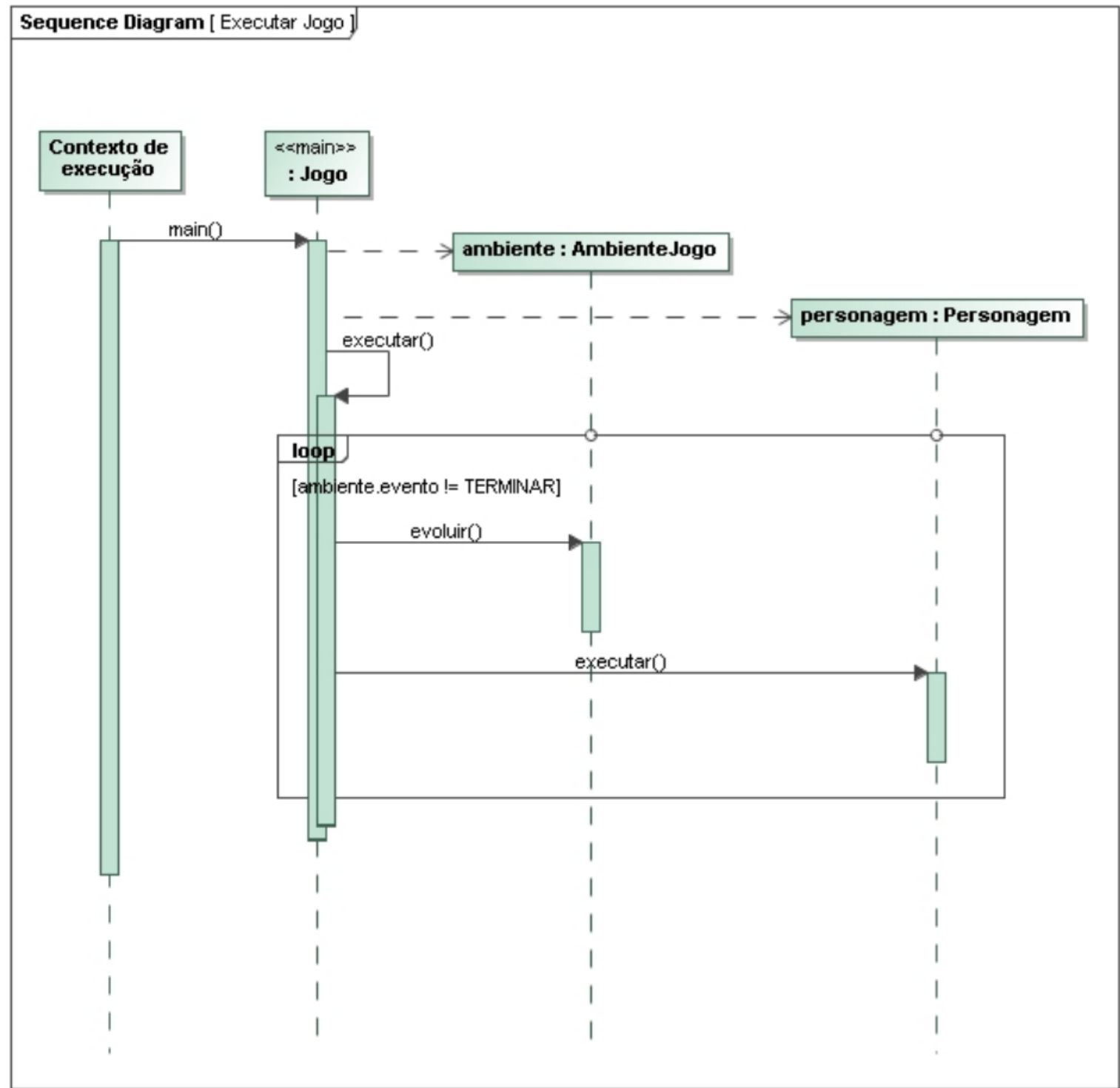


Figura 51 - Modelo de interação do Jogo

```
1 package jogo;
2
3 import jogo.ambiente.AmbienteJogo;
4 import jogo.ambiente.EventoJogo;
5 import jogo.personagem.Personagem;
6
7 public class Jogo {
8     private static AmbienteJogo ambiente;
9     private static Personagem personagem;
10
11     public static void main(String[] args) {
12         ambiente = new AmbienteJogo();
13         personagem = new Personagem(ambiente);
14         executar();
15     }
16
17     private static void executar(){
18         do{
19             ambiente.evolver();
20             personagem.executar();
21         } while(ambiente.getEvento() != EventoJogo.TERMINAR);
22     }
23 }
```

Figura 52 -Classe Jogo

A classe **Jogo** contém a lógica principal do jogo no método **main()** e no método auxiliar **executar()**.

O método **executar()** é um método privado, auxiliar ao método **main** que executa o motor de jogo. De acordo com o modelo de interação, este método invoca o método **evolver** do ambiente e o método **executar** da personagem, dentro de um *loop*. Optou-se por utilizar um loop do tipo *do-while* porque antes de se fazer a **verificação** da condição, que envolve saber se o evento do ambiente é igual a **Terminar**, é necessário que o ambiente evolua uma vez.

```
Evento?
r
Evento: RUIDO
Estado: Inspecao
Comando: APROXIMAR

Evento?
a
Evento: ANIMAL
Estado: Observacao
Comando: APROXIMAR

Evento?
o
Evento: FOTOGRAFIA
Estado: Observacao

Evento?
t
Evento: TERMINAR
Estado: Observacao
```

Figura 53 - Exemplo de uma jogada

4. PROJETO - PARTE 2

Na segunda parte do projeto, o objetivo foi a realização de um sistema autónomo inteligente capaz de navegar num espaço de dimensões discretas, com obstáculos e alvos, desviando-se dos obstáculos e recolhendo os alvos, desta vez, utilizando a linguagem Python. As direções dos movimentos e das perceções do sistema são: Norte, Sul, Este e Oeste (respetivamente, cima, baixo, direita, esquerda no ecrã).

Este tipo de agente inteligente designa-se **Agente Prospector**. O seu objetivo é recolher alvos, que foi modelado num comportamento chamado **Recolher**, e tem três sub-objetivos fulcrais para a concretização do objetivo: **aproximar alvo**, **evitar obstáculos** e **explorar**. O comportamento **Recolher** é, portanto, um comportamento composto que agrega três sub-comportamentos que correspondem aos três sub-objetivos.

O mecanismo de seleção de ação no caso do comportamento composto **Recolher** é uma hierarquia fixa cuja ordem é aproximar, evitar, explorar, sendo o aproximar o comportamento com mais prioridade dentro desta hierarquia.

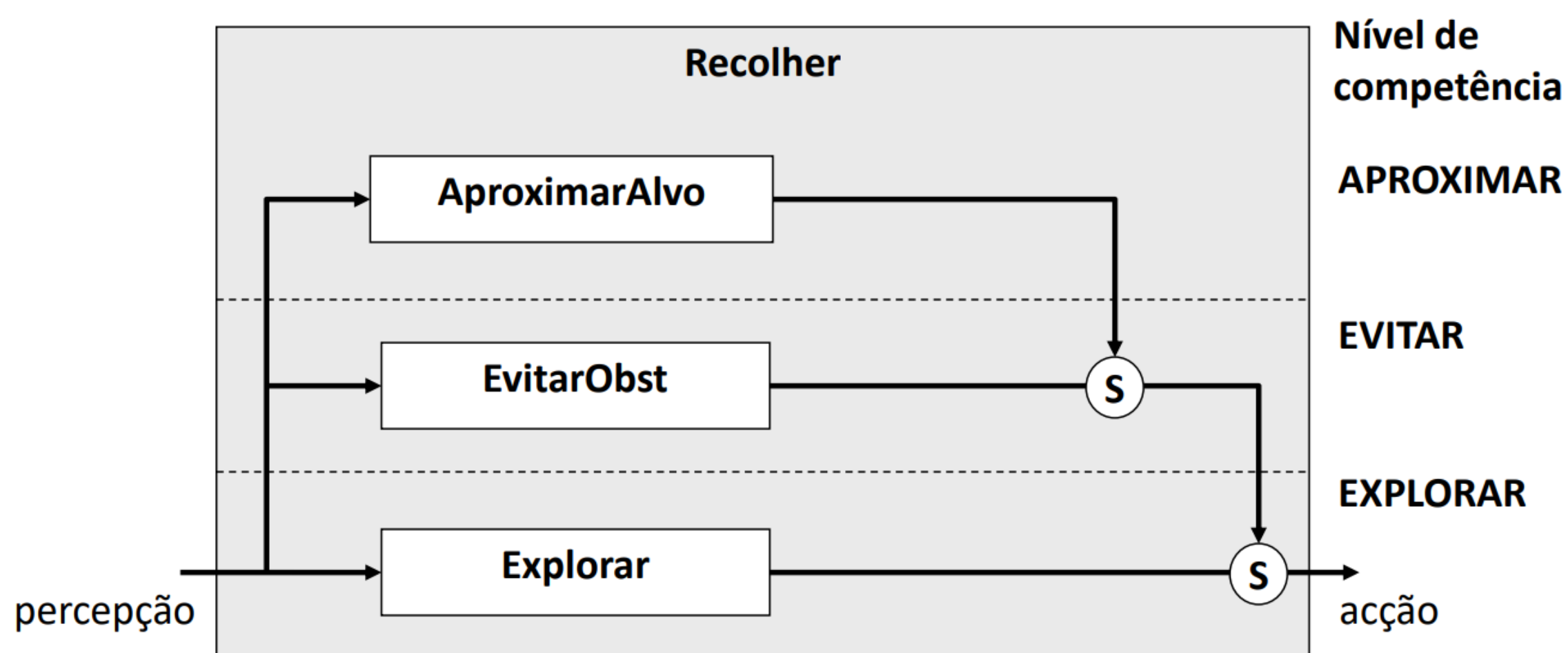


Figura 54 - Hierarquia do comportamento Recolher

Tendo em conta as características do agente, sabemos que este percebe e avança em quatro direções, o que nos leva a concluir que a certa altura terá de ser feita uma escolha relativamente à direção em que ele se deverá mover. Com isto em mente, os comportamentos aproximar e evitar também serão comportamentos compostos, organizados em quatro sub-comportamentos, um para cada direção. Neste caso, a seleção de ação será por prioridade, por exemplo, no caso do aproximar alvo, o sub-comportamento direcional com maior prioridade num dado momento será o que percecionou um alvo mais próximo.

4.1. Organização e Arquitetura do Projeto

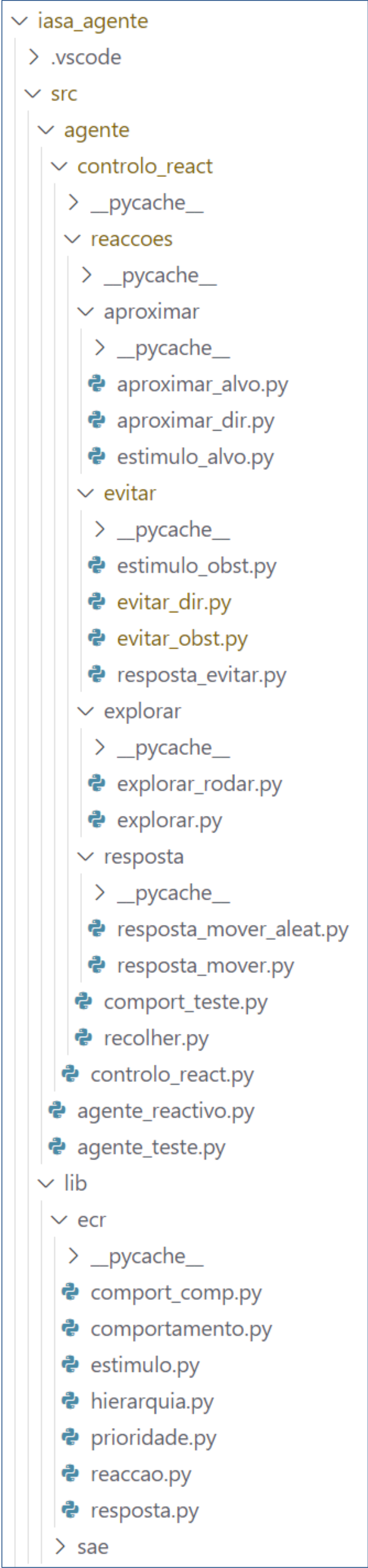


Figura 55 - Organização das pastas e módulos do projeto

• Subsistema Reação

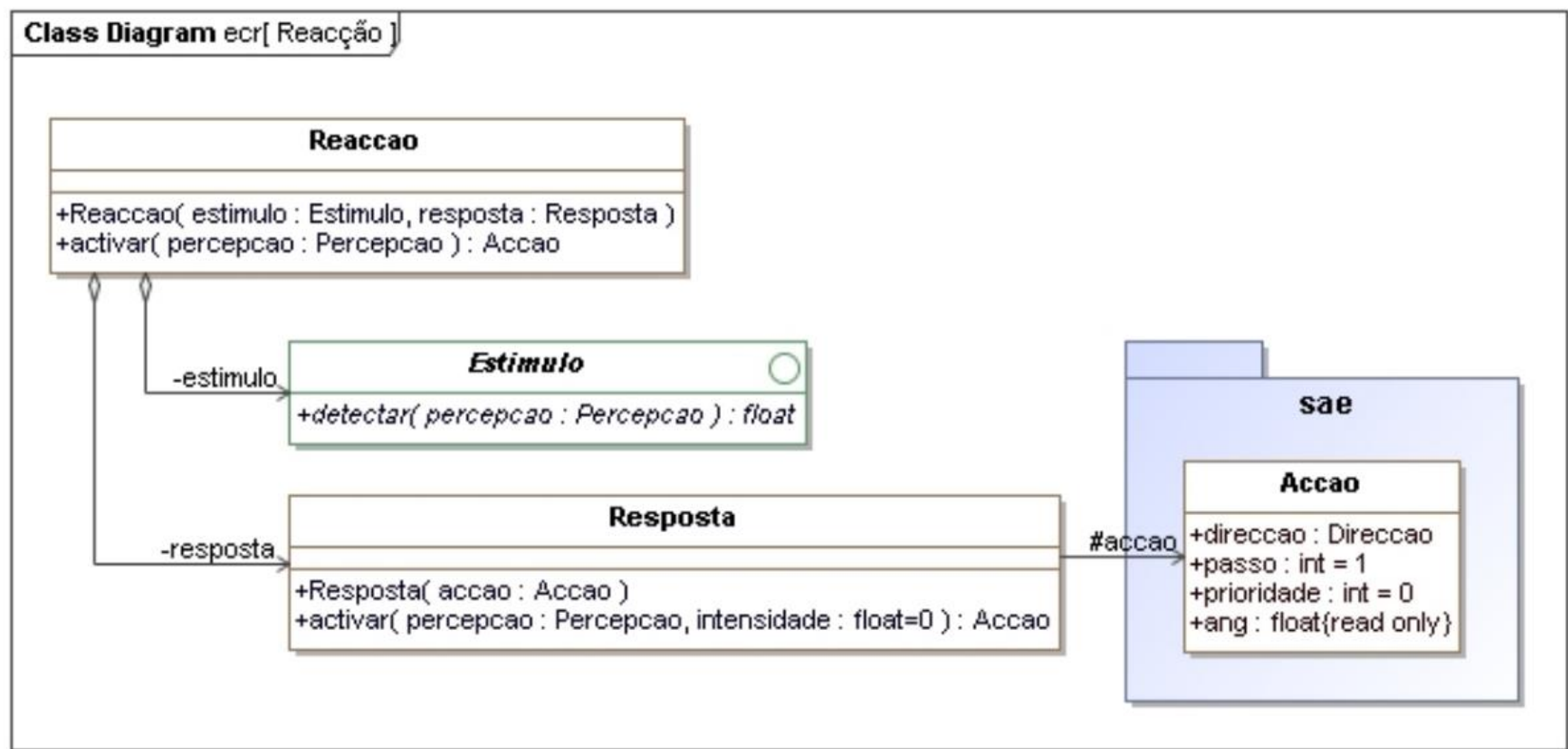


Figura 56 - Subsistema Reação

• Subsistema Comportamento

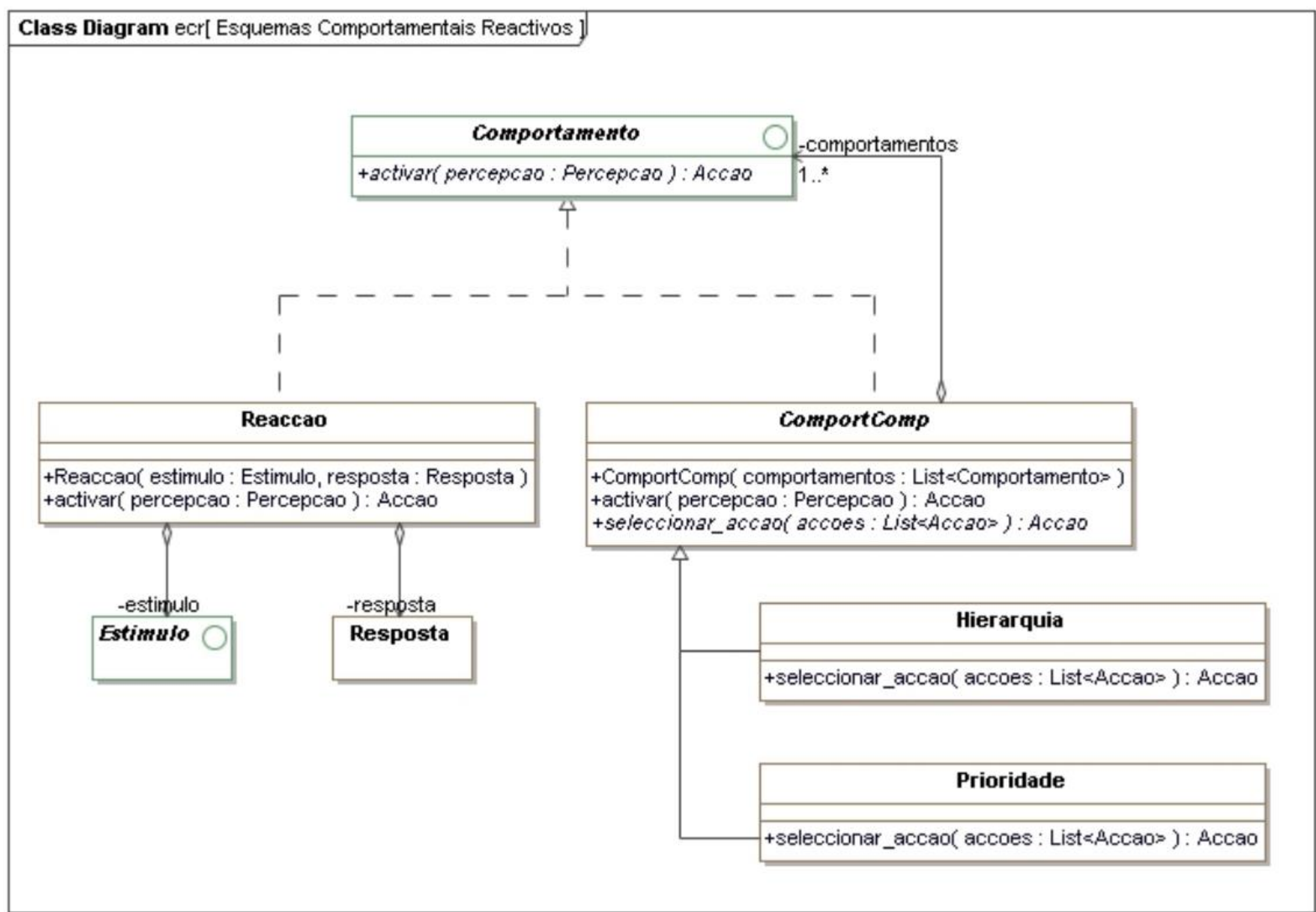


Figura 57 - Subsistema Comportamento

• Controlo Reativo

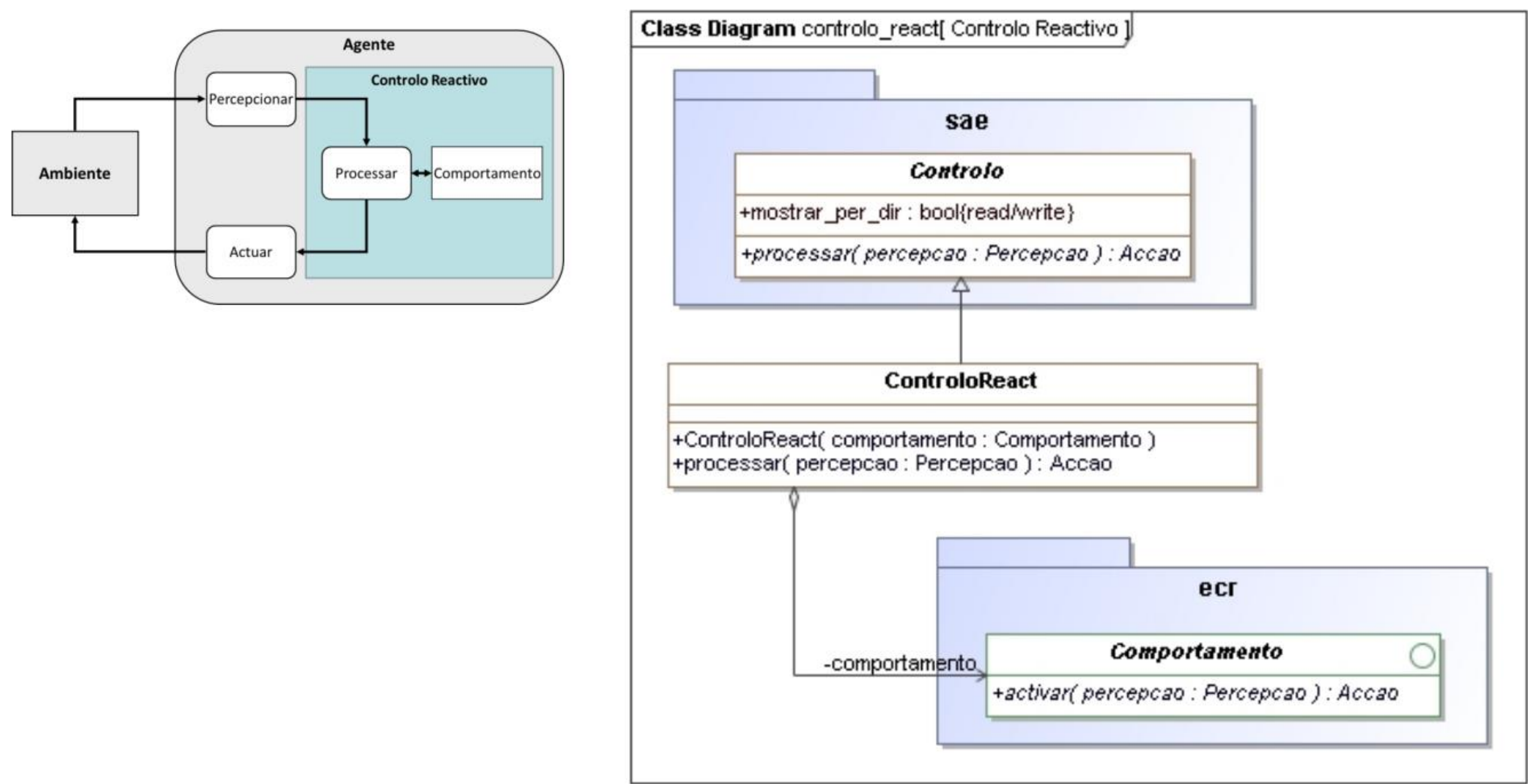


Figura 58 - Controlo Reativo

• Comportamento Explorar

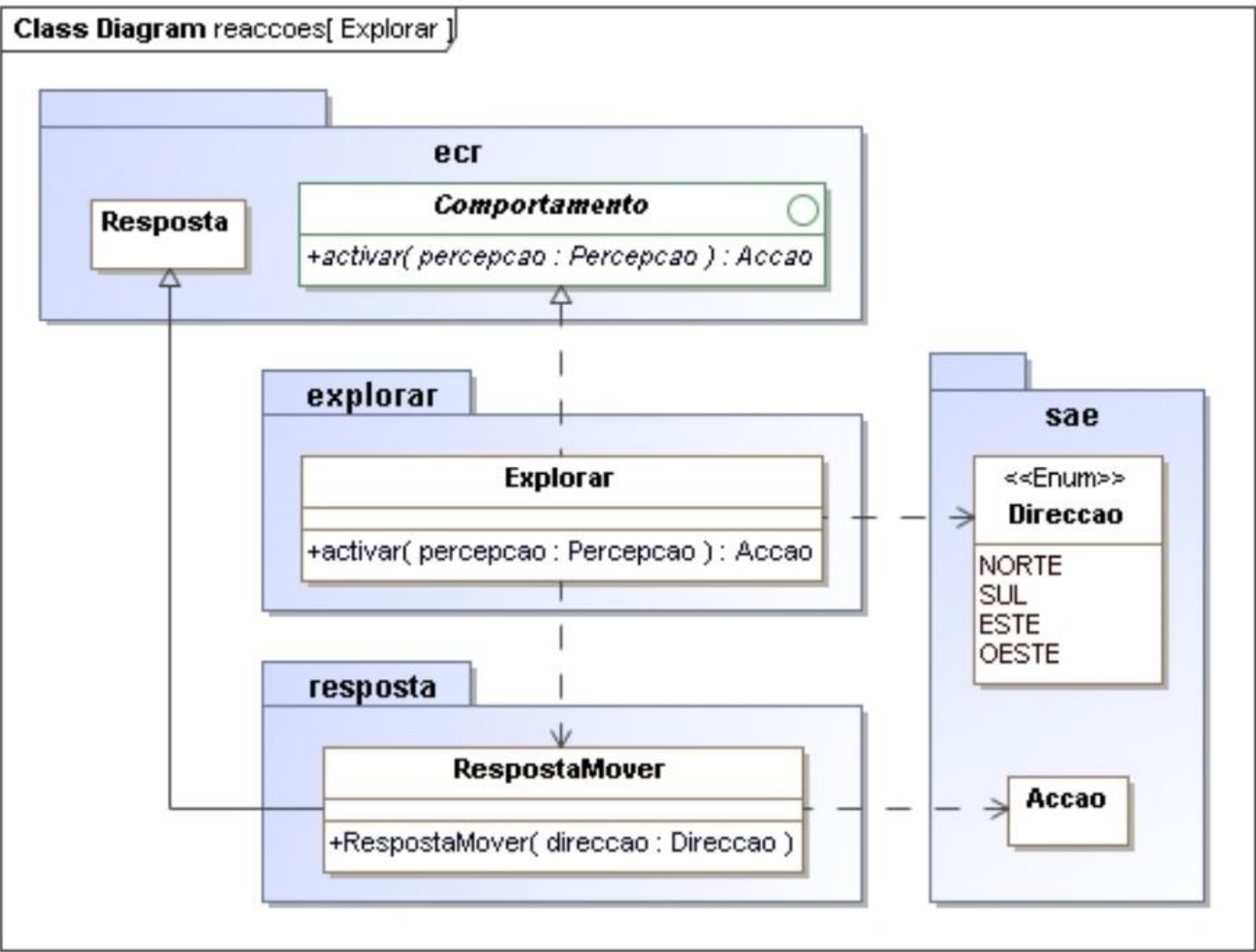


Figura 59 - Comportamento Explorar

- Comportamento Aproximar Alvo

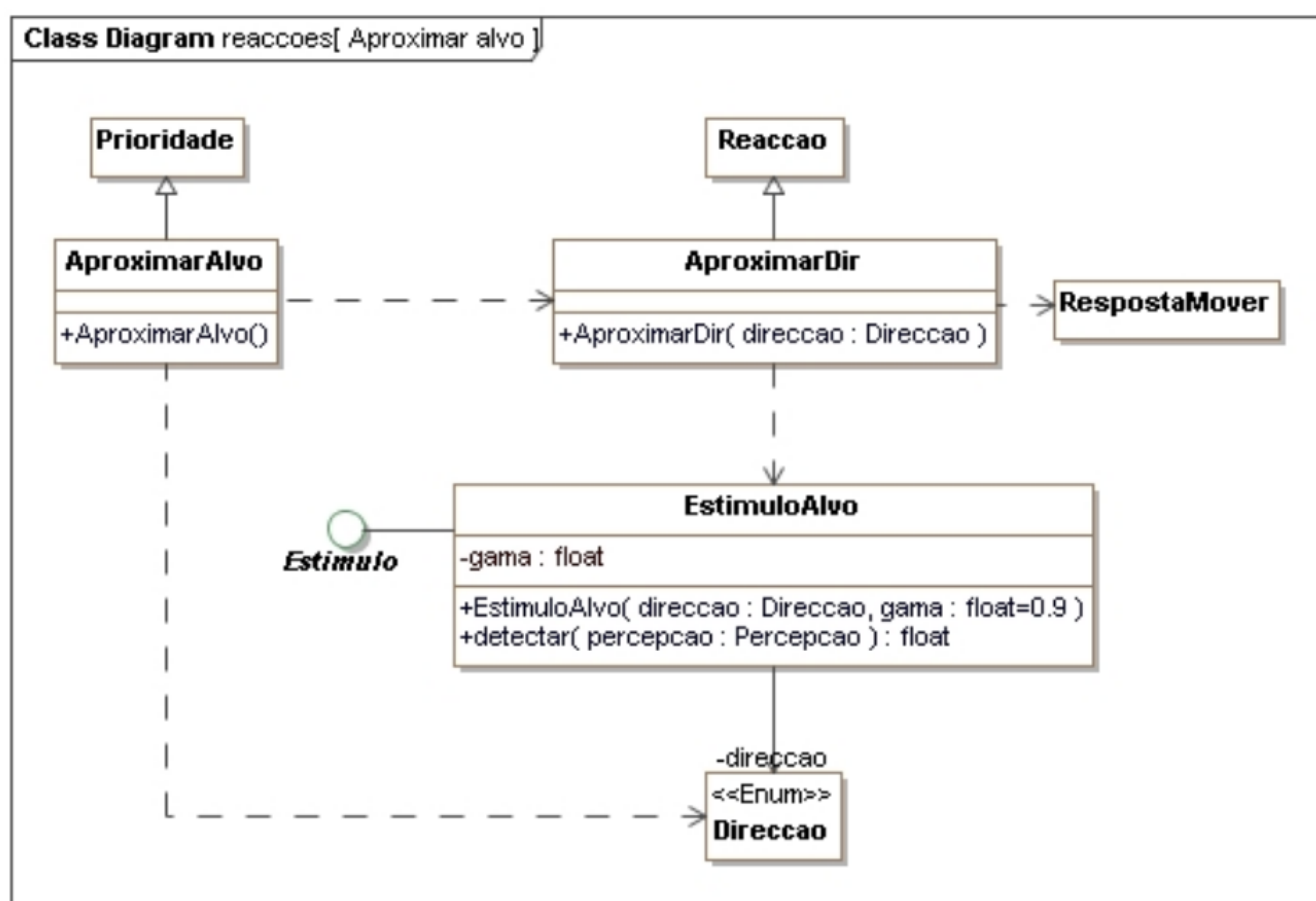


Figura 60 - Comportamento Aproximar Alvo

- Comportamento Evitar Obstáculos

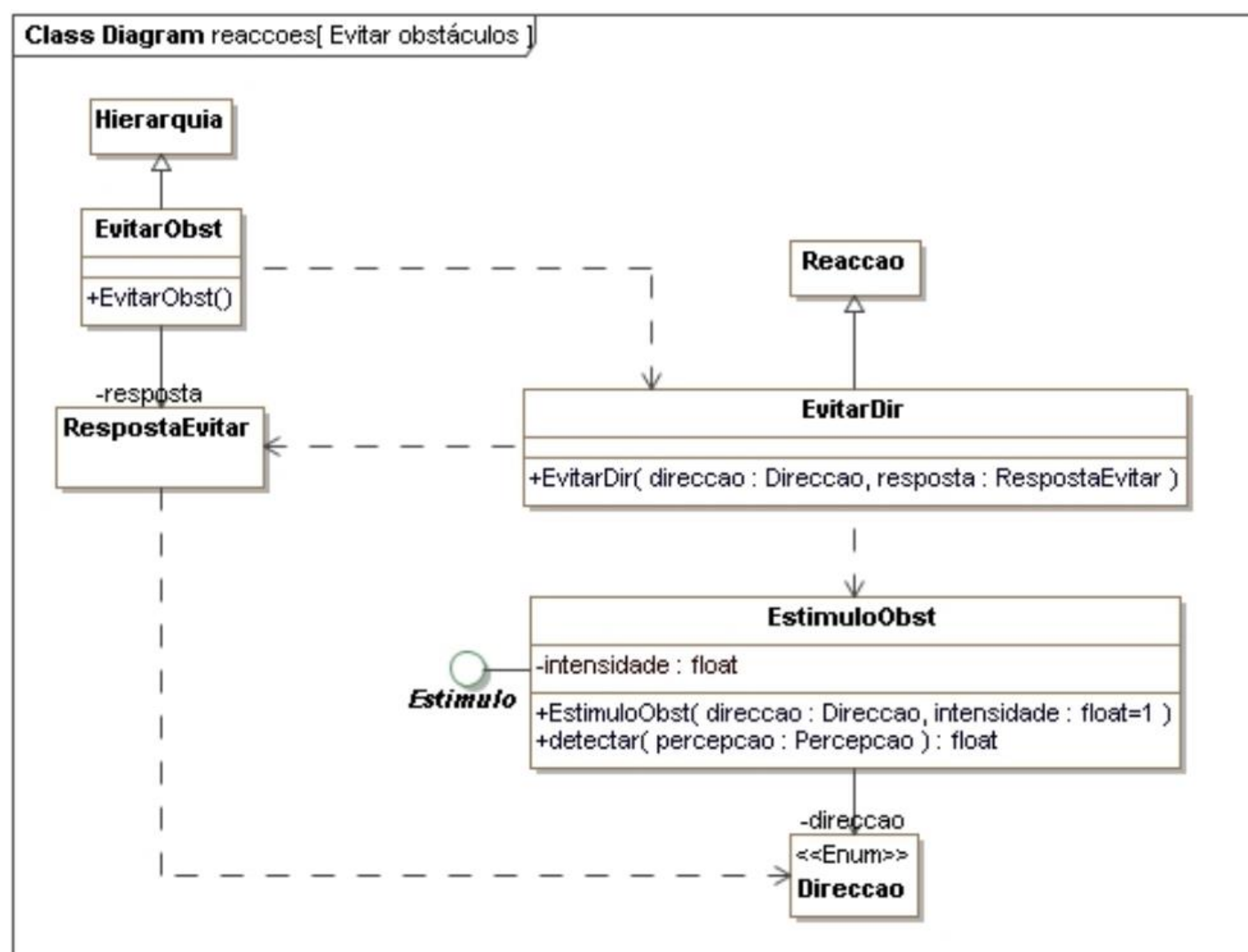


Figura 61 - Comportamento Evitar Obstáculos

- Respostas a estímulos

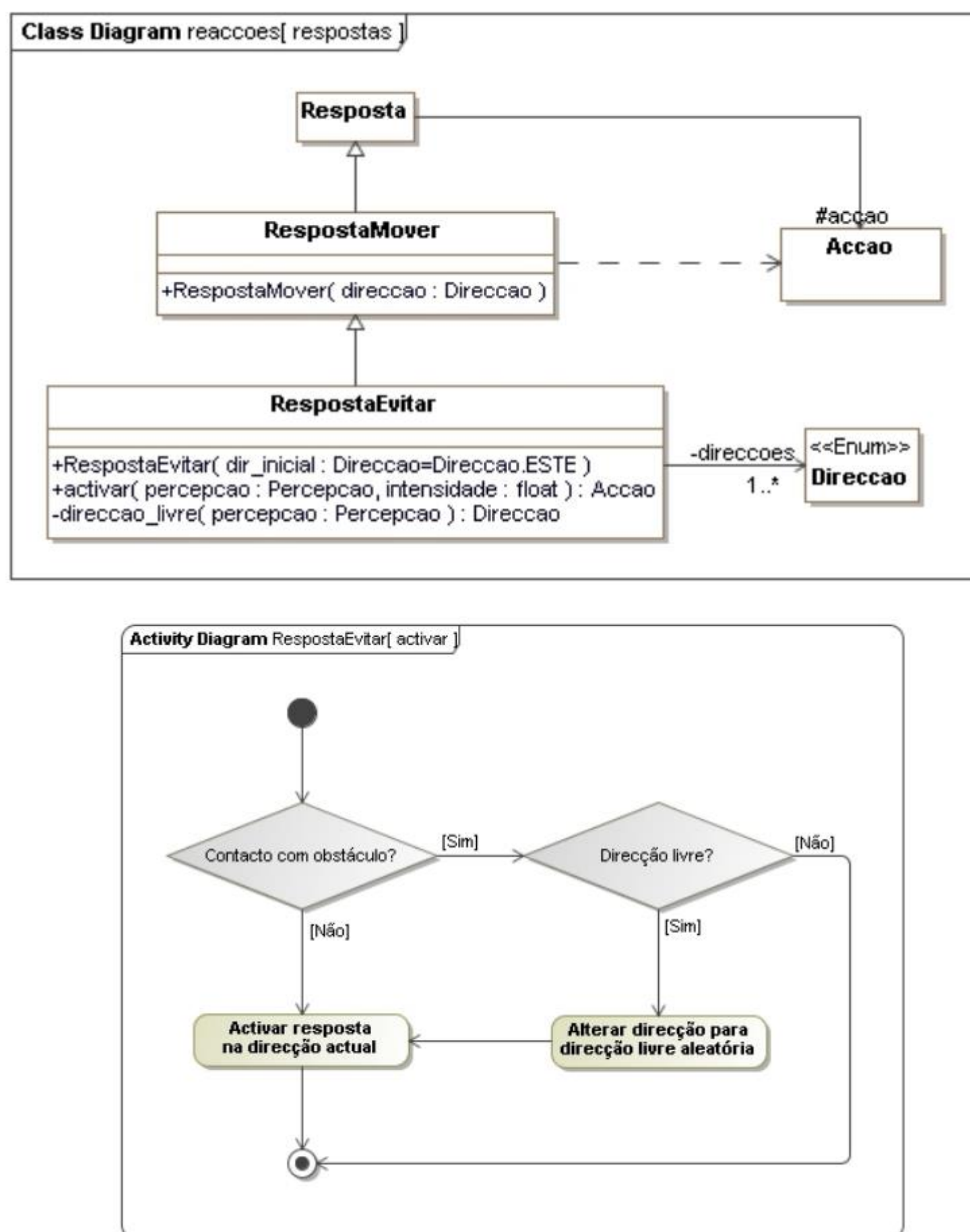


Figura 62 - Respostas a estímulos

4.2. Implementação

- Interface Comportamento

```

1  from abc import ABC, abstractmethod
2
3  class Comportamento(ABC):
4
5      @abstractmethod
6      def activar(self, percepcao):
7          .....
```

Figura 63 - Interface Comportamento

A interface **Comportamento** representa um comportamento no contexto de agentes reativos. Um agente reativo pode ser composto por várias reações, mas, de forma a reduzir complexidade da arquitetura, aumentar a coesão e reduzir o acoplamento, estas reações são **modularizadas** em comportamentos. Um comportamento é um conjunto de reações que possuem semelhança de propósito. Se um comportamento for composto por apenas uma reação é um **comportamento simples** (classe **Reacao**). Se tiver várias reações ou outros comportamentos é um **comportamento composto** (classe **ComportComp**).

- Classe Reacao

```

1  from .comportamento import Comportamento
2
3  class Reacao(Comportamento):
4
5      def __init__(self, estimulo, resposta):
6          self.__estimulo = estimulo
7          self.__resposta = resposta
8
9      def activar(self, percepcao):
10         intensidade = self.__estimulo.detectar(percepcao)
11         if intensidade > 0:
12             return self.__resposta.activar(percepcao, intensidade)
```

Figura 64 - Classe Reacao

Esta classe representa uma **reação**. Uma reação é um módulo que associa estímulos a respostas e vai encapsular esta associação. Esta classe é também um comportamento e por isso é uma realização da interface **Comportamento**.

Contém o método **activar**, que ao detetar uma perceção com intensidade maior que 0, vai ativar a resposta associada a esta reação, gerando a ação correspondente.

- Interface Estimulo

```
1  from abc import ABC, abstractmethod
2
3  class Estimulo(ABC):
4      @abstractmethod
5      def detectar(self, percepcao):
6          """Detetar estimulo"""
7
```

Figura 65 - Interface Estimulo

A interface **Estímulo** representa a deteção de um estímulo presente numa perceção. Um estímulo define informação ativadora de uma reação.

- Classe Resposta

```
1  class Resposta:
2
3      def __init__(self, accao):
4          self._acao = accao
5
6      def activar(self, percepcao, intensidade = 0):
7          if percepcao is not None:
8              self._acao.prioridade = intensidade
9              return self._acao
10
```

Figura 66 - Classe Resposta

A classe **Resposta** representa a geração de uma resposta a um estímulo. Uma resposta define a ação a realizar e a sua respetiva prioridade, em resposta a um estímulo.

Contém o método **ativar** que, caso exista perceção, vai definir a prioridade da própria resposta com a intensidade da perceção e retorna a ação associada à resposta.

- Classe ComportComp

```
1  from .comportamento import Comportamento
2  from abc import abstractmethod
3
4  class ComportComp(Comportamento):
5
6      def __init__(self, comportamentos):
7          self.__comportamentos = comportamentos
8
9
10     def activar(self, percepcao):
11         accoes = []
12         for comportamento in self.__comportamentos:
13             accao = comportamento.activar(percepcao)
14             if accao:
15                 accoes.append(accao)
16
17         if accoes:
18             return self.seleccionar_accao(accoes)
19
20
21     @abstractmethod
22     def seleccionar_accao(self, accoes):
23         .....
```

Figura 67 - Classe ComportComp

Esta classe é uma classe abstrata e é realização da interface **Comportamento**, ou seja, especificação de um comportamento, neste caso um comportamento composto. Tem um atributo **comportamentos** que é uma lista de comportamentos.

Contém um método **ativar** que vai ativar todos os comportamentos na lista, preenchendo assim uma lista de ações. Se houver ações na lista, uma vai ser selecionada e retornada, com ajuda do método **seleccionar_acao**. Este método é abstrato pois vai ser implementado pelas classes **Prioridade** e **Hierarquia**, onde vai ser definido o método de seleção de ação.

- Classe Hierarquia

```
1  from src.lib.ecr.comport_comp import ComportComp
2  from .comport_comp import ComportComp
3
4  class Hierarquia(ComportComp):
5
6      def seleccionar_acciao(self, accoes):
7          if accoes:
8              accao_sel = accoes[0]
9              return accao_sel
```

Figura 68 - Classe Hierarquia

A classe **Hierarquia** representa um comportamento composto (especialização da classe **ComportComp**) cujo mecanismo de seleção de ação é designado por hierarquia.

O método **seleccionar_acciao** é responsável por selecionar a ação com base numa lista de ações. Neste caso, partindo do pressuposto que a lista de ações é construída de modo que a lista esteja ordenada de acordo com a prioridade das ações, de forma decrescente, a ação selecionada é a primeira da lista, pois é a que tem maior prioridade.

- Classe Prioridade

```
1  from .comport_comp import ComportComp
2
3  class Prioridade(ComportComp):
4
5      def seleccionar_acciao(self, accoes):
6          if accoes:
7              accao_sel = max(accoes, key=lambda accao: accao.prioridade)
8              return accao_sel
```

Figura 69 - Classe Prioridade

A classe **Prioridade** representa um comportamento composto (especialização da classe **ComportComp**) cujo mecanismo de seleção de ação é designado por prioridade.

O método **seleccionar_acciao** é responsável por selecionar a ação, com base numa lista de ações. Utiliza a função *max*, que retorna o valor máximo num conjunto de valores que, com auxílio da função *lambda*, vai retornar a ação com maior prioridade. A função *lambda* é uma função anónima *inline*, usada em situações onde é necessário uma pequena função para auxiliar uma função maior, útil para implementar código conciso, evitando a definição de uma função inteira. Requer a utilização da *keyword lambda* e recebe um argumento (uma ação) e uma expressão para ser avaliada (a prioridade da ação).

- Classe ControloReact

```
1  from sae import Controlo
2
3  class ControloReact(Controlo):
4
5      def __init__(self, comportamento):
6          self.__comportamento = comportamento
7          self.mostrar_per_dir = True
8
9      def processar(self, percecao):
10         return self.__comportamento.activar(percecao)
```

Figura 70 - Classe ControloReact

A classe **ControloReact** representa o processamento interno de um agente reativo. Esta classe é uma especialização da classe **Controlo** e por isso implementa o método **processar**. Este método é responsável por associar uma perceção a uma ação, através de um comportamento.

- Classe Explorar

```
1  from ecr.comportamento import Comportamento
2  from ..resposta.resposta_mover_aleat import RespostaMoverAleat
3
4  class Explorar(Comportamento):
5
6      def activar(self, percecao):
7          resposta = RespostaMoverAleat()
8          return resposta.activar(percecao)
```

Figura 71 - Classe Explorar

A classe **Explorar** representa um comportamento em que o agente se move aleatoriamente pelo ambiente. A classe é uma realização da classe **Comportamento** e por isso implementa o método **ativar**. O comportamento explorar é um comportamento fixo, ou seja, é um comportamento que produz uma resposta sem necessitar de um estímulo.

O método **ativar** ativa o comportamento, isto é, ativa a resposta com base numa perceção, gerando uma ação. Neste caso a resposta é uma **RespostaMoverAleat**, pois gera ações com uma direção aleatória.

- Classe AproximarAlvo

```

1  from ecr.prioridade import Prioridade
2  from .aproximar_dir import AproximarDir
3  from sae import Direccao
4
5  class AproximarAlvo(Prioridade):
6      __comportamentos = [AproximarDir(Direccao.NORTE),
7                          AproximarDir(Direccao.SUL),
8                          AproximarDir(Direccao.ESTE),
9                          AproximarDir(Direccao.OESTE)]
10
11     def __init__(self):
12         super().__init__(self.__comportamentos)

```

Figura 72 - Classe AproximarAlvo

A classe **AproximarAlvo** representa um comportamento de um agente reativo, cujo objetivo é aproximar-se de um alvo. Este comportamento é um comportamento composto cujo mecanismo de seleção de ação é a prioridade. Por isso, esta classe é uma especialização da classe **Prioridade**. Este comportamento terá 4 sub-comportamentos, cada um deles uma instância da classe **AproximarDir**, para cada uma das direções possíveis do movimento do agente. Neste caso, a ordem da lista de comportamentos é indiferente pois o mecanismo de seleção é a prioridade e esta vai ser definida pela **distancia** do agente ao alvo.

- Classe AproximarDir

```

1  from ecr.reacao import Reacao
2  from ..resposta.resposta_mover import RespostaMover
3  from .estimulo_alvo import EstimuloAlvo
4
5  class AproximarDir(Reacao):
6      def __init__(self, direccao):
7          super().__init__(EstimuloAlvo(direccao), RespostaMover(direccao))

```

Figura 73 - Classe AproximarDir

A classe **AproximarDir** representa uma reação (comportamento simples) que, neste caso, vai associar um estímulo (**EstimuloAlvo**, os alvos) a uma resposta (**RespostaMover**, movimento numa determinada direção). Esta classe é uma especialização da classe **Reacao**. Esta classe associa um estímulo numa determinada direção, a uma resposta nessa mesma direção, invocando o construtor da classe **Reacao**.

- Classe EstimuloAlvo

```

1  from ecr.estimulo import Estimulo
2  from sae import Elemento
3
4  class EstimuloAlvo(Estimulo):
5
6      def __init__(self, direccao, gama = 0.9):
7          self.__direccao = direccao
8          self.__gama = gama
9
10     def detectar(self, percepcao):
11         elemento, distancia, _ = percepcao[self.__direccao]
12         if elemento == Elemento.ALVO:
13             intensidade = self.__gama ** distancia
14         else:
15             intensidade = 0
16         return intensidade

```

Figura 74 - Classe EstimuloAlvo

A classe **EstimuloAlvo** implementa a interface **Estimulo**. Representa um estímulo que está associado a existência de um alvo, numa determinada direção.

O método **detetar** deteta ou não o alvo, caso detete calcula a intensidade do estímulo, quanto maior a distância, menor a intensidade. Caso não detete alvo, a intensidade é zero.

- Classe EvitarObst

```

1  from ecr.hierarquia import Hierarquia
2  from .resposta_evitar import RespostaEvitar
3  from .evitar_dir import EvitarDir
4  from sae import Direccao
5
6  class EvitarObst(Hierarquia):
7
8      def __init__(self):
9          self.__resposta = RespostaEvitar()
10         super().__init__([EvitarDir(direccao, self.__resposta) for direccao in Direccao])

```

Figura 75 - Classe EvitarObst

A classe **EvitarObst** representa um comportamento de um agente reativo, cujo objetivo é evitar obstáculos. Este comportamento é um comportamento composto cujo mecanismo de seleção de ação é a hierarquia. Por isso, esta classe é uma especialização da classe **Hierarquia**. Este comportamento terá 4 sub-comportamentos, cada um deles uma instância da classe **EvitarDir**, para cada uma das direções possíveis do movimento do agente.

- Classe EvitarDir

```
1  from ecr.reacao import Reacao
2  from .estimulo_obst import EstimuloObst
3
4  class EvitarDir(Reacao):
5      def __init__(self, direccao, resposta):
6          super().__init__(EstimuloObst(direccao), resposta)
```

Figura 76 - Classe EvitarDir

A classe **EvitarDir** representa uma reação que, neste caso vai associar um estímulo (**EstimuloObst**, os obstáculos) a uma resposta (**RespostaEvitar**, movimento numa direção livre de obstáculos). Esta classe é uma especialização da classe **Reacao**.

- Classe EstimuloObst

```
1  from ecr.estimulo import Estimulo
2
3  class EstimuloObst(Estimulo):
4
5      def __init__(self, direccao, intensidade=1):
6          self.__direccao = direccao
7          self.__intensidade = intensidade
8
9      def detectar(self, percepcao):
10         if percepcao.contacto_obst(self.__direccao):
11             intensidade = self.__intensidade
12         else:
13             intensidade = 0
14         return intensidade
```

Figura 77 - Classe EstimuloObst

A classe **EstimuloObst** implementa a interface **Estimulo**. Representa um estímulo que está associado à existência de um obstáculo, numa determinada direção.

O método **detectar** deteta ou não o obstáculo, caso detete, a intensidade do estímulo é máxima, corresponde á intensidade iniciada no construtor, definida a 1 por omissão. Caso não detete obstáculo, a intensidade é zero.

- Classe RespostaMover

```

1  from ecr.resposta import Resposta
2  from sae import Accao
3
4  class RespostaMover(Resposta):
5
6      def __init__(self, direccao):
7          super().__init__(Accao(direccao))

```

Figura 78 - Classe RespostaMover

A classe **RespostaMover**, é uma especialização da classe **Resposta**. O que a diferencia é que a ação que esta classe tem associada tem uma direção específica, que é passada no construtor.

- Classe RespostaEvitar

```

1  from ..resposta.resposta_mover import RespostaMover
2  from sae import Direccao
3  from random import choice
4
5  class RespostaEvitar(RespostaMover):
6
7      def __init__(self, dir_inicial=Direccao.ESTE):
8          self.__direccoes = list(Direccao)
9          super().__init__(dir_inicial)
10
11      def activar(self, percepcao, intensidade):
12          if percepcao.contacto_obst(self._acciao.direccao):
13              if direccao_livre := self.__direccao_livre(percepcao):
14                  self._acciao.direccao = direccao_livre
15              else:
16                  return None
17
18          return super().activar(percepcao, intensidade)
19
20      def __direccao_livre(self, percepcao):
21          direccoes_livres = [direccao for direccao in self.__direccoes
22                             if not percepcao.contacto_obst(direccao)]
23          if direccoes_livres:
24              return choice(direccoes_livres)

```

Figura 79 - Classe RespostaEvitar

A classe **RespostaEvitar** é uma especialização da classe **RespostaMover**, pois a sua ação também é um movimento numa determinada direção. No entanto, o que a diferencia é que esta classe dá *override* ao método **activar** da classe **Resposta**, pois esta resposta só é ativada caso haja contacto com um obstáculo. Se houver, procura-se uma direção livre, com o auxílio do método **direccao_livre**, altera-se a direção da ação para essa direção livre e só depois se ativa a resposta. O método **direccao_livre** cria uma lista de direções onde não existe contacto com um obstáculo e retorna uma delas aleatoriamente.

- Classe Recolher

```

1  from ecr.hierarquia import Hierarquia
2  from .explorar.explorar import Explorar
3  from .aproximar.aproximar_alvo import AproximarAlvo
4  from .evitar.evitar_obst import EvitarObst
5
6  class Recolher(Hierarquia):
7      __comportamentos = [AproximarAlvo(), EvitarObst(), Explorar()]
8
9      def __init__(self):
10         super().__init__(self.__comportamentos)

```

Figura 80 - Classe Recolher

A classe **Recolher** representa um comportamento de um agente reativo, cujo objetivo é recolher alvos. Este comportamento é um comportamento composto cujo mecanismo de seleção de ação é a hierarquia e, por isso, esta classe é uma realização da classe **Hierarquia**. Existem três sub-objetivos: aproximar alvo, evitar obstáculos e explorar. Por isso, vão existir também três sub-comportamentos, neste caso: **AproximarAlvo**, **EvitarObst** e **Explorar**. O sub-comportamento com maior prioridade é o **AproximarAlvo** e o com menor é o **Explorar**.

O construtor da classe ativa o construtor da superclasse **ComportComp**. A superclasse necessita de receber uma lista de comportamentos e, como esta classe é uma hierarquia, essa lista tem de estar ordenada de acordo com a prioridade de cada comportamento. Essa lista é definida no atributo da classe `__comportamentos`.

- Classe AgenteReactivo

```

1  from sae import Agente, Simulador
2  from agente.controlo_react.controlo_react import ControloReact
3  from agente.controlo_react.reaccoes.recolher import Recolher
4
5  class AgenteReactivo(Agente):
6
7      def __init__(self):
8          comportamento = Recolher()
9          controle = ControloReact(comportamento)
10         super().__init__(controle)
11
12     # Executar simulação
13     if __name__ == '__main__':
14         Simulador(1, AgenteReactivo()).executar()

```

Figura 81 - Classe AgenteReactivo

A classe **AgenteReactivo** representa um agente reativo cujo objetivo é explorar um ambiente e recolher alvos. O comportamento deste agente é o **Recolher** e o seu controlo é o **ControloReact**. É neste módulo que está a função *main* que corre a aplicação.

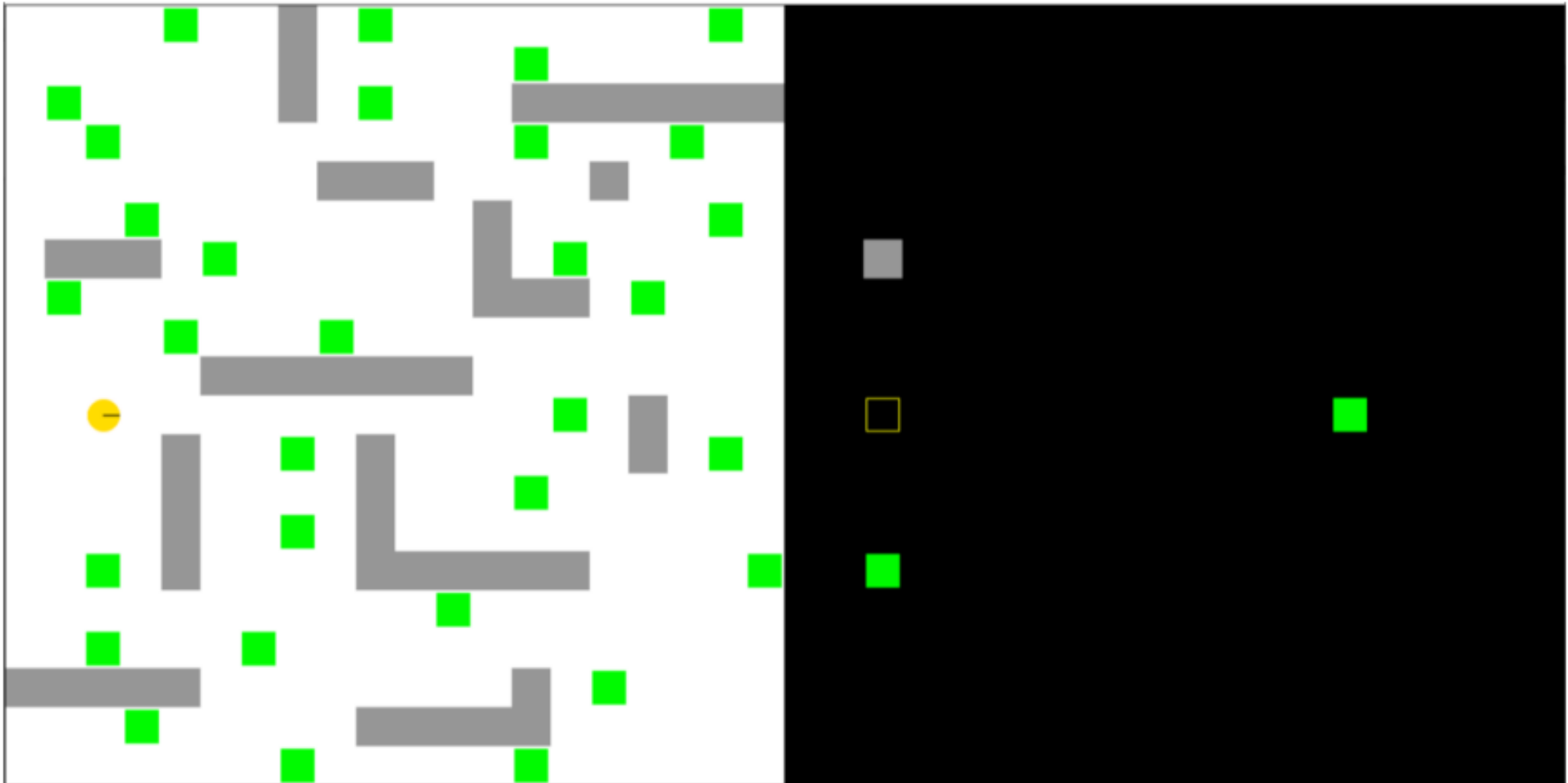


Figura 82 - Aplicação Agente Reativo

5. PROJETO – PARTE 3

Na terceira parte do projeto, o objetivo foi a implementação de uma biblioteca de raciocínio automático, nomeadamente, de modelação de problemas e de procuras em espaço de estados, com o intuito de oferecer a capacidade de resolver desde problemas simples, como um problema de contagem, até problemas complexos, como problemas de planeamento de trajetos.

5.1. Organização e Arquitetura do Projeto

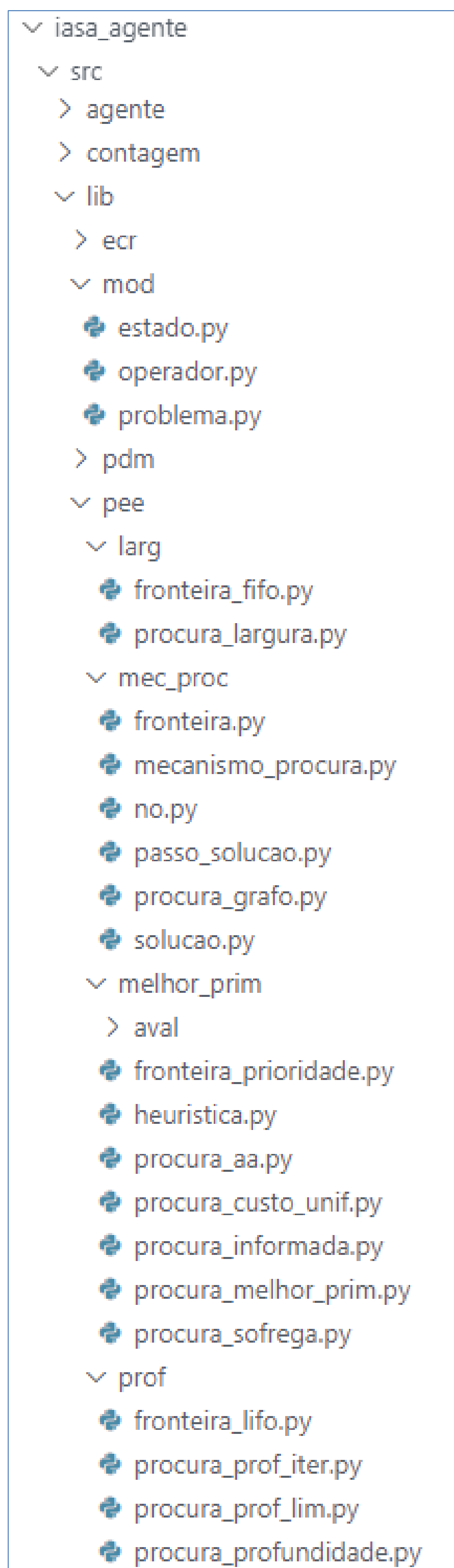


Figura 83 - Organização das pastas e módulos do projeto

• Modelo de problema

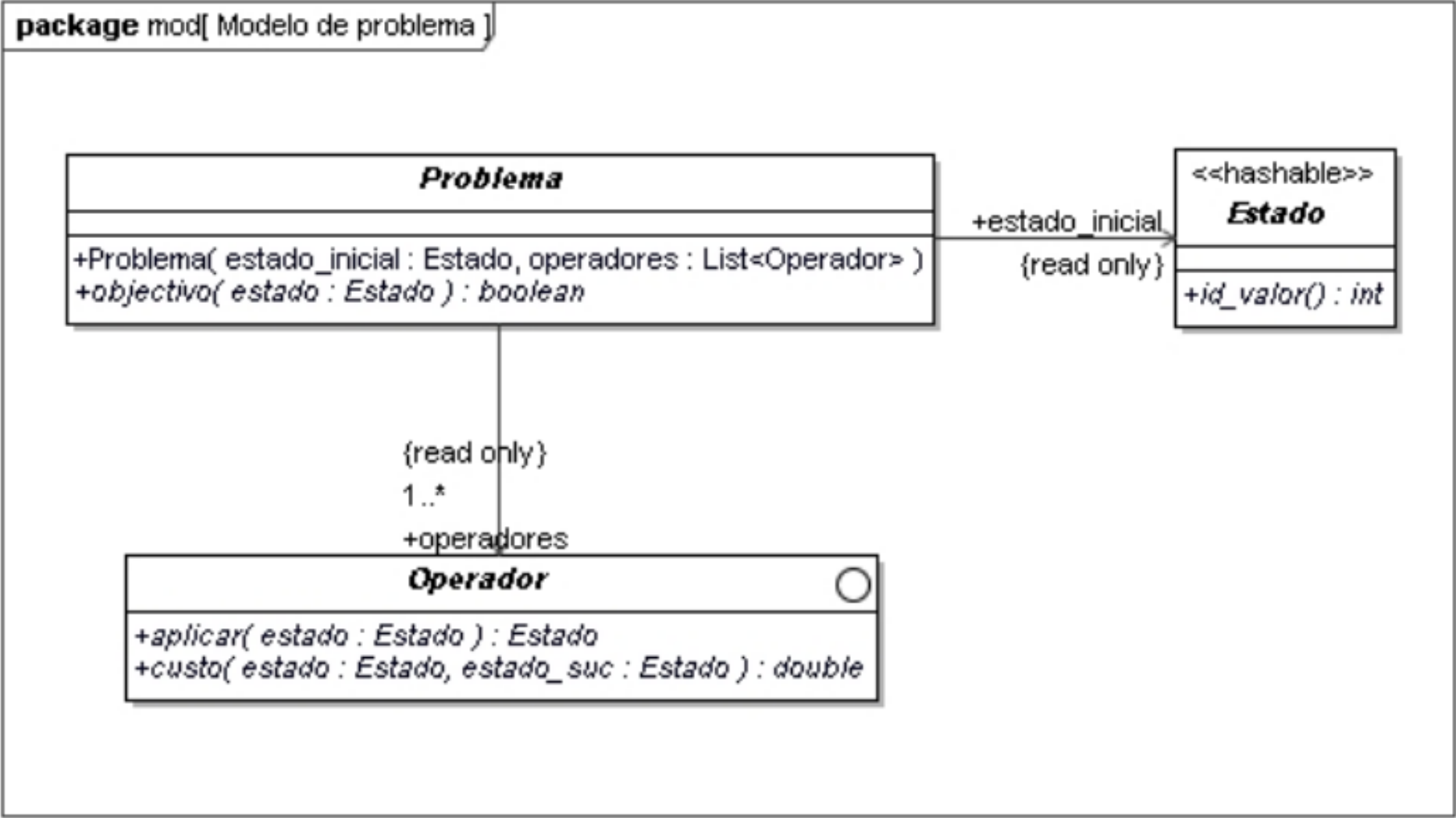


Figura 84 - Modelo de problema

• Sistema mecanismo de procura

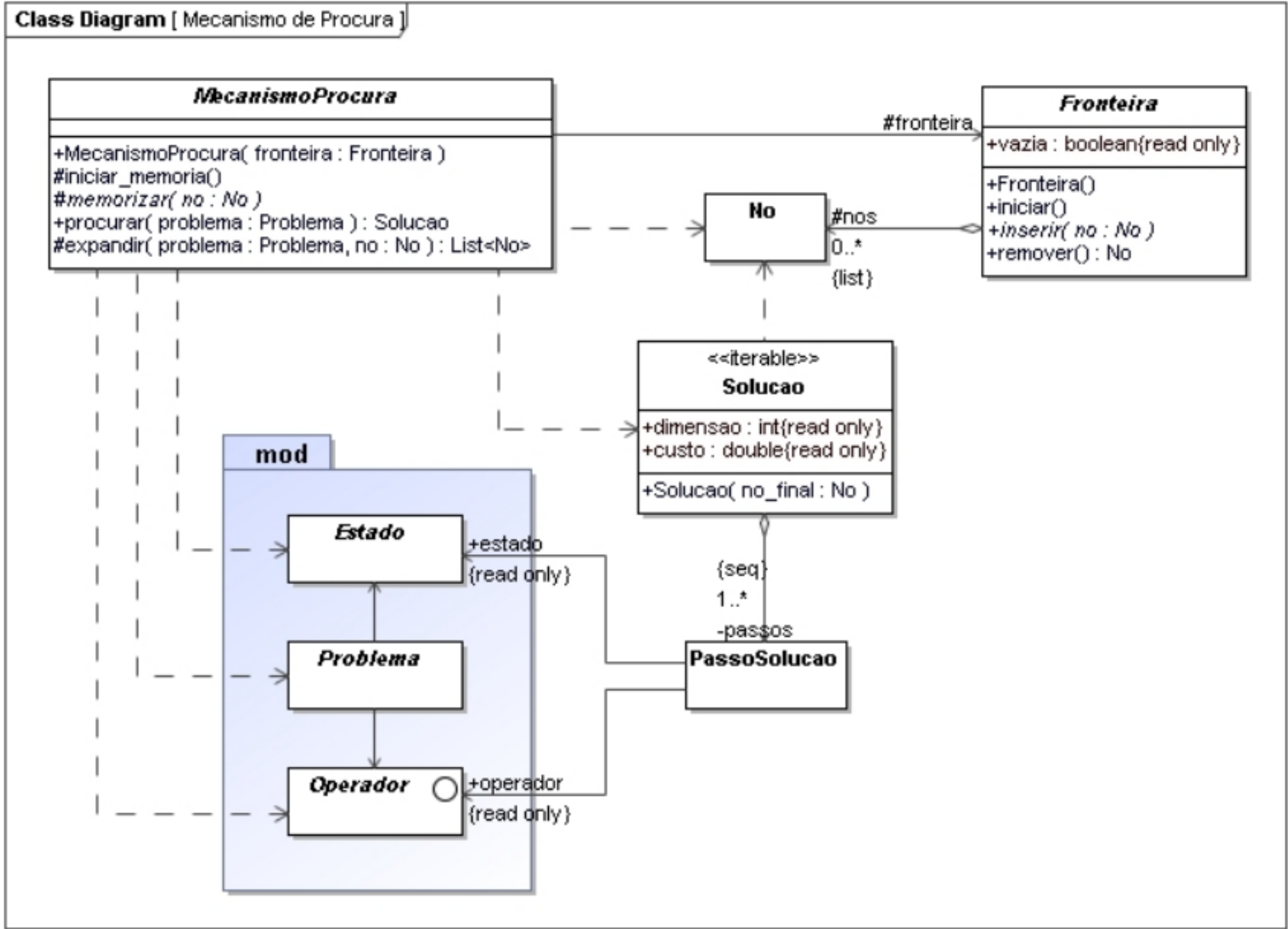


Figura 85 - Sistema mecanismo de procura

- Classe No (mecanismo de procura)

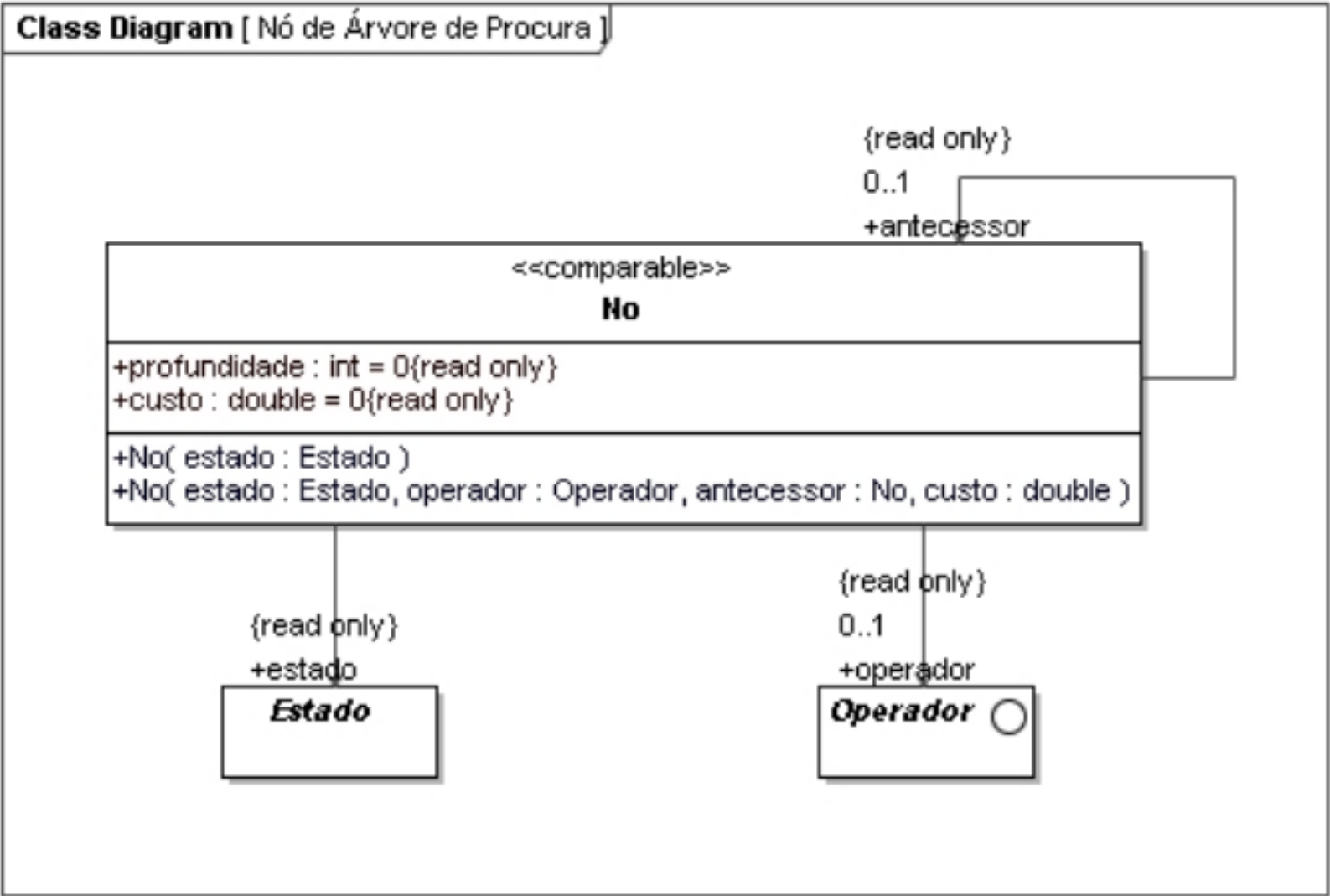


Figura 86 - Classe No do sistema mecanismo de procura

- Classe Fronteira (mecanismo de procura não informada)

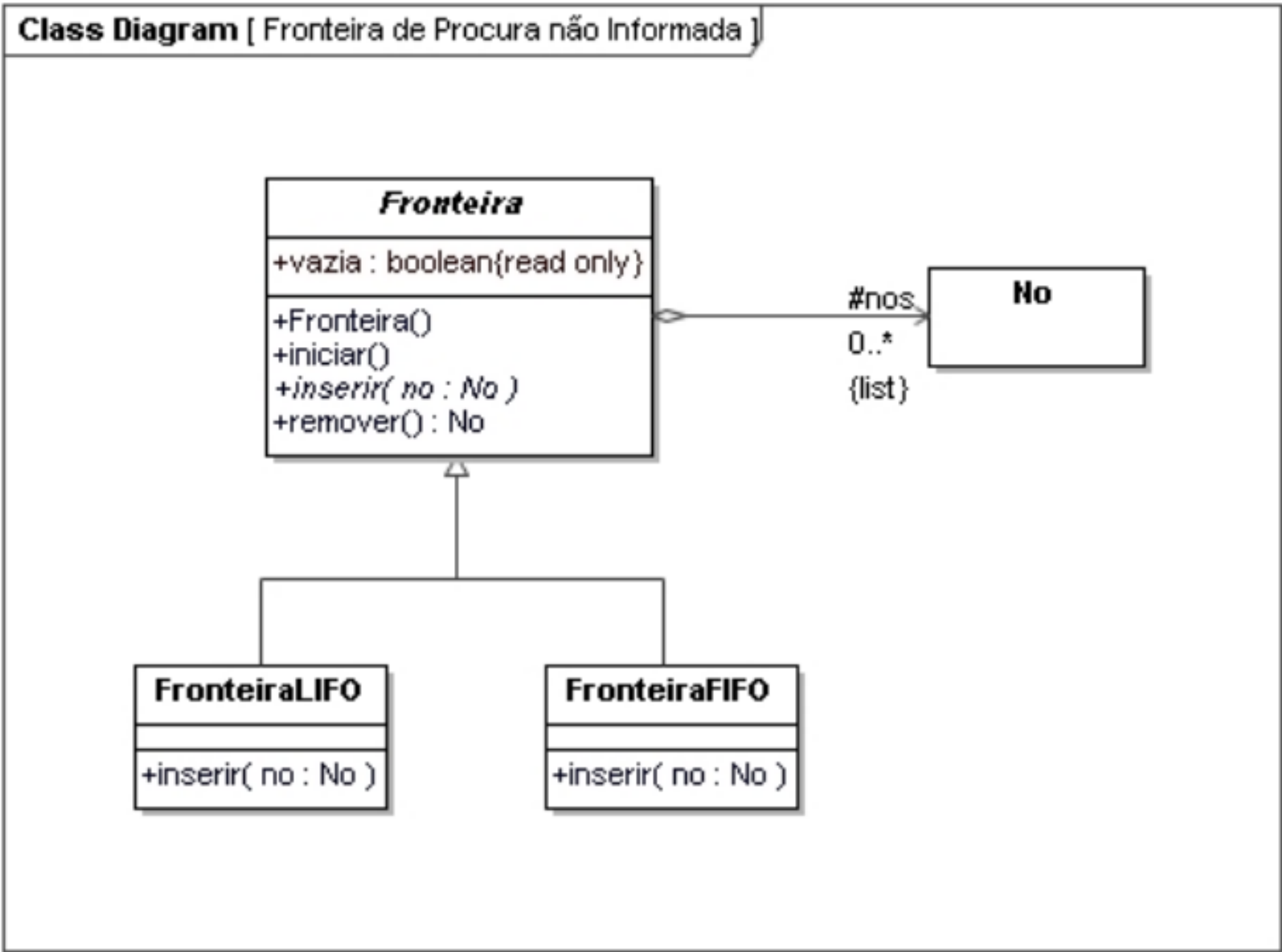


Figura 87 - Fronteira de procura não informada

• Mecanismos de procura não informada

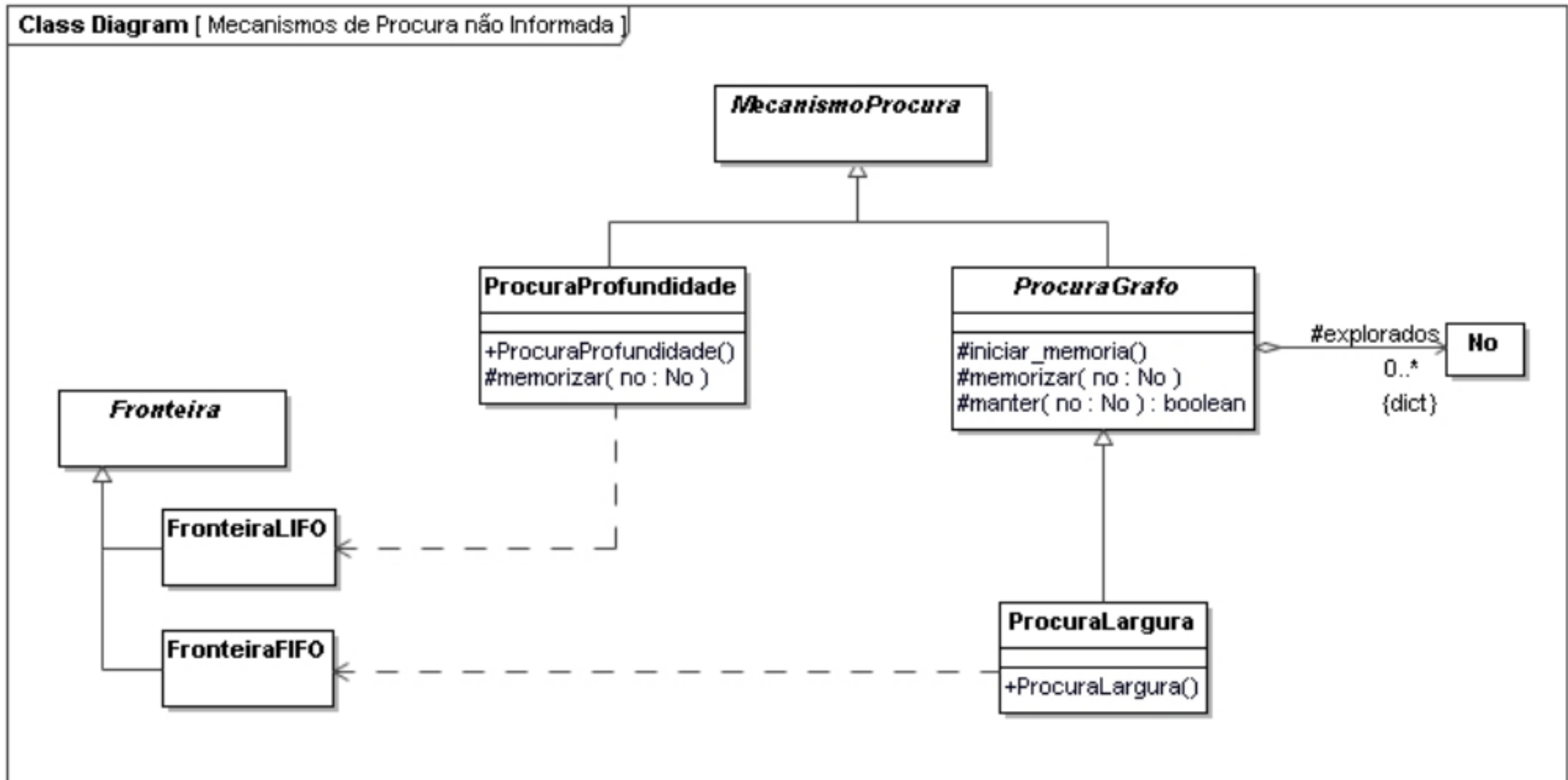


Figura 88 - Mecanismos de procura não informada

• Procuras em Profundidade

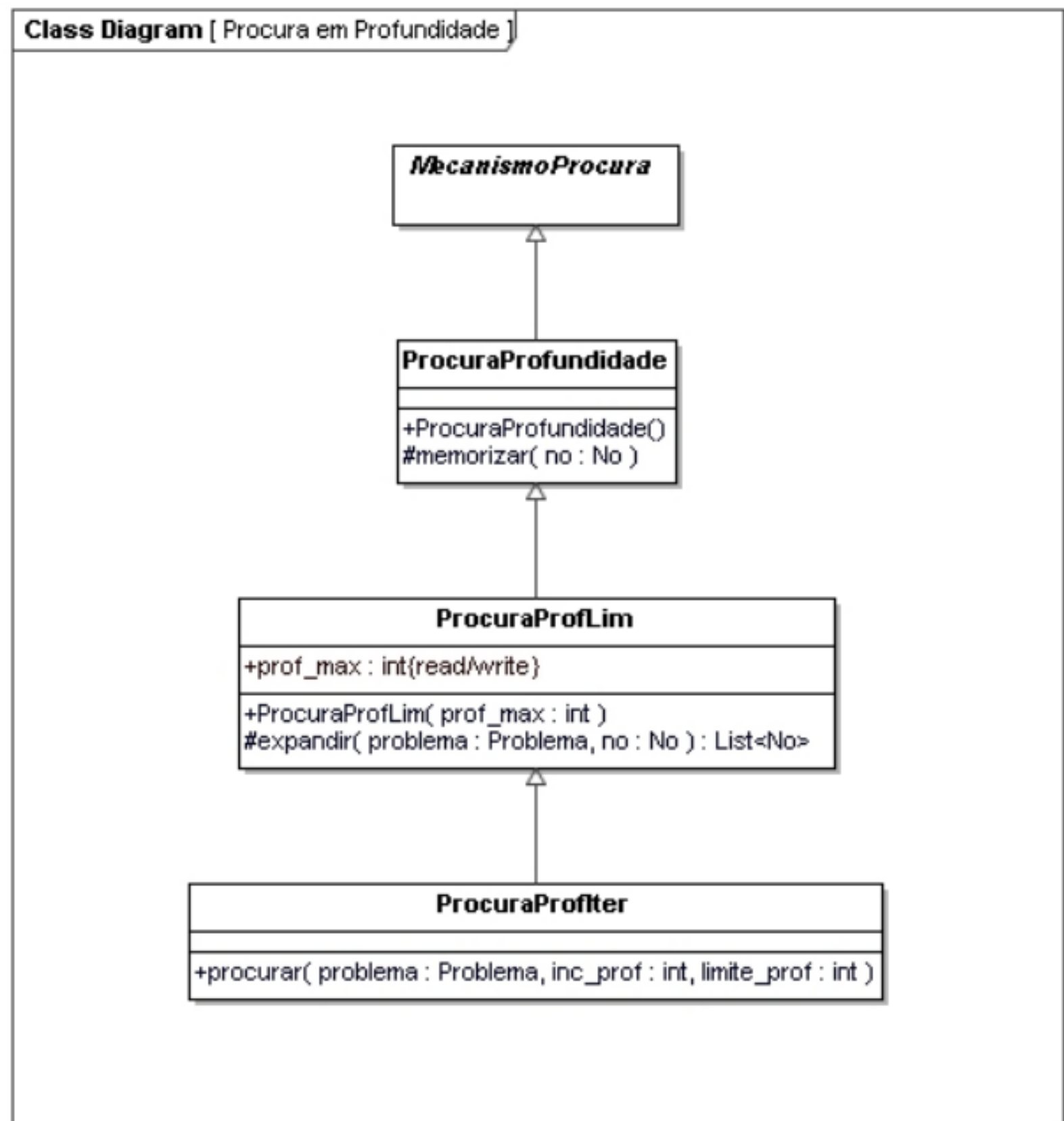


Figura 89 - Procuras em Profundidade

- Fronteira de procura com prioridade

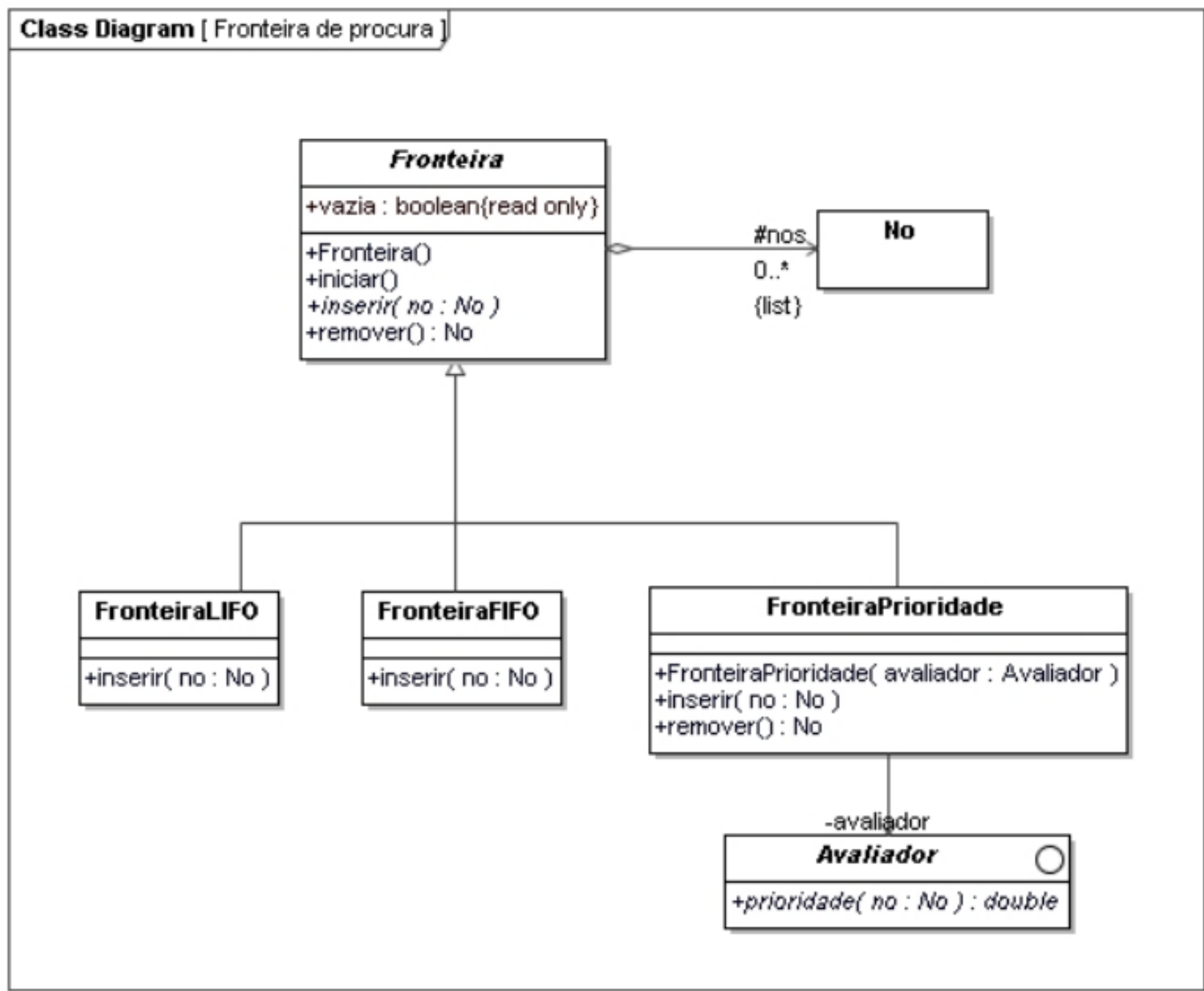


Figura 90 - Fronteira de procura com prioridade

- Procura Melhor-Primeiro

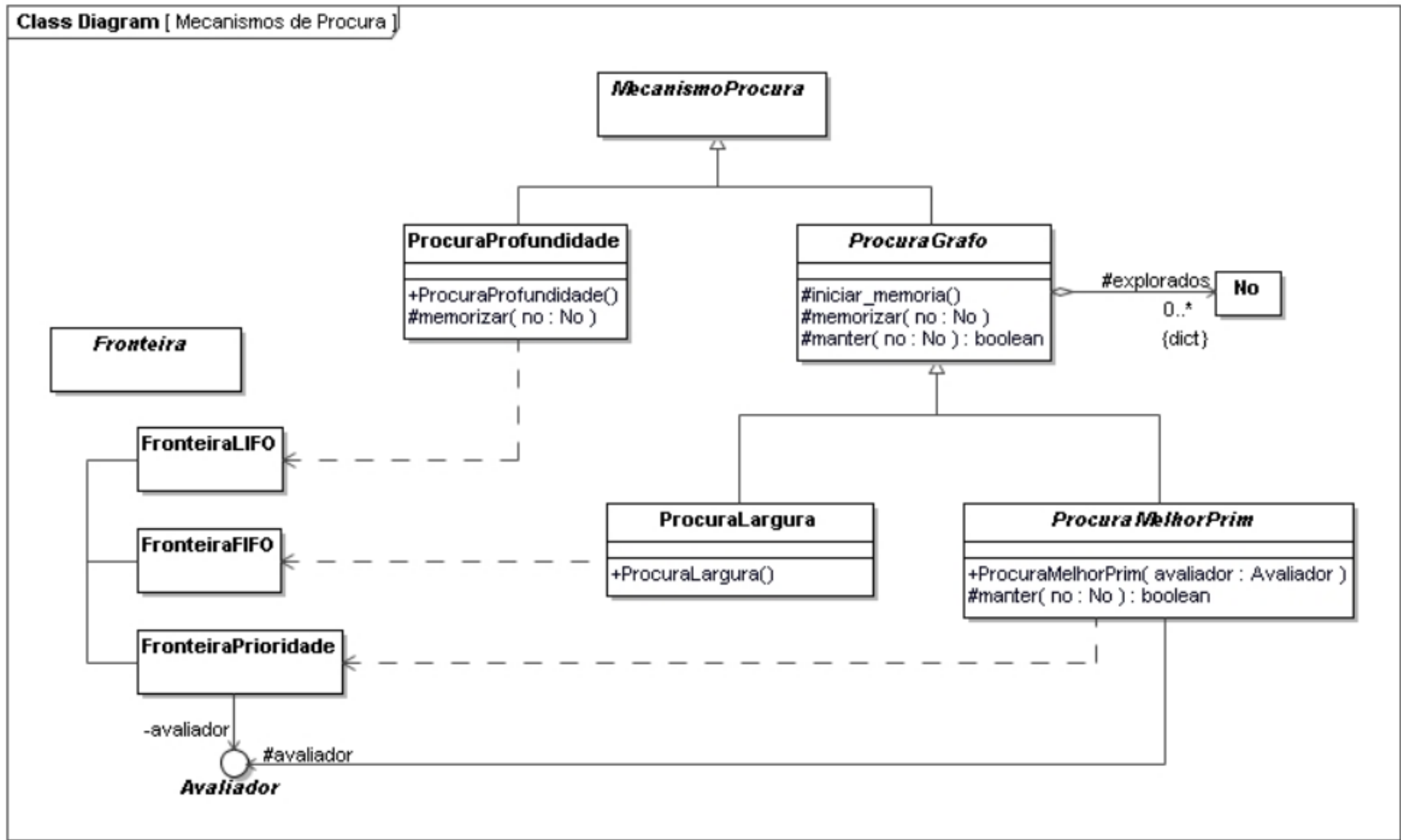


Figura 91 - Procura Melhor-Primeiro

• Procura de Custo Uniforme

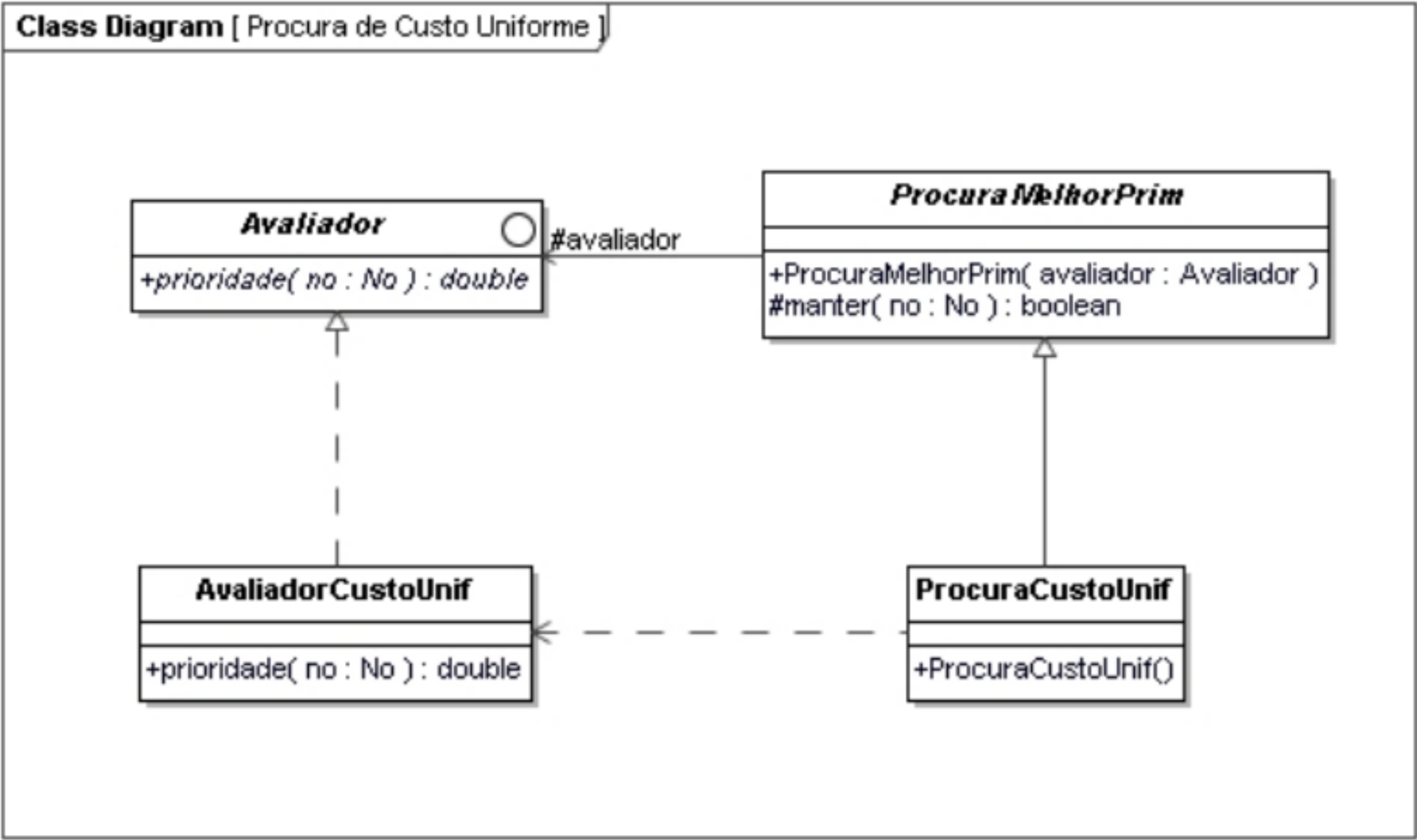


Figura 92 - Procura de Custo Uniforme

• Procuras Informadas

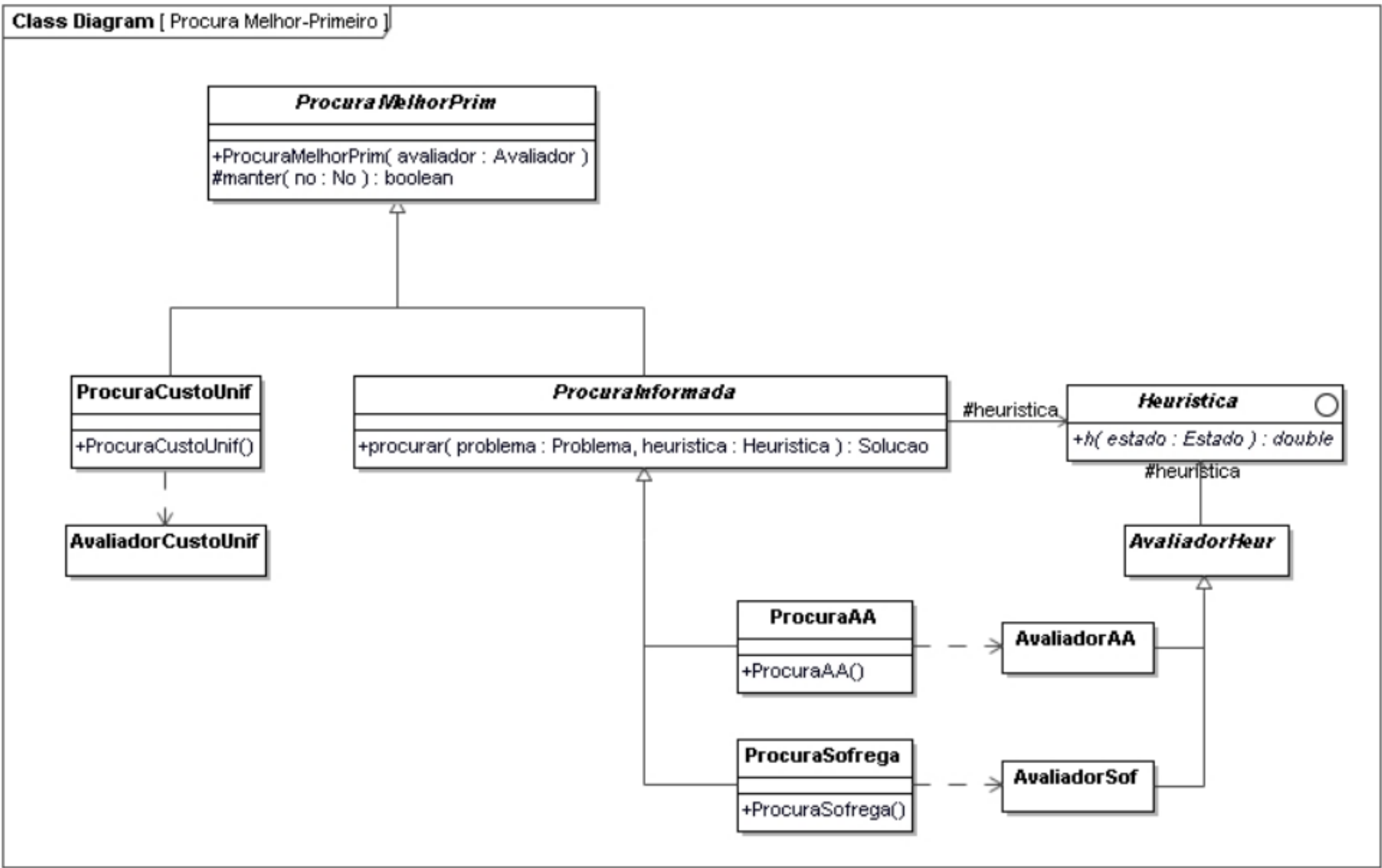


Figura 93 - Procuras Informadas

• Avaliadores de Nós

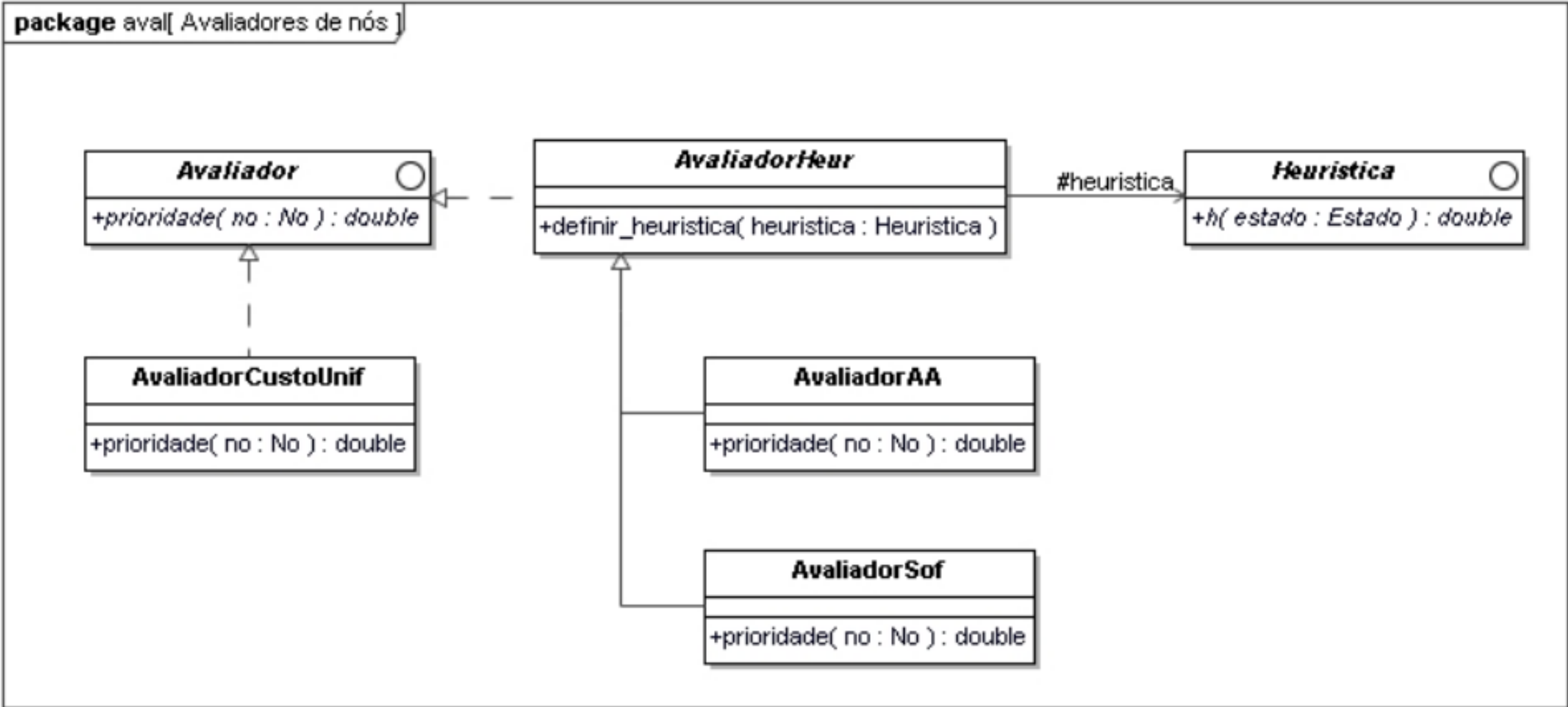


Figura 94 - Avaliadores de Nós

5.2. Implementação

- Classe Problema

```

1  from abc import ABC, abstractmethod
2
3  class Problema(ABC):
4
5      def __init__(self, estado_inicial, operadores):
6          self.__estado_inicial = estado_inicial
7          self.__operadores = operadores
8
9      @property
10     def estado_inicial(self):
11         return self.__estado_inicial
12
13     @property
14     def operadores(self):
15         return self.__operadores
16
17     @abstractmethod
18     def objetivo(self, estado):
19         .....

```

Figura 95 - Classe Problema

A classe abstrata **Problema** representa uma situação que se pretende resolver com recurso a **raciocínio automático**. Um problema é definido por: um **estado inicial**, a primeira configuração do problema; um conjunto de **operadores**, conjunto de ações que permitem fazer transformações de estado; **objetivo** ou função objetivo, configuração do problema que se quer atingir.

Contém as propriedades para leitura **estado_inicial** e **operadores** e o método abstrato **objetivo** que será usado para verificar se um estado recebido é um estado objetivo do problema.

- Interface Operador

```

1  from abc import ABC, abstractmethod
2
3  class Operador(ABC):
4
5      @abstractmethod
6      def aplicar(self, estado):
7          .....
8
9      @abstractmethod
10     def custo(self, estado, estado_suc):
11         .....

```

Figura 96 - Interface Operador

A interface **Operador** representa um operador no contexto de um problema com raciocínio automático. Um operador representa uma ação que produz

uma **mudança de estado**, isto é, operam sobre as representações internas de estado, produzindo transições de estado que correspondem à geração de novos estados.

Contém o método abstrato **aplicar** que será utilizado para aplicar um operador a um estado, para transformar este num novo estado, e o método abstrato **custo** que será responsável por obter o custo associado à transição de estado.

- Classe Estado

```
1  from abc import ABC, abstractmethod
2
3  class Estado(ABC):
4
5      @abstractmethod
6      def id_valor(self):
7          .....
8
9      def __hash__(self):
10         return self.id_valor()
11
12     def __eq__(self, other):
13         if isinstance(other, Estado):
14             return self.__hash__() == other.__hash__()
```

Figura 97 - Classe Estado

A classe abstrata **Estado** representa um estado no contexto de um problema com raciocínio automático. Um estado representa uma situação ou configuração de um determinado problema e este possui uma identificação única para ser possível a distinção entre diferentes estados.

Contém o método abstrato **id_valor** que irá permitir **definir a identidade** do estado baseado no seu conteúdo; o método **__hash__** que é responsável por **alterar a representação de uma instância** da classe Estado para a sua identidade utilizando o método **id_valor**, pois por omissão a representação de uma instância de uma classe é o seu endereço em memória; e o método **__eq__** que permite fazer a **comparação** entre dois estados, usando a identidade de cada um, fazendo uso do método **__hash__**.

• Classe MecanismoProcura

```

1  from abc import ABC, abstractmethod
2  from .no import No
3  from .solucao import Solucao
4
5  class MecanismoProcura(ABC):
6
7      def __init__(self, fronteira):
8          self._fronteira = fronteira
9
10     def _iniciar_memoria(self):
11         self._fronteira.iniciar()
12
13     @abstractmethod
14     def _memorizar(self, no):
15         """
16
17     def procurar(self, problema):
18         self._iniciar_memoria()
19         no = No(problema.estado_inicial)
20         self._memorizar(no)
21
22         while not self._fronteira.vazia:
23             no = self._fronteira.remove()
24
25             if problema.objectivo(no.estado):
26                 return Solucao(no)
27
28             for no_sucessor in self._expandir(problema, no):
29                 self._memorizar(no_sucessor)
30
31     def _expandir(self, problema, no):
32         sucessores = []
33         estado = no.estado
34
35         for operador in problema.operadores:
36             estado_suc = operador.aplicar(estado)
37
38             if estado_suc is not None:
39                 custo = no.custo + operador.custo(estado, estado_suc)
40                 no_sucessor = No(estado_suc, operador, no, custo)
41                 sucessores.append(no_sucessor)
42
43         return sucessores

```

Figura 98 - Classe MecanismoProcura

A classe **MecanismoProcura** representa uma implementação geral de um **mecanismo de procura num espaço de estados**. Este é um mecanismo tipicamente utilizado em **problemas de planeamento**, nos quais se pretende encontrar uma sequência de situações e de ações que levem de uma situação inicial a uma situação final. O processo de procura consiste em **explorar sucessivamente cada nó**, verificando se o estado atual corresponde ao **objetivo**, se não corresponder, esse estado é **expandido** gerando todos os **estados sucessores** por aplicação dos **operadores** disponíveis e para cada estado sucessor é repetido o processo. Caso um estado corresponda ao objetivo, o processo termina e é retornado o **percurso** do estado inicial ao objetivo. Se o não existirem sucessores, o processo termina **sem solução**.

Contém o método **__init__** que apenas instancia o tipo de fronteira do mecanismo de procura, o método protegido **iniciar_memoria** que inicia a fronteira de exploração do mecanismo, o método abstrato e protegido **memorizar** que é responsável por memorizar um nó na fronteira, é abstrato para que seja definido por cada mecanismo de procura, consoante a forma indicada de memorização de nós na sua fronteira, o método protegido **expandir** que é utilizado para expandir o nó atual, gerando estados sucessores

por aplicação dos operadores, e o método **procurar** que implementa o algoritmo base de procura em espaço de estados:

- É iniciada a fronteira e nela é memorizado o nó inicial;
- Enquanto a fronteira não estiver vazia, é removido o primeiro nó da fronteira para este ser expandido;
- Se o estado do nó for objetivo, é retornada a solução;
- Se não for, o nó é expandido e cada nó sucessor é memorizado na fronteira.

• Classe No

```
1 class No:
2
3     def __init__(self, estado, operador=None, antecessor=None, custo=0):
4         self.__estado = estado
5         self.__operador = operador
6         self.__antecessor = antecessor
7         self.__custo = custo
8
9         if self.__antecessor:
10             self.__profundidade = self.__antecessor.profundidade + 1
11         else:
12             self.__profundidade = 0
13
14     @property
15     def estado(self):
16         return self.__estado
17
18     @property
19     def operador(self):
20         return self.__operador
21
22     @property
23     def antecessor(self):
24         return self.__antecessor
25
26     @property
27     def profundidade(self):
28         return self.__profundidade
29
30     @property
31     def custo(self):
32         return self.__custo
33
34     def __lt__(self, outro_no):
35         return self.custo < outro_no.custo
```

Figura 99 - Classe No

A classe **No** representa um **nó de uma árvore de procura**. Um nó representa uma **etapa de procura** que mantém informação relativa a cada transição de estado explorada. A informação mantida é o estado do **nó atual**, o **nó antecessor**, o **operador** que foi aplicado ao nó antecessor para chegar ao estado do nó atual, o **custo** do nó atual, que representa o custo de todas as operações desde o nó inicial até ao atual e a **profundidade** do nó. Como um nó mantém toda esta informação, nomeadamente o seu antecessor e operador, é criada implicitamente a **árvore de procura**.

Contém o método **__init__** que instancia e guarda em atributos **todas as informações** referidas acima e que permite o **polimorfismo** para o caso do nó inicial, que não tem operador nem antecessor, e o seu custo é zero, então os valores são definidos por omissão. Contém **propriedades** que utilizam os atributos criados no construtor, referentes às informações já referidas, para que sejam apenas para leitura. Contém ainda o método **__lt__** (less than) que faz

override ao método do Python para tornar a classe **No** comparável, ou seja, instâncias de **No** podem ser comparadas com outras instâncias de **No**.

- Classe Solucao

```
1  from .passo_solucao import PassoSolucao
2
3  class Solucao:
4
5      def __init__(self, no_final):
6          self.__no_final = no_final
7          self.__passos = []
8          no = no_final
9          while no.anteecessor:
10             passo = PassoSolucao(no.anteecessor.estado, no.operador)
11             self.__passos.insert(0, passo)
12             no = no.anteecessor
13
14     @property
15     def dimensao(self):
16         return self.__no_final.profundidade
17
18     @property
19     def custo(self):
20         return self.__no_final.custo
21
22     def __iter__(self):
23         return iter(self.__passos)
24
25     def __getitem__(self, index):
26         return self.__passos[index]
```

Figura 100 - Classe Solucao

A classe **Solucao** representa a solução para o problema, ou seja, os passos necessários para chegar ao objetivo, partindo do nó inicial. Cada passo é guardado como uma instância de **PassoSolucao** que são colocados numa lista de passos.

Contém o método `__init__` que cria a lista de passos, com todos os **PassoSolucao** da solução. Contém a propriedade **dimensão**, que guarda a dimensão da solução, isto é, a profundidade do nó final e a propriedade **custo**, que guarda o custo total da solução, equivalente ao custo do nó final. Finalmente, contém os métodos `__iter__` e `__getitem__` que permitem que uma solução seja iterável e indexável, respetivamente, através da lista de passos.

- Classe PassoSolucao

```
1  from dataclasses import dataclass
2  from mod.estado import Estado
3  from mod.operador import Operador
4
5  @dataclass
6  class PassoSolucao:
7      estado: Estado
8      operador: Operador
```

Figura 101 - Classe PassoSolucao

Classe de dados cujo objetivo é guardar informação relativamente a qual operador aplicar num determinado estado para chegar à solução.

- Classe Fronteira

```
1  from abc import ABC, abstractmethod
2
3  class Fronteira(ABC):
4
5      def __init__(self):
6          self.iniciar()
7
8      @property
9      def vazia(self):
10         return len(self._nos) == 0
11
12     def iniciar(self):
13         self._nos = []
14
15     @abstractmethod
16     def inserir(no):
17         """
18
19     def remover(self):
20         return self._nos.pop(0)
```

Figura 102 - Classe Fronteira

A classe abstrata **Fronteira** representa uma fronteira de exploração (estrutura de dados) que contem nós que já foram gerados, mas que ainda não foram expandidos (explorados).

Contém o método **__init__** que é responsável por inicializar a fronteira através do método **iniciar**; o método **iniciar** que é responsável por criar um atributo que representa uma lista vazia de nós; o método abstrato **inserir**, responsável por inserir um nó na fronteira, dependendo do tipo de fronteira; o método **remover** que permite remover o primeiro elemento da lista de nós. Contém ainda a propriedade **vazia** que indica se a lista de nós está vazia ou não.

- Classe FronteiraLIFO

```
1  from ..mec_proc.frenteira import Fronteira
2
3  class FronteiraLIFO(Frenteira):
4
5      def inserir(self, no):
6          self._nos.insert(0, no)
```

Figura 103 - Classe FronteiraLIFO

A classe **FronteiraLIFO** (LIFO = Last In First Out) é uma **especialização** da classe **Fronteira**, responsável por definir uma fronteira do tipo LIFO, ou seja, em que os nós são inseridos no **início** da lista para que os primeiros nós a sair sejam os mais recentes. Este é o tipo de fronteira utilizada na **procura em profundidade** pois, nesta procura, os primeiros nós a ser expandidos são os mais recentes, os últimos a ser gerados, aumentando a profundidade do ramo corrente de procura.

- Classe FronteiraFIFO

```
1  from ..mec_proc.frenteira import Fronteira
2
3  class FronteiraFIFO(Frenteira):
4
5      def inserir(self, no):
6          self._nos.append(no)
```

Figura 104 - Classe FronteiraFIFO

A classe **FronteiraFIFO** (FIFO = First In First Out) é uma **especialização** da classe **Fronteira**, responsável por definir uma fronteira do tipo FIFO, ou seja, em que os nós são inseridos no **fim** da lista para que os primeiros nós a sair sejam os mais antigos. Esta fronteira é utilizada na **procura em largura** pois, nesta procura, os primeiros nós a ser expandidos são os mais antigos, os primeiros a ser gerados, levando à exploração exaustiva de cada nível de procura antes da exploração de nós a um nível de maior profundidade.

- Classe ProcuraProfundidade

```
1  from ..mec_proc.mecanismo_procura import MecanismoProcura
2  from ..frenteira_lifo import FronteiraLIFO
3
4  class ProcuraProfundidade(MecanismoProcura):
5
6      def __init__(self):
7          super().__init__(FronteiraLIFO())
8
9      def _memorizar(self, no):
10         self._frenteira.inserir(no)
```

Figura 105 - Classe ProcuraProfundidade

A classe **ProcuraProfundidade** representa o mecanismo de procura em profundidade. Esta classe é uma especialização da classe **MecanismoProcura**. A procura explora os nós mais recentes primeiro, aumentando a profundidade do ramo corrente de procura, logo, a fronteira utilizada é uma **FronteiraLIFO**.

- Classe ProcuraProfLim

```

1  from .procura_profundidade import ProcuraProfundidade
2
3  class ProcuraProfLim(ProcuraProfundidade):
4
5      def __init__(self, prof_max):
6          self.__prof_max = prof_max
7          super().__init__()
8
9      @property
10     def prof_max(self):
11         return self.__prof_max
12
13     @prof_max.setter
14     def prof_max(self, valor):
15         self.__prof_max = valor
16
17     def _expandir(self, problema, no):
18         return super()._expandir(problema, no) if no.profundidade < self.__prof_max else []

```

Figura 106 - Classe ProcuraProfLim

A classe **ProcuraProfLim** representa o mecanismo de procura em profundidade limitada. Esta classe é uma **especialização** da classe **ProcuraProfundidade**. Este mecanismo é idêntico ao mecanismo de procura em profundidade, com a particularidade que este tem um **limite de profundidade**, ou seja, só expande nós enquanto esse limite não for atingido. Este mecanismo não é ótimo nem completo, ou seja, não garante uma solução ótima nem garante que uma solução seja encontrada.

Contém o método **__init__** que inicializa a profundidade máxima para o mecanismo e invoca o construtor da superclasse e o método protegido **expandir** responsável pela expansão dos nós. Neste caso, é feito override ao método da superclasse pois, neste mecanismo, um nó só é expandido caso a sua profundidade seja menor que a profundidade máxima definida no construtor. Contém ainda o getter e o setter da propriedade **prof_max**, que mantém a profundidade máxima.

- Classe ProcuraProflter

```

1  from .procura_prof_lim import ProcuraProfLim
2
3  class ProcuraProfIter(ProcuraProfLim):
4
5      def procurar(self, problema, inc_prof=1, limite_prof=100):
6          for profundidade in range(0, limite_prof + 1, inc_prof):
7              self.prof_max = profundidade
8              solucao = super().procurar(problema)
9              if solucao:
10                 return solucao

```

Figura 107 - Classe ProcuraProflter

A classe **ProcuraProflter** representa o mecanismo de procura em profundidade iterativa. Esta classe é uma **especialização** da classe **ProcuraProfLim**. Este mecanismo consiste em fazer várias procuras em profundidade limitada, iterativamente, ou seja, com o limite de profundidade a aumentar consoante um certo incremento, através do método **procurar**. Este mecanismo é ótimo e completo.

- Classe ProcuraGrafo

```
1  from .mecanismo_procura import MecanismoProcura
2
3  class ProcuraGrafo(MecanismoProcura):
4
5      def _iniciar_memoria(self):
6          super()._iniciar_memoria()
7          self._explorados = {}
8
9      def _memorizar(self, no):
10         estado = no.estado
11         if self._manter(no):
12             self._fronteira.inserir(no)
13             self._explorados[estado] = no
14
15     def _manter(self, no):
16         return no.estado not in self._explorados
```

Figura 108 - Classe ProcuraGrafo

A classe **ProcuraGrafo** representa uma **procura em grafo**. Esta classe é uma especialização da classe **MecanismoProcura**. Este mecanismo tem a particularidade de **manter em memória os nós que já foram explorados** de modo a evitar que estes sejam explorados novamente, reduzindo o desperdício de recursos como o tempo e a memória.

Contém o método protegido **iniciar_memoria** que invoca o mesmo método, mas da superclasse, e inicia o dicionário de nós explorados. Contém também o método protegido **memorizar** responsável por memorizar um nó na fronteira de exploração e no dicionário de explorados. Um nó só é memorizado caso seja para manter, ou seja, ainda não tenha sido explorado. Esta verificação é feita no método **manter**.

- Classe ProcuraLargura

```
1  from .fronteira_fifo import FronteiraFIFO
2  from ..mec_proc.procura_grafo import ProcuraGrafo
3
4  class ProcuraLargura(ProcuraGrafo):
5
6      def __init__(self):
7          super().__init__(FronteiraFIFO())
```

Figura 109 - Classe ProcuraLargura

A classe **ProcuraLargura** representa o mecanismo de procura em largura. Esta classe é uma **especialização** da classe **ProcuraGrafo**. Neste mecanismo, a procura decorre explorando os nós mais antigos primeiro (primeiros a ser gerados), levando à exploração exaustiva de cada nível de procura antes da exploração de nós a um nível de maior profundidade e, por isso, a fronteira utilizada é uma **FronteiraFIFO**.

• Classe FronteiraPrioridade

```

1  from ..mec_proc.frenteira import Fronteira
2  from heapq import heappush, heappop
3
4  class FronteiraPrioridade(Frenteira):
5
6      def __init__(self, avaliador):
7          super().__init__()
8          self.__avaliador = avaliador
9
10
11     def inserir(self, no):
12         prioridade = self.__avaliador.prioridade(no)
13         heappush(self._nos, (prioridade, no))
14
15     def remover(self):
16         _, no = heappop(self._nos)
17         return no

```

Figura 110 - Classe FronteiraPrioridade

A classe **FronteiraPrioridade** que representa a fronteira que é utilizada nas procuras melhor-primeiro. Essas procuras usam uma função $f(n)$ para a avaliação de cada nó gerado. A fronteira é, portanto, ordenada com base nessa função cujo valor é calculado no método **prioridade** dos avaliadores.

• Classe ProcuraMelhorPrim

```

1  from ..mec_proc.procura_grafo import ProcuraGrafo
2  from .frenteira_prioridade import FronteiraPrioridade
3
4  class ProcuraMelhorPrim(ProcuraGrafo):
5
6      def __init__(self, avaliador):
7          super().__init__(FronteiraPrioridade(avaliador))
8          self._avaliador = avaliador
9
10     def _manter(self, no):
11         return super()._manter(no) or no < self._explorados[no.estado]

```

Figura 111 - Classe ProcuraMelhorPrim

A classe **ProcuraMelhorPrim** representa o mecanismo de procura melhor-primeiro. Esta classe é uma **especialização** da classe **ProcuraGrafo**. Este mecanismo de procura utiliza uma função de avaliação que avalia cada nó gerado, tipicamente com base no custo da solução através desse nó. Quanto menor o valor dessa função, mais promissor para expansão é esse nó. A fronteira de exploração, neste mecanismo, ordena os nós por ordem crescente do valor obtido pela função de avaliação.

Contém o método **__init__**, que invoca o construtor da superclasse e instancia o avaliador deste mecanismo, e o método **manter** que faz override do método da superclasse pois, neste caso, um nó pode ser mantido caso ainda não esteja no dicionário de explorados, mas também se o seu custo for inferior ao custo de um nó com o mesmo estado.

- Classe ProcuraCustoUnif

```
1 from .procura_melhor_prim import ProcuraMelhorPrim
2 from .aval.avaliador_custo_unif import AvaliadorCustoUnif
3 class ProcuraCustoUnif(ProcuraMelhorPrim):
4
5     def __init__(self):
6         super().__init__(AvaliadorCustoUnif())
```

Figura 112 - Classe ProcuraCustoUnif

A classe **ProcuraCustoUnif** representa o mecanismo de procura de custo uniforme. Esta classe é uma **especialização** da classe **ProcuraMelhorPrim** e um caso particular desse mecanismo de procura. Neste caso, a função de avaliação corresponde diretamente ao custo de cada nó, como está definido no **AvaliadorCustoUnif**.

- Interface Avaliador

```
1 from abc import ABC, abstractmethod
2
3 class Avaliador(ABC):
4
5     @abstractmethod
6     def prioridade(self, no):
7         .....
```

Figura 113 - Interface Avaliador

A interface **Avaliador** representa o contrato funcional para definir um avaliador. Um avaliador representa a função de avaliação utilizada pelas procuras melhor-primeiro, definida no método **prioridade**, e este será definido pelas classes que implementarem esta interface, pois esta função pode ter em conta o custo do nó, uma heurística ou ambos, dependendo do método de procura.

- Classe AvaliadorCustoUnif

```
1 from .avaliador import Avaliador
2
3 class AvaliadorCustoUnif(Avaliador):
4
5     def prioridade(self, no):
6         return no.custo
```

Figura 114 - Classe AvaliadorCustoUnif

A classe **AvaliadorCustoUnif** é uma **realização** da interface **Avaliador**. Este avaliador representa a função de avaliação que será utilizada na procura de custo uniforme, que depende apenas do custo nó.

- Classe ProcuraInformada

```
1 from .procura_melhor_prim import ProcuraMelhorPrim
2
3 class ProcuraInformada(ProcuraMelhorPrim):
4
5     def procurar(self, problema, heuristica):
6         self._avaliador.definir_heuristica(heuristica)
7         return super().procurar(problema)
```

Figura 115 - Classe ProcuraInformada

A classe **ProcuraInformada** é uma **especialização** da classe **ProcuraMelhorPrim**. Esta procura é um tipo de procura melhor primeiro que utiliza **heurísticas**, ou seja, que tira partido do conhecimento sobre o domínio do problema para ordenar a fronteira de exploração. Foram estudados dois métodos de procura informada: a **procura sôfrega** e a **procura A***.

- Classe ProcuraAA

```
1 from .procura_informada import ProcuraInformada
2 from .aval.avaliador_aa import AvaliadorAA
3
4 class ProcuraAA(ProcuraInformada):
5
6     def __init__(self):
7         super().__init__(AvaliadorAA())
```

Figura 116 - Classe ProcuraAA

A classe **ProcuraAA** é uma **especialização** da classe **ProcuraInformada**. Esta procura é uma procura informada que utiliza uma função de avaliação que depende da **soma entre o custo do nó atual e a heurística**. A heurística utilizada nesta procura tem de ser necessariamente **admissível**. Dos estudados, este é o método com eficiência ótima, na medida em que nenhum outro expandirá menos nós, mantendo as características de ser completo e ótimo, ou seja, encontra uma solução ótima no menor número de nós expandidos possíveis.

- Classe AvaliadorAA

```
1 from .avaliador_heur import AvaliadorHeur
2
3 class AvaliadorAA(AvaliadorHeur):
4
5     def prioridade(self, no):
6         return no.custo + self._heuristica.h(no.estado)
```

Figura 117 - Classe AvaliadorAA

A classe **AvaliadorAA** é uma **especialização** da classe **AvaliadorHeur**. Este avaliador representa a função de avaliação usada na procura A*. Esta depende, neste caso, da soma entre o custo do nó e uma heurística.

- Classe ProcuraSofrega

```

1  from .procura_informada import ProcuraInformada
2  from .aval.avaliador_sof import AvaliadorSof
3
4  class ProcuraSofrega(ProcuraInformada):
5
6      def __init__(self):
7          super().__init__(AvaliadorSof())

```

Figura 118 - Classe ProcuraSofrega

A classe **ProcuraSofrega** é uma **especialização** da classe **ProcuraInformada**. Esta procura é uma procura informada que utiliza uma função de avaliação **apenas baseada numa heurística**. Tem como objetivo a minimização da estimativa de custo para atingir o objetivo não tendo em conta o custo do percurso já explorado. Esta procura produz soluções **sub-ótimas**, isto é, não existe garantia que a solução encontrada seja a melhor, no entanto, revela menor complexidade computacional.

- Classe AvaliadorSof

```

1  from .avaliador_heur import AvaliadorHeur
2
3  class AvaliadorSof(AvaliadorHeur):
4
5      def prioridade(self, no):
6          return self._heuristica.h(no.estado)

```

Figura 119 - Classe AvaliadorSof

A classe **AvaliadorSof** é uma **especialização** da classe **AvaliadorHeur**. Este avaliador representa a função de avaliação que será utilizada na procura sôfrega, que depende apenas de uma heurística.

- Interface Heuristica

```

1  from abc import ABC, abstractmethod
2
3  class Heuristica(ABC):
4
5      @abstractmethod
6      def h(self, estado):
7          .....

```

Figura 120 - Interface Heuristica

A interface **Heuristica** representa o contrato funcional para definir uma heurística. Uma heurística é uma função que classifica alternativas em algoritmos de pesquisa em cada etapa de ramificação com base nas informações disponíveis para decidir qual ramificação seguir. Ou seja, reflete conhecimento do domínio do problema para guiar a procura. Pode ser uma estimativa do custo do percurso do nó atual até ao objetivo. De certa forma, troca a exatidão ou precisão pela otimização do tempo de execução do algoritmo. A função heurística é definida no método **h**. Uma heurística pode ser admissível, isto significa que a estimativa de custo é sempre igual ou inferior ao custo efetivo mínimo. Um exemplo de uma heurística admissível é a

distância euclidiana. Esta é admissível porque a estimativa de custo representa a distância em linha reta até ao objetivo, portanto, o custo efetivo nunca será menor que a estimativa.

- Classe AvaliadorHeur

```
1  from .avaliador import Avaliador
2
3  class AvaliadorHeur(Avaliador):
4
5      def definir_heuristica(self, heuristica):
6          self._heuristica = heuristica
```

Figura 121 - Classe AvaliadorHeur

A classe **AvaliadorHeur** é uma **realização** da interface **Avaliador**. Este avaliador representa um avaliador, ou função de avaliação, que é baseada numa heurística. Esta heurística é definida pelo método **definir_heuristica**, que recebe uma heurística como parâmetro.

6. PROJETO – PARTE 4

Na quarta parte do projeto, o objetivo foi implementar um agente autónomo inteligente, capaz de deliberar e tomar decisões com base no ambiente que o rodeia. Como já foi estudado, para além da fase de deliberação, existe também a fase de planeamento automático, crucial para a geração de sequências de ações que levem o agente ao seu objetivo. Foram desenvolvidas bibliotecas de planeamento baseadas em procura em espaço de estados e processos de decisão de Markov.

6.1. Organização e Arquitetura do Projeto

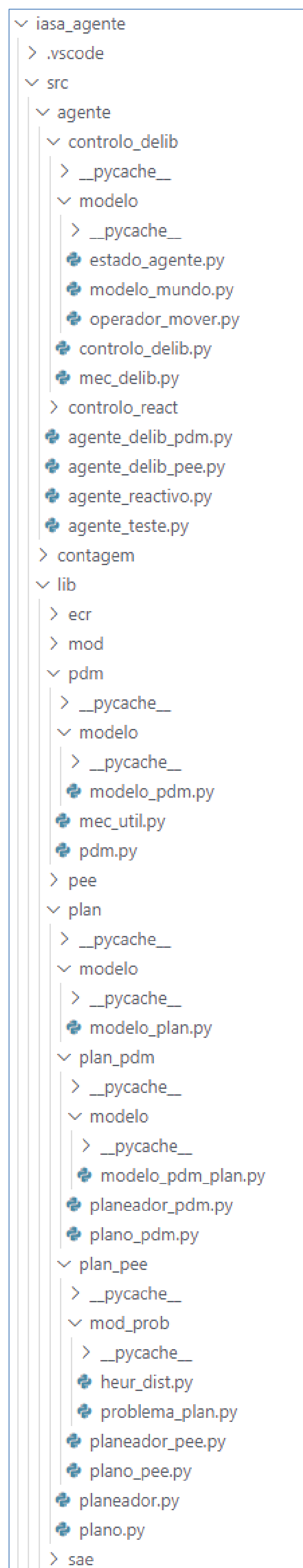


Figura 122 - Organização das pastas e módulos do projeto

• Sistema Controlo Deliberativo

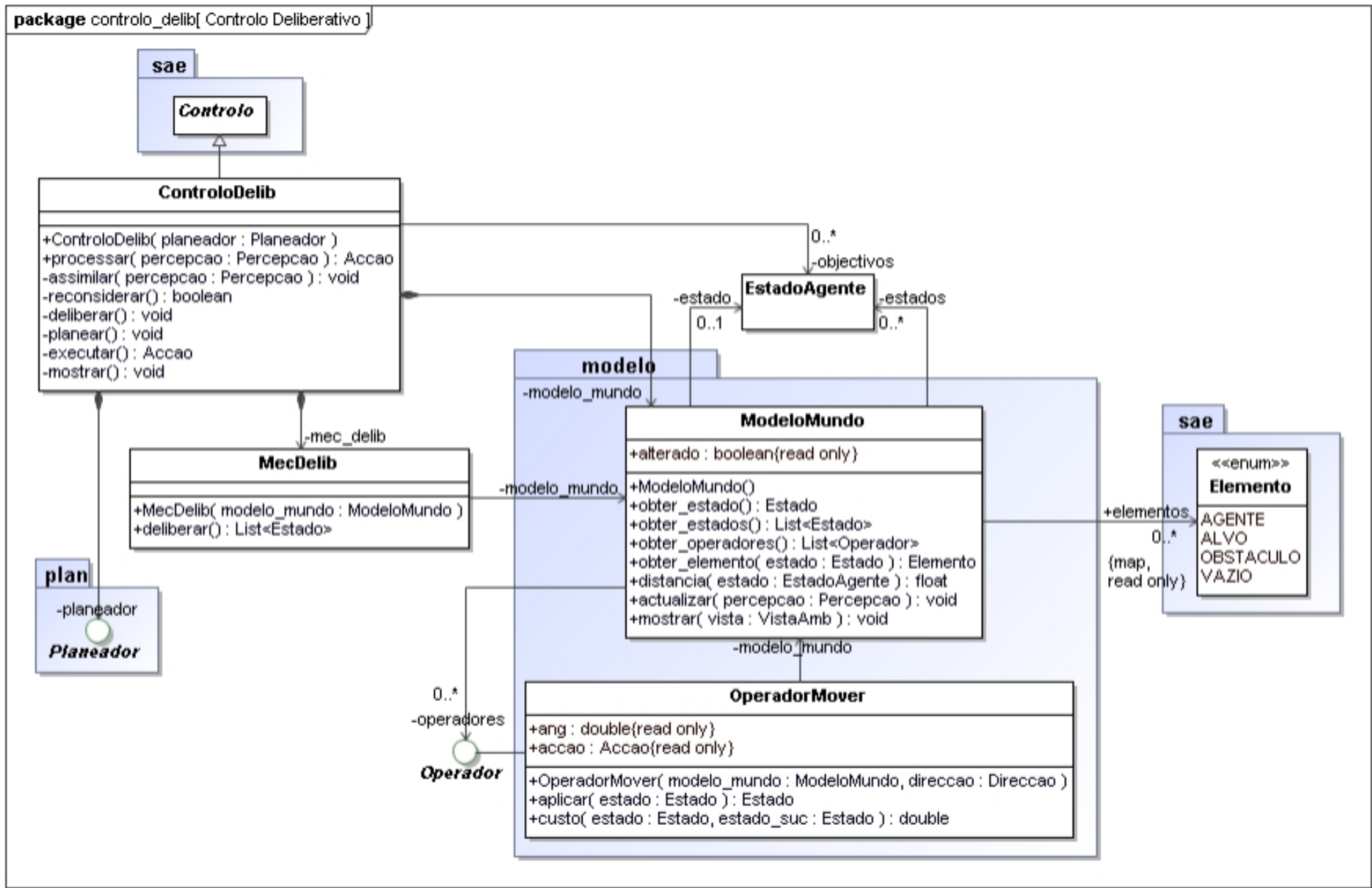


Figura 123 - Sistema Controlo Deliberativo

• Estado do Agente

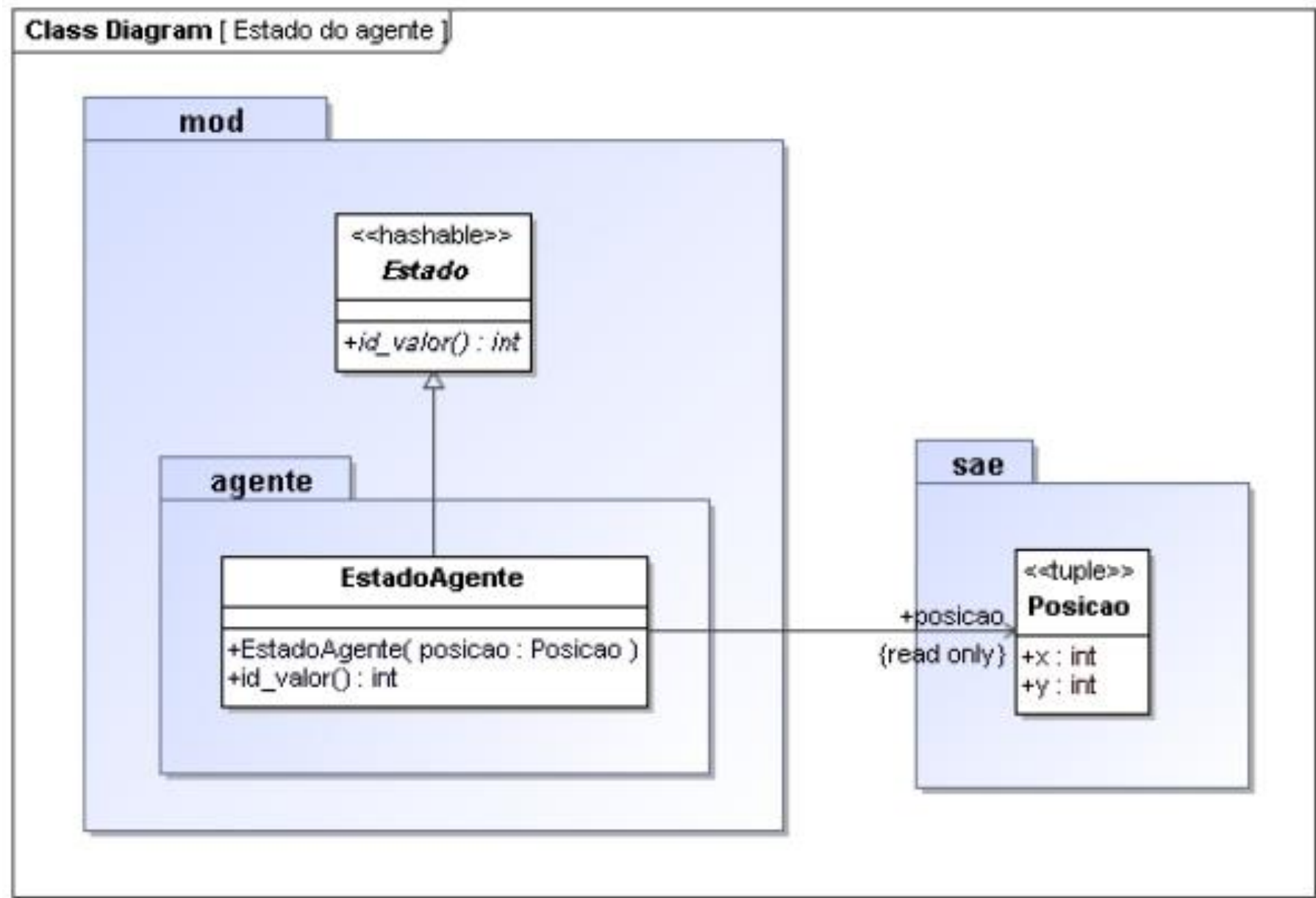


Figura 124 - Estado do Agente

• Sistema de Planeamento Automático

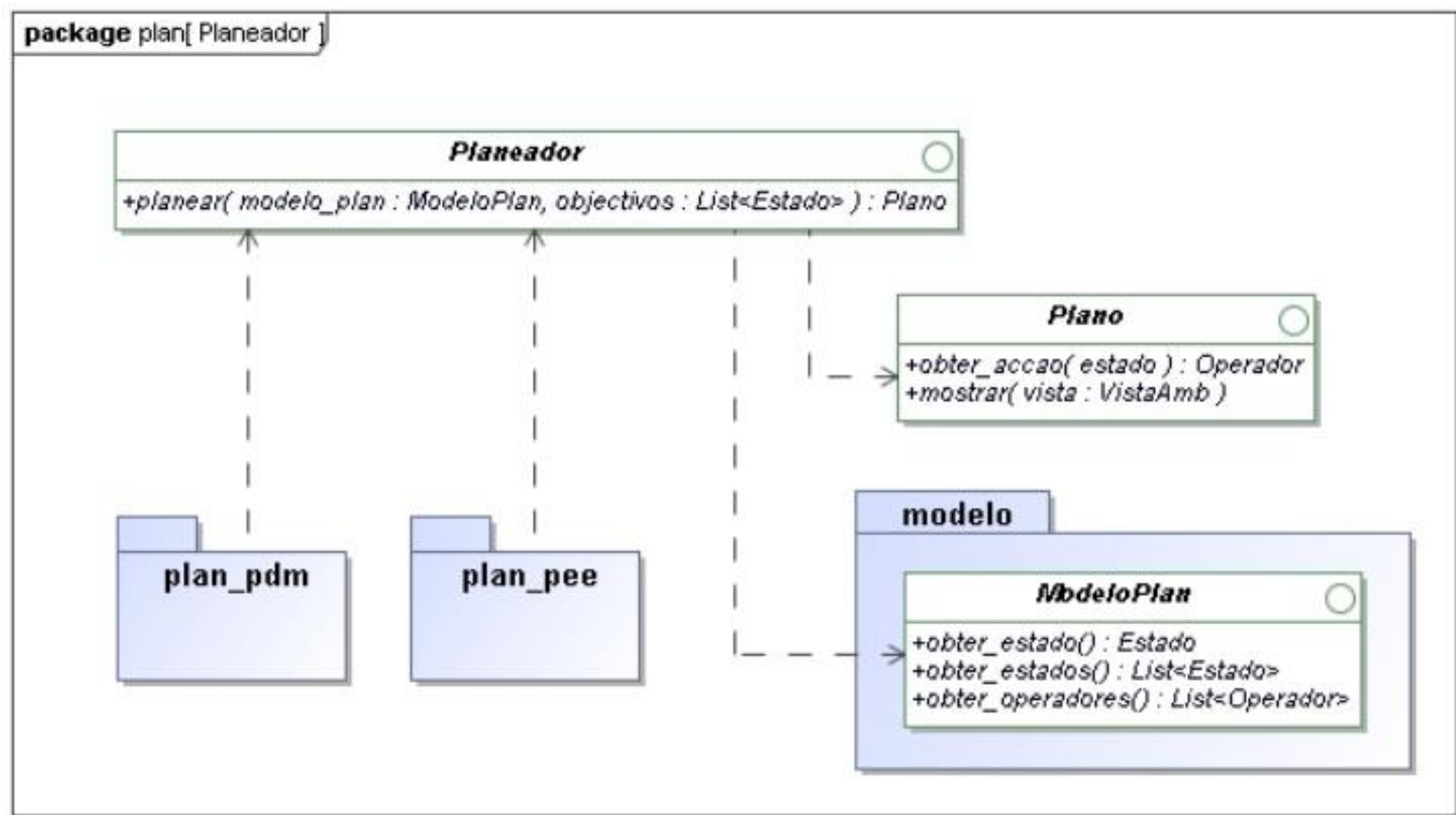


Figura 125 - Sistema do Planeamento Automático

• Relação ModeloMundo - ModeloPlan

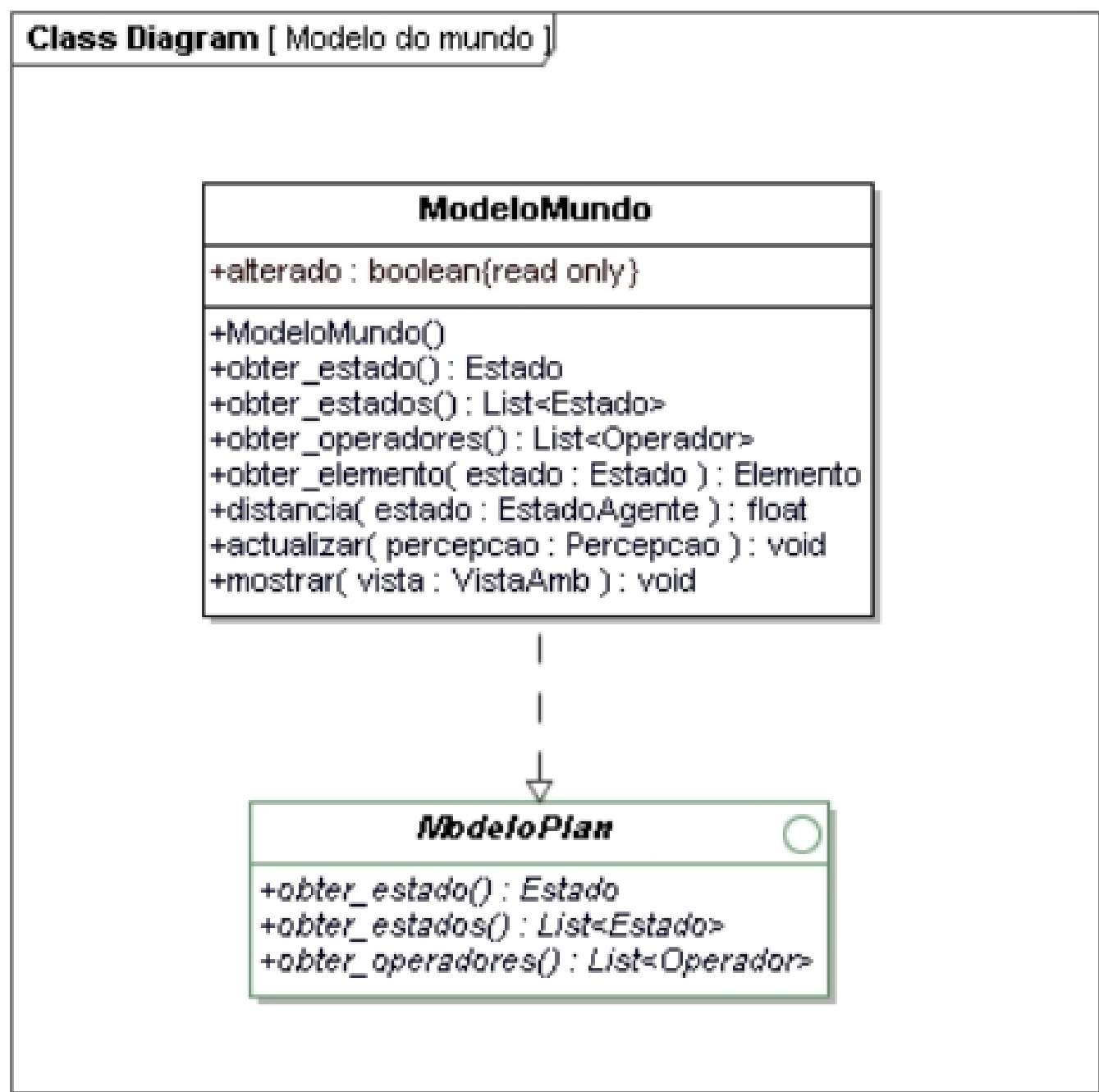


Figura 126 - Relação ModeloMundo - ModeloPlan

- Planeador com base em PEE

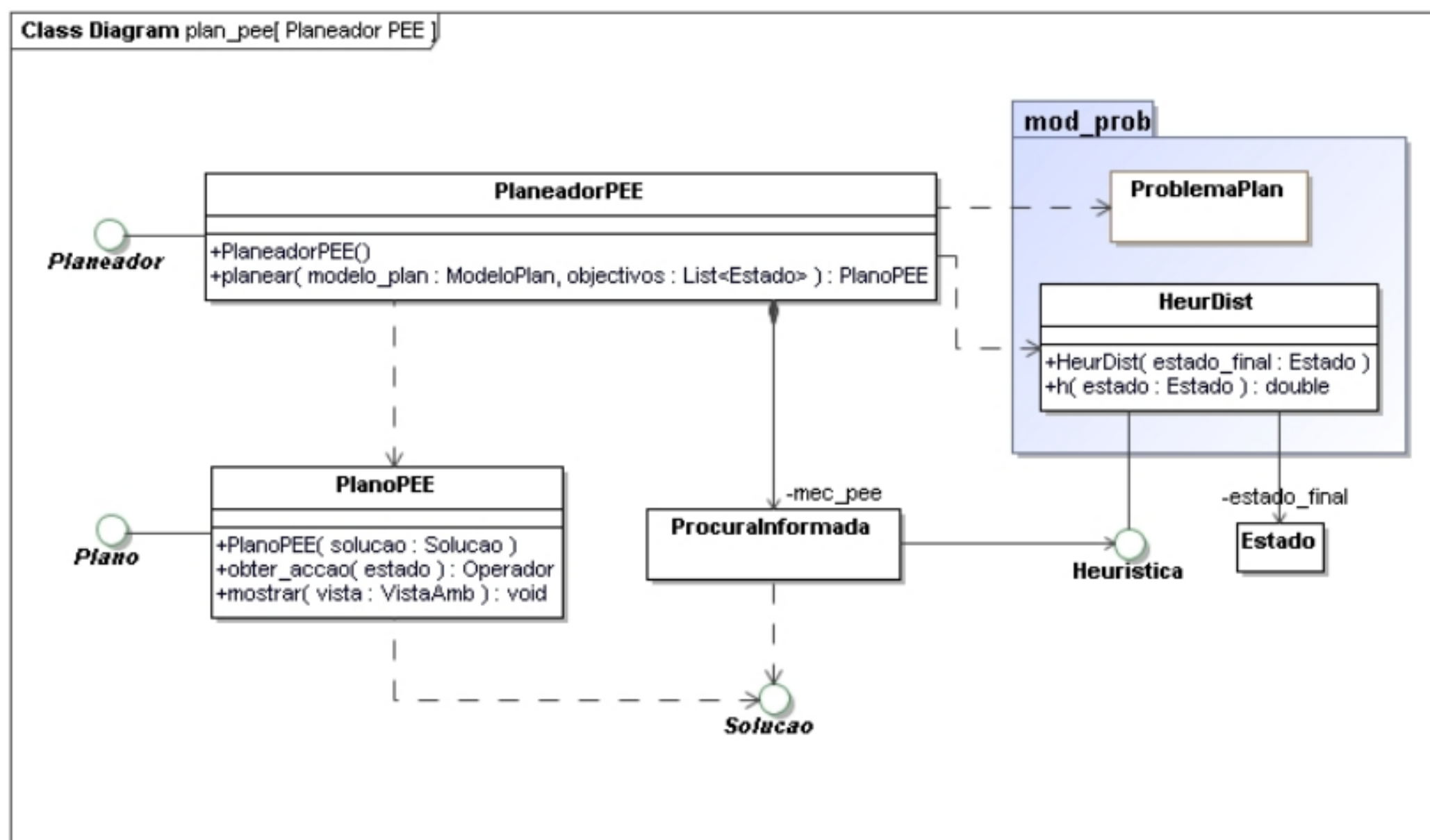


Figura 127 - Planeador com base em PEE

- Problema de Planeamento

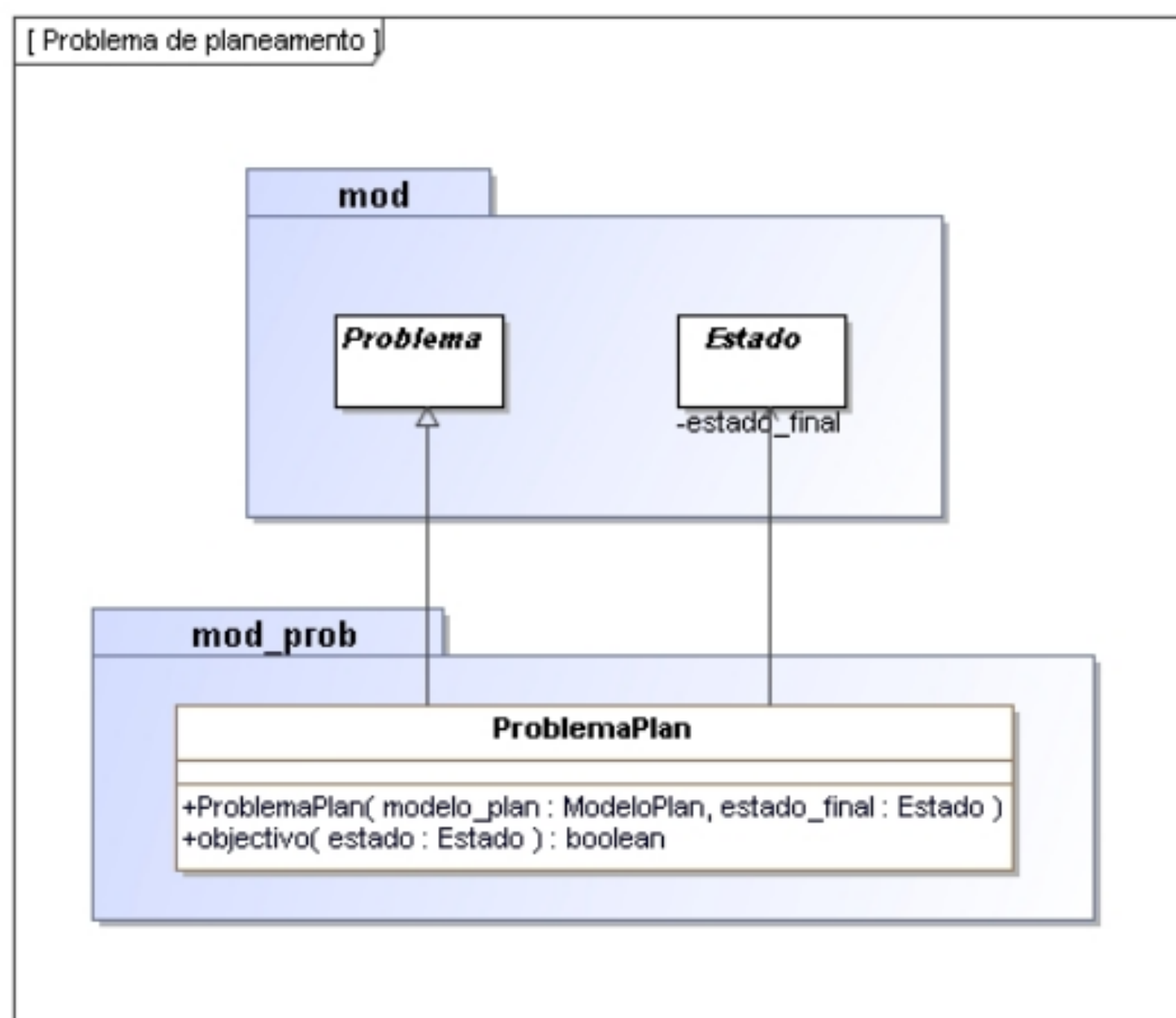


Figura 128 - Problema de Planeamento

• Mecanismo PDM

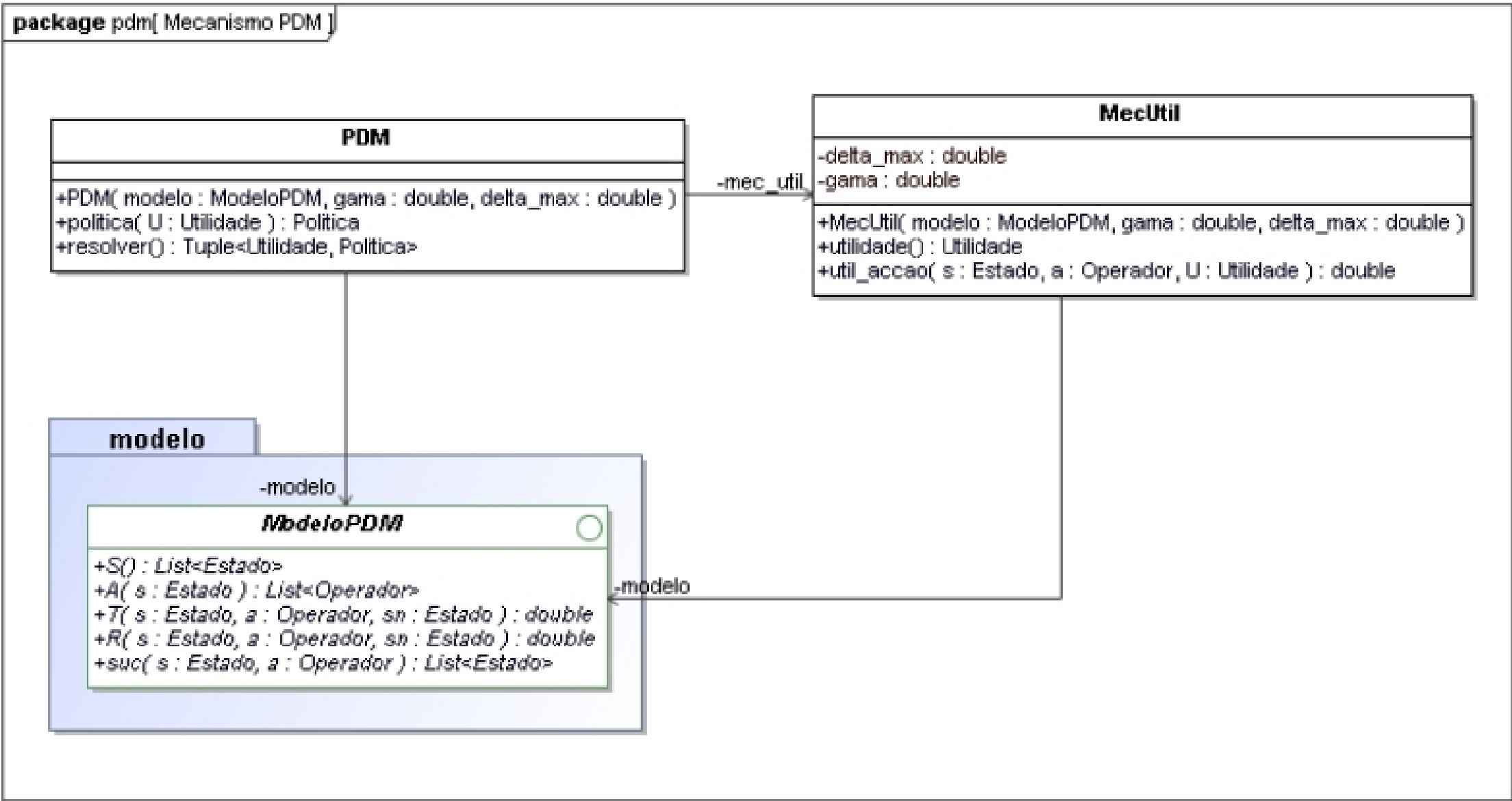


Figura 129 - Mecanismo PDM

• Planeador com base em PDM

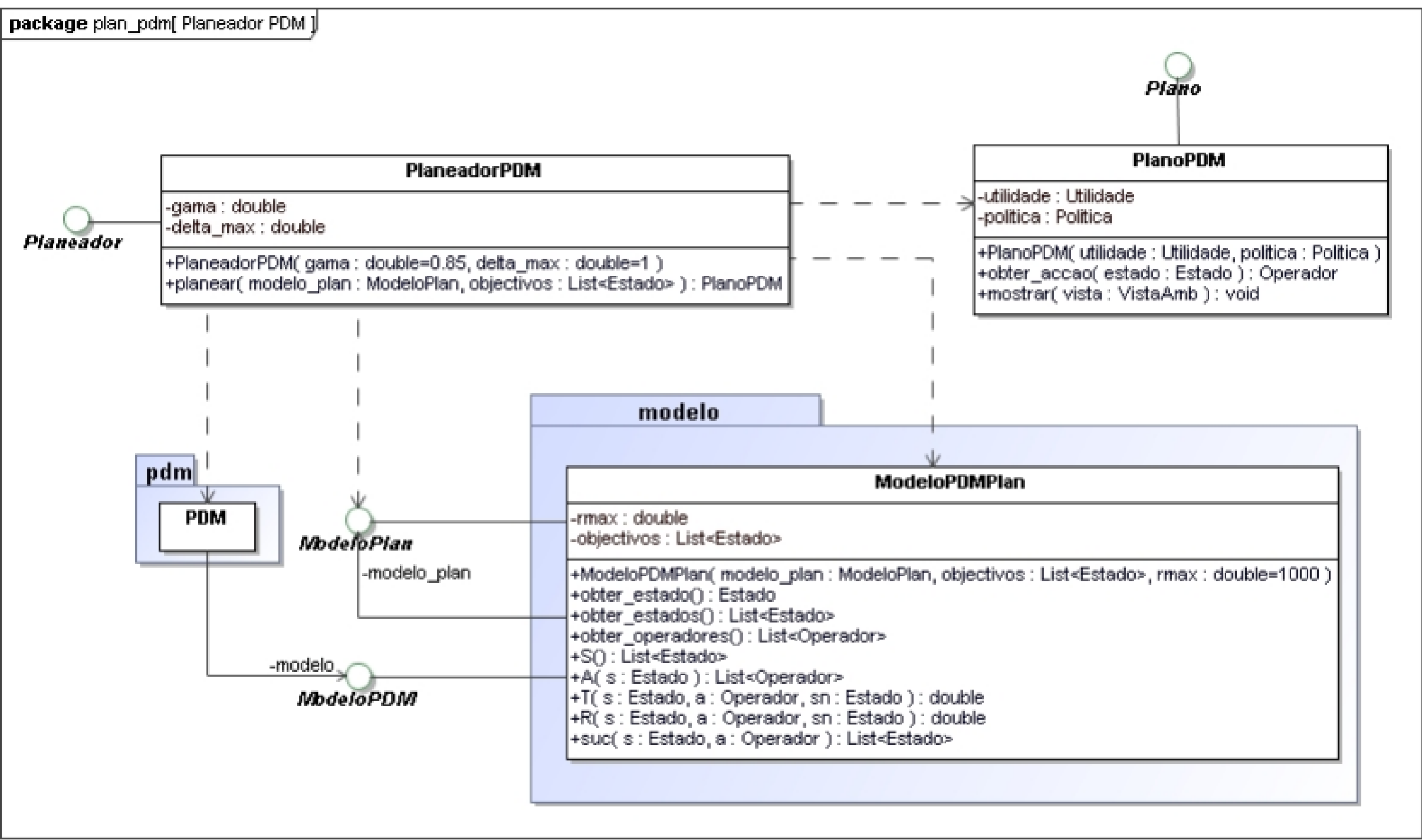


Figura 130 - Planeador com base em PDM

6.2. Implementação

- Classe EstadoAgente

```
1  from mod.estado import Estado
2
3  class EstadoAgente(Estado):
4
5      def __init__(self, posicao):
6          self.__posicao = posicao
7
8      @property
9      def posicao(self):
10         return self.__posicao
11
12     def id_valor(self):
13         return hash(self.posicao)
```

Figura 131 - Classe EstadoAgente

A classe **EstadoAgente** é uma **realização** da interface **Estado**. Esta classe representa um estado do agente no contexto de um problema de planeamento em que um agente se desloca num ambiente. No domínio do problema, o estado do agente é representado pela sua posição no ambiente, um **tuplo** (x,y), guardado na propriedade **posicao**. O **id_valor** desta classe é definido pelo **hash code** relativo à propriedade **posicao**.

- Interface ModeloPlan

```
1  from abc import ABC, abstractmethod
2
3  class ModeloPlan(ABC):
4
5      @abstractmethod
6      def obter_estado(self):
7          .....
8
9      @abstractmethod
10     def obter_estados(self):
11         .....
12
13     @abstractmethod
14     def obter_operadores(self):
15         .....
```

Figura 132 - Interface ModeloPlan

A Interface **ModeloPlan** representa o contrato funcional que será utilizado na implementação de **modelos de representação interna de problemas**, tanto para a fase de **planeamento**, como de **deliberação**. Um modelo, numa arquitetura deliberativa, representa a **memória do sistema**. Esta permite ter em consideração a dimensão temporal futuro, para além do passado e presente, como acontece nas arquiteturas reativas. Esta torna possível ao agente comportar-se da forma mais adequada para atingir os seus objetivos, pois consegue antecipar estados futuros através da simulação interna de situações e ações futuras.

- Classe ModeloMundo

```

1  from plan.modelo.modelo_plan import ModeloPlan
2  from .estado_agente import EstadoAgente
3  from .operador_mover import OperadorMover
4  from sae import Elemento, Direccao
5  from math import dist
6
7  class ModeloMundo(ModeloPlan):
8      def __init__(self):
9          self.__estado = None
10         self.__estados = []
11         self.__elementos = {}
12         self.__recolha = False
13         self.__operadores = [OperadorMover(self, direccao) for direccao in Direccao]
14
15     @property
16     def alterado(self):
17         return self.__recolha
18     @property
19     def elementos(self):
20         return self.__elementos
21
22     def obter_estado(self):
23         return self.__estado
24
25     def obter_estados(self):
26         return self.__estados
27
28     def obter_operadores(self):
29         return self.__operadores
30
31     def obter_elemento(self, estado):
32         return self.__elementos.get(estado.posicao)
33
34     def distancia(self, estado):
35         return dist(self.__estado.posicao, estado.posicao)
36
37     def atualizar(self, percepcao):
38         self.__estado = EstadoAgente(percepcao.posicao)
39         self.__estados = [EstadoAgente(posicao) for posicao in percepcao.posicoes]
40         self.__elementos = percepcao.elementos
41         self.__recolha = percepcao.recolha
42
43     def mostrar(self, vista):
44         for posicao, elemento in self.__elementos.items():
45             if elemento in [Elemento.ALVO, Elemento.OBSTACULO]:
46                 vista.mostrar_elemento(posicao, elemento)
47                 vista.marcar_posicao(self.__estado.posicao)

```

Figura 133 - Classe ModeloMundo

A classe **ModeloMundo** é uma **realização** da interface **ModeloPlan**. O modelo do mundo é a representação interna do ambiente (memória) que serve de suporte para as componentes do raciocínio prático (deliberação e planeamento). Este serve de repositório de informação, permite guardar o estado atual do agente, os estados válidos que o agente pode tomar, os operadores que o agente tem capacidade de realizar, os elementos do ambiente captados por uma perceção, a distância para um outro estado e se o modelo já foi alterado ou não. Para além de guardar informação, permite que esta seja atualizada a partir de perceções obtidas do ambiente ou por efeito dos mecanismos de raciocínio prático.

- Classe MecDelib

```
1  from sae import Elemento
2
3  class MecDelib:
4
5      def __init__(self, modelo_mundo):
6          self.__modelo_mundo = modelo_mundo
7
8      def deliberar(self):
9          estados = self.__modelo_mundo.obter_estados()
10         objetivos = [estado for estado in estados if self.__modelo_mundo.obter_elemento(estado) == Elemento.ALVO]
11         if objetivos:
12             objetivos.sort(key=self.__modelo_mundo.distancia)
13         return objetivos
```

Figura 134 - Classe MecDelib

A classe **MecDelib** representa uma das principais componentes do raciocínio prático, a deliberação ou raciocínio sobre fins, que tem por objetivo decidir o que fazer, mais concretamente, obter um conjunto de objetivos a concretizar.

Contém o método **__init__** que instancia um modelo de mundo e método **deliberar** que é responsável por realizar a deliberação. Partindo dos estados válido obtidos através do modelo do mundo, gera uma lista de objetivos que correspondem a estados cujo elemento é um alvo. Adicionalmente, esta lista é ordenada por distância, isto é, os objetivos que estão no início da lista são os que estão mais próximos do agente, utilizando a distância euclidiana.

- Classe ControloDelib

```

1  from sae import Controlo
2  from .modelo.modelo_mundo import ModeloMundo
3  from .mec_delib import MecDelib
4
5  class ControloDelib(Controlo):
6      def __init__(self, planeador):
7          self.__planeador = planeador
8          self.__objectivos = None
9          self.__modelo_mundo = ModeloMundo()
10         self.__mec_delib = MecDelib(self.__modelo_mundo)
11         self.__plano = None
12
13     def processar(self, percepcao):
14         self.__assimilar(percepcao)
15         if self.__reconsiderar():
16             self.__deliberar()
17             self.__planear()
18         accao = self.__executar()
19         self.__mostrar()
20         return accao
21
22     def __assimilar(self, percepcao):
23         self.__modelo_mundo.actualizar(percepcao)
24
25     def __reconsiderar(self):
26         return not self.__plano or self.__modelo_mundo.alterado
27
28     def __deliberar(self):
29         self.__objectivos = self.__mec_delib.deliberar()
30
31     def __planear(self):
32         self.__plano = self.__planeador.planear(self.__modelo_mundo, self.__objectivos)
33
34     def __executar(self):
35         if self.__plano:
36             operador = self.__plano.obter_accao(self.__modelo_mundo.obter_estado())
37             if operador:
38                 return operador.accao
39             else:
40                 self.__plano = None
41     def __mostrar(self):
42         self.vista.limpar()
43         self.__modelo_mundo.mostrar(self.vista)
44         if self.__plano:
45             self.__plano.mostrar(self.vista)
46         if self.__objectivos:
47             for objectivo in self.__objectivos:
48                 self.vista.marcar_posicao(objectivo.posicao)

```

Figura 135 - Classe ControloDelib

A classe **ControloDelib** é uma especialização da classe **Controlo** para o contexto de uma arquitetura deliberativa. Este permite organizar de forma **modular**, o modelo do mundo e os mecanismos de raciocínio prático envolvidos no problema. O controlo deliberativo realiza um processamento interno que, perante perceções obtidas do ambiente, gera ações a realizar. Este processamento corresponde a um ciclo de tomada de decisão e ação definido por: **1. Observar o mundo, 2. Atualizar o modelo do mundo, 3. Se necessário Reconsiderar: 4. Deliberar e 5. Planear**, e finalmente **6. Executar o plano de ação**. Um dos problemas que podem ocorrer durante o raciocínio prático é que o ambiente pode alterar-se a meio do processo, fazendo com que o resultado do raciocínio deixe de ser consistente com a situação do ambiente. É por esta razão que é crucial o passo 3. Reconsideração, para que tanto os objetivos, como os planos de ação, possam ser ajustados conformemente.

- Classe OperadorMover

```

1  from mod.operador import Operador
2  from sae import Accao
3  from .estado_agente import EstadoAgente
4  import math
5
6  class OperadorMover(Operador):
7
8      def __init__(self, modelo_mundo, direccao):
9          self.__modelo_mundo = modelo_mundo
10         self.__acao = Accao(direccao)
11
12         @property
13         def ang(self):
14             return self.__acao.direccao.value
15
16         @property
17         def acao(self):
18             return self.__acao
19
20         def aplicar(self, estado):
21             nova_posicao = self.__translacao(estado.posicao, self.acao.passo, self.ang)
22             novo_estado = EstadoAgente(nova_posicao)
23             if novo_estado in self.__modelo_mundo.obter_estados():
24                 return novo_estado
25
26         def __translacao(self, posicao, distancia, angulo):
27             x, y = posicao
28             dx = round(distancia * math.cos(angulo))
29             dy = -round(distancia * math.sin(angulo))
30             nova_posicao = x + dx, y + dy
31             return nova_posicao
32
33         def custo(self, estado, estado_suc):
34             return math.dist(estado.posicao, estado_suc.posicao)

```

Figura 136 - Classe OperadorMover

A classe **OperadorMover** é uma **realização** da interface **Operador**. Este operador é utilizado no problema de planeamento. Este representa a transformação do estado atual do agente para outro estado, com outra posição, na direção que é recebida no construtor.

Contém o método **aplicar** onde é realizada a transformação de estado aquando da aplicação do operador. Com o auxílio do método **translacao**, é calculada a nova posição e é retornado um novo estado, com essa posição, se esse novo estado estiver na lista de estados admissíveis do modelo do mundo. Contém o método privado **translacao** que foi criado para auxiliar ao método **aplicar**, com o objetivo de **maximizar a coesão e a modularidade** da implementação. Calcula a nova posição do agente partindo da posição atual, com a distância e ângulo recebidos como parâmetros do método e de acordo com a arquitetura disponibilizada. Finalmente, contém o método **custo**. O custo associado a este operador é definido pela distância euclidiana entre a posição de um estado e a posição do seu estado sucessor, calculada com recurso ao método **dist** do módulo **math** do Python.

- Interface Planeador

```
1  from abc import ABC, abstractmethod
2
3  class Planeador(ABC):
4
5      @abstractmethod
6      def planear(self, modelo_plan, objetivos):
7          .....
```

Figura 137 - Interface Planeador

A interface **Planeador** representa o contrato funcional a utilizar para implementar planeadores no contexto de planeamento automático. Este é realizado com base em métodos de raciocínio automático como a procura em espaço de estados ou processo de decisão de Markov. O planeador é responsável por gerar os planos de ação, com base no modelo do problema e nos objetivos que se pretendem atingir.

- Interface Plano

```
1  from abc import ABC, abstractmethod
2
3  class Plano(ABC):
4
5      @abstractmethod
6      def obter_accao(self, estado):
7          .....
8
9      @abstractmethod
10     def mostrar(self, vista):
11         .....
```

Figura 138 - Interface Plano

A interface **Plano** que representa o contrato funcional para a implementação de planos de ação, no contexto de problemas resolvidos com raciocínio prático, mais especificamente, raciocínio sobre meios (planeamento). Um plano de ação é o resultado deste tipo de raciocínio, ou seja, partindo dos objetivos do agente e das ações que o agente é capaz de realizar, são gerados planos de ação que vão determinar o comportamento do agente.

• Classe PlaneadorPEE

```

1  from .mod_prob.heur_dist import HeurDist
2  from ..planeador import Planeador
3  from pee.melhor_prim.procura_aa import ProcuraAA
4  from .mod_prob.problema_plan import ProblemaPlan
5  from .plano_pee import PlanoPEE
6
7  class PlaneadorPEE(Planeador):
8
9      def __init__(self):
10         self.__mec_pee = ProcuraAA()
11
12     def planear(self, modelo_plan, objetivos):
13         if objetivos:
14             estado_final = objetivos[0]
15             problema = ProblemaPlan(modelo_plan, estado_final)
16             heuristica = HeurDist(estado_final)
17
18             solucao = self.__mec_pee.procurar(problema, heuristica)
19             if solucao:
20                 return PlanoPEE(solucao)

```

Figura 139 - Classe PlaneadorPEE

A classe **PlaneadorPEE** é uma **realização** da interface **Planeador**. Esta classe é um planeador que será utilizado num problema que será resolvido com procura em espaço de estados. Neste caso, é necessário que o planeador defina o modelo do problema de planeamento, o mecanismo de procura e a heurística a utilizar, caso seja necessária.

Contém o método **__init__** que instancia o mecanismo de procura e o método **planear** que realiza o planeamento. Caso haja objetivos, define o estado final como um dos objetivos, neste caso, o primeiro da lista; define o problema, problema de planeamento; define a heurística, pois o mecanismo de procura é a Procura A* e procura uma solução. Se existir, retorna o plano de ação referente à solução.

• Classe PlanoPEE

```

1  from ..plano import Plano
2
3  class PlanoPEE(Plano):
4
5      def __init__(self, solucao):
6         self.__solucao = solucao
7         self.__passos = [passo for passo in self.__solucao]
8
9      def obter_accao(self, estado):
10         if self.__passos:
11             passo = self.__passos.pop(0)
12             if passo.estado == estado:
13                 return passo.operador
14
15     def mostrar(self, vista):
16         if self.__passos:
17             for passo in self.__passos:
18                 vista.mostrar_vector(passo.estado.posicao, passo.operador.ang)

```

Figura 140 - Classe PlanoPEE

A classe **PlanoPEE** é uma **realização** da interface **Plano**. Esta representa um plano num problema PEE. Partindo dos passos de uma solução, este obtém o operador que deve ser utilizado, para um determinado estado do problema.

- Classe HeurDist

```
1  from pee.melhor_prim.heuristica import Heuristica
2  from math import dist
3
4  class HeurDist(Heuristica):
5
6      def __init__(self, estado_final):
7          self.__estado_final = estado_final
8
9      def h(self, estado):
10         return dist(estado.posicao, self.__estado_final.posicao)
```

Figura 141 - Classe HeurDist

A classe **HeurDist** é uma **realização** da interface **Heuristica**. Esta classe representa uma heurística que, conhecendo o estado final, é definida pela distância euclidiana entre um determinado estado e o estado final.

- Classe ProblemaPlan

```
1  from mod.problema import Problema
2
3  class ProblemaPlan(Problema):
4
5      def __init__(self, modelo_plan, estado_final):
6          super().__init__(modelo_plan.obter_estado(), modelo_plan.obter_operadores())
7          self.__estado_final = estado_final
8
9      def objetivo(self, estado):
10         return estado == self.__estado_final
```

Figura 142 - Classe ProblemaPlan

A classe **ProblemaPlan** é uma **especialização** da classe **Problema** no contexto de um problema de planeamento. Esta recebe como parâmetros o modelo e o estado final e invoca o construtor da superclasse, fornecendo o estado atual do modelo como estado inicial e os seus operadores.

Contém o método **objetivo**. O objetivo é atingido quando o estado atual for igual ao estado final recebido.

- Interface ModeloPDM

```

1  from abc import ABC, abstractmethod
2
3  class ModeloPDM(ABC):
4
5      @abstractmethod
6      def S(self):
7          .....
8
9      @abstractmethod
10     def A(self, s):
11         .....
12
13     @abstractmethod
14     def T(self, s, a, sn):
15         .....
16
17     @abstractmethod
18     def R(self, s, a, sn):
19         .....
20
21     @abstractmethod
22     def suc(self, s, a):
23         .....

```

Figura 143 - Interface ModeloPDM

A interface **ModeloPDM** representa o contrato funcional a utilizar por classes que definam modelos que utilizem o processo de decisão de Markov. As classes que realizem esta interface têm de implementar métodos que ajudam a definir a representação do mundo.

- Classe MecUtil

```

1  class MecUtil:
2
3      def __init__(self, modelo, gama, delta_max):
4          self.__modelo = modelo
5          self.__delta_max = delta_max
6          self.__gama = gama
7
8      def utilidade(self):
9          S, A = self.__modelo.S, self.__modelo.A
10         U = {s: 0.0 for s in S()}
11
12         while True:
13             Uant = U.copy()
14             delta = 0
15
16             for s in S():
17                 U[s] = max([self.util_accao(s, a, Uant) for a in A(s)], default=0)
18                 delta = max(delta, abs(U[s] - Uant[s]))
19
20             if delta <= self.__delta_max:
21                 break
22
23         return U
24
25     def util_accao(self, s, a, U):
26         T, R, suc = self.__modelo.T, self.__modelo.R, self.__modelo.suc
27         somatorio = sum([T(s, a, sn) * (R(s, a, sn) + self.__gama * U[sn]) for sn in suc(s, a)])
28         return somatorio

```

Figura 144 - Classe MecUtil

A classe **MecUtil** representa um mecanismo baseado na utilidade.

Contém o método **utilidade** responsável por associar a cada estado a utilidade da sua ação com maior utilidade. Esta associação corresponde ao dicionário U, que é inicializado com utilidade zero para todos os estados. Em seguida, num ciclo while com break, de modo a replicar um ciclo do-while, que não existe na

linguagem Python, é criada uma shallow copy de U , para guardar os valores de utilidade anteriores e é iniciado o valor do delta a zero. Ainda dentro do ciclo while, dentro de um ciclo for que irá percorrer o espaço de estados, é calculada a utilidade desse estado, que corresponde à maior utilidade das suas ações, com recurso ao método `util_accao`, e é atualizado o delta, com o maior valor entre o delta anterior e o resultado do módulo da diferença entre a utilidade atual do estado e a utilidade anterior. Finalmente, é verificada a condição de saída do ciclo while, que realiza um break caso o delta atualizado seja menor ou igual ao delta máximo definido no construtor.

Contém o método `útil_accao` que calcula a utilidade de uma ação. A utilidade é calculada através de um somatório em que, para cada estado sucessor, é feito o produto entre a probabilidade de transição e a soma entre a recompensa imediata e a utilidade descontada pelo gama.

• Classe PDM

```

1  from .mec_util import MecUtil
2
3  class PDM:
4
5      def __init__(self, modelo, gama, delta_max):
6          self.__modelo = modelo
7          self.__mec_util = MecUtil(self.__modelo, gama, delta_max)
8
9      def politica(self, U):
10         S, A = self.__modelo.S, self.__modelo.A
11         pol = {}
12         for s in S():
13             if A(s):
14                 pol[s] = max(A(s), key=lambda a: self.__mec_util.util_accao(s, a, U))
15         return pol
16
17     def resolver(self):
18         U = self.__mec_util.utilidade()
19         pol = self.politica(U)
20         return U, pol

```

Figura 145 - Classe PDM

A classe PDM que representa um processo de decisão sequencial, o processo de decisão de Markov. Um processo de decisão sequencial é um processo em que a evolução entre estados ocorre por efeito de ações cujo o seu resultado pode não ser totalmente previsível, existindo incerteza no resultado da ação. As transições de estado associadas a este processo de decisão, ocorrem por efeito de ações com uma probabilidade associada, às quais pode estar associada uma recompensa, ou seja, um ganho ou perda associado a essa transição. No processo de decisão de Markov, a previsão dos estados seguintes só depende do estado presente e a representação do mundo é a seguinte:

- S , conjunto de estados
- $A(s)$, conjunto de ações possíveis no estado s pertencente a S
- $T(s,a,s')$, probabilidade de transição de s para s' através de a
- $R(s,a,s')$, recompensa esperada na transição de s para s' através de a
- γ , fator de desconto para recompensas diferidas no tempo

Um conceito importante é o conceito de Utilidade. É um efeito cumulativo da

evolução do sistema, representa a recompensa associada a cada estado, um valor finito positivo ou negativo, que reflete a utilidade de cada estado na perspectiva de alcançar um objetivo. Outro conceito importante é o conceito de Política. A Política Comportamental representa o comportamento do agente, que define qual a ação que deve ser realizada em cada estado, a estratégia de ação. Uma política pode ser determinista se para cada estado é conhecida a ação a realizar, ou pode ser não determinista, se para cada estado existir um conjunto de ações com uma probabilidade associada.

• Classe ModeloPDMPlan

```

1  from ...modelo.modelo_plan import ModeloPlan
2  from pdm.modelo.modelo_pdm import ModeloPDM
3
4  class ModeloPDMPlan(ModeloPlan, ModeloPDM):
5
6      def __init__(self, modelo_plan, objetivos, rmax = 1000.0):
7          self.__modelo_plan = modelo_plan
8          self.__objetivos = objetivos
9          self.__rmax = rmax
10
11         self.__transicoes = {(s, a): a.aplicar(s)
12                             for s in self.obter_estados()
13                             for a in self.obter_operadores()}
14
15     def obter_estado(self):
16         return self.__modelo_plan.obter_estado()
17
18     def obter_estados(self):
19         return self.__modelo_plan.obter_estados()
20
21     def obter_operadores(self):
22         return self.__modelo_plan.obter_operadores()
23
24     def S(self):
25         return self.__modelo_plan.obter_estados()
26
27     def A(self, s):
28         return self.__modelo_plan.obter_operadores() if s not in self.__objetivos else []
29
30     def T(self, s, a, sn):
31         #return 1 if self.suc(s, a) else 0
32         return 1 if self.__transicoes.get((s, a)) == sn else 0
33
34     def R(self, s, a, sn):
35         return self.__rmax if sn in self.__objetivos else -a.custo(s, sn)
36
37     def suc(self, s, a):
38         sn = self.__transicoes.get((s, a))
39         return [sn] if sn else []

```

Figura 146 - Classe ModeloPDMPlan

A classe **ModeloPDMPlan** que é uma **realização**, tanto da interface **ModeloPlan**, como da interface **ModeloPDM**. Esta classe representa o modelo do mundo num sistema em que são utilizados os PDM. Na nossa implementação, o espaço de estados é determinista, isto é, para cada estado, se tiver uma ação associada, a transição para o novo estado tem probabilidade 1, é certa. Adicionalmente, no atributo **self.__transicoes**, são pré-calculadas todas as transições possíveis, de maneira a tornar a implementação mais eficiente, evitando a necessidade de calcular transições sempre que se quer descobrir estados sucessores. Os métodos **obter_estado**, **obter_estados** e **obter_operadores** satisfazem a realização da interface **ModeloPlan** mas não necessitam de implementação adicional, por isso delegam a sua função para a instância **__modelo_plan**.

- Classe PlanoPDM

```

1  from ..plano import Plano
2
3  class PlanoPDM(Plano):
4
5      def __init__(self, utilidade, politica):
6          self.__utilidade = utilidade
7          self.__politica = politica
8
9      def obter_accao(self, estado):
10         if self.__politica:
11             return self.__politica.get(estado)
12
13     def mostrar(self, vista):
14         if self.__politica:
15             for estado, valor in self.__utilidade.items():
16                 vista.mostrar_valor_posicao(estado.posicao, valor)
17
18             for estado, accao in self.__politica.items():
19                 vista.mostrar_vector(estado.posicao, accao.ang)

```

Figura 147 - Classe PlanoPDM

A classe **PlanoPDM** é uma **realização** da interface **Plano** utilizada no contexto dos PDM. Partindo da política, que contem a estratégia de ação do agente, é possível obter o operador que se deve utilizar num determinado estado.

- Classe PlaneadorPDM

```

1  from ..planeador import Planeador
2  from .modelo.modelo_pdm_plan import ModeloPDMPPlan
3  from pdm.pdm import PDM
4  from .plano_pdm import PlanoPDM
5
6  class PlaneadorPDM(Planeador):
7
8      def __init__(self, gama = 0.85, delta_max = 1.0):
9          self.__gama = gama
10         self.__delta_max = delta_max
11
12     def planear(self, modelo_plan, objetivos):
13         if objetivos:
14             modelo_pdm = ModeloPDMPPlan(modelo_plan, objetivos)
15             pdm = PDM(modelo_pdm, self.__gama, self.__delta_max)
16             U, pol = pdm.resolver()
17             return PlanoPDM(U, pol)

```

Figura 148 - Classe PlaneadorPDM

A classe **PlaneadorPDM** é uma **realização** da interface **Planeador** utilizada no contexto de PDM. O objetivo desta classe é gerar um plano, partindo de um modelo de planeamento e de uma estratégia de ação (política).

Contém o método **planear** que é responsável por realizar o planeamento. Caso haja objetivos, é necessário definir o modelo do mundo e resolver os processos de decisão de Markov, de modo a obter a utilidade e a política.

7. REVISÃO DO PROJETO

O capítulo da revisão do projeto tem como objetivo a identificação, descrição e retificação de eventuais erros cometidos ao longo da realização do projeto. É uma fase de elevada importância e utilidade pois, não só é uma forma de melhor consolidar todos os conceitos lecionados ao longo do semestre, como também uma forma de demonstrar sentido crítico e evolução no conhecimento adquirido ao longo desta tarefa.

Com isto em mente, considero que o processo decorreu de forma bastante satisfatória. Não foram encontrados obstáculos nem dificuldades de relevância, tanto no que diz respeito ao entendimento da matéria teórica, como no que diz respeito às técnicas de programação apresentadas pelo docente. O conteúdo das entregas semanais não possui erros, nem apresentou grandes dificuldades na sua implementação.

8. CONCLUSÃO

A realização deste projeto proporcionou uma valiosa oportunidade para aplicar os conhecimentos adquiridos durante o semestre de maneira sistemática, consolidando-os gradualmente e aplicá-los diretamente na resolução dos desafios propostos, com poucas dificuldades.

A importância das boas práticas da engenharia de software ficou evidente, nomeadamente no que toca ao desenvolvimento de sistemas inteligentes e na gestão de sistemas complexos no geral.

Todas as partes do projeto apresentadas neste relatório foram concluídas com sucesso, comprovando a importância do trabalho semanal realizado ao longo deste período. Esta consistência e a natureza iterativa deste projeto promoveram uma melhor compreensão dos conceitos e uma autonomia muito valiosa, que permitiu sempre avançar para próxima fase, sem que nunca houvesse lacunas no conhecimento.

9. BIBLIOGRAFIA

[Morgado, 2024]
L. Morgado, IASA - Introdução a Inteligência Artificial, 2024
[Morgado, 2024]
L. Morgado, IASA - Introdução a Engenharia de Software, 2024
[Morgado, 2024]
L. Morgado, IASA - Arquitetura de agentes reactivos, 2024
[Morgado, 2024]
L. Morgado, IASA - Procura em espaços de estados, 2024
[Morgado, 2024]
L. Morgado, IASA - Arquitetura de agentes deliberativos, 2024
[Morgado, 2024]
L. Morgado, IASA - Raciocínio Automático, 2024
[Morgado, 2024]
L. Morgado, IASA - Processos de Decisão Sequencial, 2024
[Morgado, 2024]
L. Morgado, IASA - Planeamento Automático com Base em PDM, 2024
[Morgado, 2024]
L. Morgado, IASA - Projeto - Parte 1, 2024
[Morgado, 2024]
L. Morgado, IASA - Projeto - Parte 2, 2024
[Morgado, 2024]
L. Morgado, IASA - Projeto - Parte 3, 2024
[Morgado, 2024]
L. Morgado, IASA - Projeto - Parte 4, 2024